



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática – Ingeniería del Software

Explorando la complejidad y el caos en series temporales económicas

**Realizado por
Francisco Rodríguez-Carretero Roldán**

**Dirigido por
Juan Vicente Gutiérrez Santacreu**

**Departamento
Departamento de Matemática Aplicada I**

Sevilla, Julio de 2026

Resumen

Este Trabajo Fin de Grado desarrolla un procedimiento computacional para analizar y predecir la volatilidad de datos financieros intradía. El trabajo combina una parte metodológica, una aplicación empírica sobre Bitcoin y un prototipo web que permite utilizar una parte de los modelos obtenidos.

El objetivo principal es construir un flujo de análisis reproducible, desde la obtención y preparación de los datos hasta la comparación de modelos predictivos. Para ello, primero se utiliza un caso sintético conocido, que sirve como control del procedimiento en un entorno donde la estructura de los datos está definida de antemano. Después, el mismo enfoque se aplica a datos reales de Bitcoin con frecuencia de cinco minutos, a partir de los cuales se construye una medida de volatilidad futura a corto plazo.

El trabajo no busca predecir la dirección del precio ni generar señales de compra o venta. La variable estudiada representa intensidad de movimiento, no rentabilidad esperada. Por ello, las salidas del sistema deben interpretarse como estimaciones de volatilidad, y no como recomendaciones de inversión.

La metodología desarrollada incluye preparación y validación de datos, construcción de variables, caracterización de la serie, comparación con modelos de referencia y evaluación predictiva fuera de muestra. Además, se incorpora una comprobación sintética para distinguir entre el comportamiento del procedimiento en un sistema controlado y su aplicación a una serie financiera real, donde existen ruido, persistencia y cambios de comportamiento.

Como cierre práctico, el proyecto integra los modelos y artefactos generados en un MVP web desarrollado con Streamlit. Esta aplicación permite cargar datos recientes o de ejemplo, validar su calidad, calcular las variables necesarias y mostrar predicciones de volatilidad junto con información de diagnóstico. El MVP no sustituye al análisis completo, sino que actúa como demostración funcional de una parte del trabajo realizado.

La aportación principal del TFG consiste en conectar metodología, análisis empírico e implementación software dentro de un mismo sistema reproducible. El resultado es una herramienta que permite estudiar la volatilidad de Bitcoin con un enfoque prudente, comparando distintos modelos y separando claramente lo que puede afirmarse a partir de los datos de aquello que requiere cautela interpretativa.

Agradecimientos

A mis padres y a mi familia por su apoyo y comprensión.

A mi tutor, Juan Vicente Gutiérrez Santacreu, por su ayuda y orientación.

Y a mis amigos por escucharme y darme su opinión.

Índice general

Índice general	III
Índice de cuadros	VI
Índice de figuras	VII
Índice de código	VIII
1 Introducción	1
1.1 Contexto y motivación	1
1.2 Planteamiento del problema	2
1.3 Naturaleza del trabajo realizado	2
1.4 Estructura de la memoria	3
2 Definición de objetivos	5
2.1 Objetivo general	5
2.2 Objetivos específicos	5
2.3 Alcance del trabajo	6
2.4 Limitaciones del trabajo	6
2.5 Aportación principal del TFG	7
3 Antecedentes y marco teórico	8
3.1 Conceptos básicos para un lector no especialista	8
3.2 Complejidad, no linealidad y caos determinista	11
3.3 Indicios de caos en series económicas	12
3.4 Sistemas dinámicos y mapa logístico	14
3.5 Series temporales financieras y volatilidad realizada	16
3.6 Caracterización lineal de series temporales	18
3.7 Estacionariedad, heterocedasticidad y no linealidad	21
3.8 Reconstrucción del espacio de estados	23
3.9 Cuantificación de dinámicas complejas	25
3.10 Predicción local y modelos de referencia	27
3.11 Modelo HAR-logRV para volatilidad realizada	30
3.12 Relación con la tesis de referencia	31
3.13 Aportación realizada respecto a los antecedentes	33
4 Planificación, metodología de trabajo y costes	34
4.1 Organización general del proyecto	34
4.2 Fases principales del trabajo	34
4.3 Planificación temporal	35
4.4 Distribución del esfuerzo	35

4.5	Herramientas de seguimiento y desarrollo	35
4.6	Estimación de costes	36
4.7	Desviaciones respecto a la planificación inicial	36
5	Análisis de requisitos	37
5.1	Requisitos generales del sistema	37
5.2	Requisitos conceptuales y de comunicación	37
5.3	Requisitos de datos	38
5.4	Requisitos metodológicos	38
5.5	Requisitos de la comprobación sintética	38
5.6	Requisitos funcionales del procedimiento de análisis	39
5.7	Requisitos funcionales del MVP web	39
5.8	Requisitos no funcionales	40
5.9	Criterios de validación	40
6	Diseño de la solución	41
6.1	Visión general de la solución propuesta	41
6.2	Arquitectura global del sistema	41
6.3	Diseño del procedimiento metodológico	42
6.4	Diseño de la comprobación sintética	42
6.5	Diseño del flujo de datos y variables principales	43
6.6	Diseño experimental y separación temporal	44
6.7	Diseño de la arquitectura del MVP web	44
7	Implementación del procedimiento de análisis	45
7.1	Estructura general del procedimiento	45
7.2	Entorno de desarrollo y herramientas utilizadas	45
7.3	Obtención y validación inicial de los datos	46
7.4	Construcción de variables y medidas de volatilidad	46
7.5	Visualización inicial y estadísticos descriptivos	47
7.6	Análisis de estacionariedad	48
7.7	Análisis de autocorrelación y espectro	48
7.8	Gráficos de recurrencia	48
7.9	Filtrado lineal mediante modelos autorregresivos	49
7.10	Contrastes de no linealidad	49
7.11	Reconstrucción del espacio de estados	50
7.12	Cuantificación de la dinámica reconstruida	51
7.13	Contrastes con barajado y datos subrogados	52
7.14	Predicción local de volatilidad	52
7.15	Experimentos de robustez y configuraciones alternativas	53
7.16	Comprobación sintética con mapa logístico	54
7.17	Modelo HAR-logRV y exportación al MVP	55
7.18	Cierre técnico del procedimiento y artefactos generados	55
8	Implementación del MVP web	57
8.1	Objetivo del prototipo web	57
8.2	Funcionalidades implementadas	57
8.3	Arquitectura de la aplicación	58
8.4	Integración del modelo validado	59
8.5	Interfaz de usuario	60
8.6	Flujo de uso de la aplicación	61
8.7	Limitaciones del MVP	61

9	Manual de usuario	63
9.1	Objetivo de la herramienta	63
9.2	Requisitos para la ejecución	64
9.3	Puesta en marcha del sistema	64
9.4	Ejecución del procedimiento de análisis	65
9.5	Uso del MVP web	66
9.6	Interpretación básica de las salidas	67
9.7	Generación de figuras, tablas e informes	68
10	Conclusiones y desarrollos futuros	69
10.1	Conclusiones generales	69
10.2	Conclusiones sobre la metodología desarrollada	69
10.3	Conclusiones sobre el mapa logístico	69
10.4	Conclusiones sobre la aplicación a Bitcoin	69
10.5	Conclusiones sobre la utilidad predictiva	69
10.6	Conclusiones sobre el MVP web	69
10.7	Limitaciones finales del trabajo	69
10.8	Desarrollos futuros	69
A	Glosario de conceptos técnicos	70
B	Uso de la inteligencia artificial	73
	Referencias	74

Índice de cuadros

3.1	Relación entre la tesis de referencia y la adaptación realizada en este trabajo. . . .	32
4.1	Distribución inicial de horas planificadas.	35
4.2	Comparación entre esfuerzo planificado y esfuerzo real registrado.	35
4.3	Estimación inicial de costes del proyecto.	36
6.1	Bloques principales de la arquitectura global del proyecto.	42
6.2	Variables principales consideradas en el diseño del sistema.	43

Índice de figuras

Índice de código

Introducción

1.1– Contexto y motivación

Una parte importante del conocimiento científico ha consistido en intentar separar lo regular de lo aleatorio. Cuando un fenómeno puede describirse mediante relaciones estables, parece razonable pensar que puede explicarse y, al menos hasta cierto punto, predecirse. Cuando esa regularidad no aparece de forma clara, se recurre con frecuencia al azar, al ruido o a la falta de información. Sin embargo, muchos sistemas reales no encajan cómodamente en ninguno de esos dos extremos: no son completamente previsibles, pero tampoco se comportan como puro desorden.

Esta tensión entre orden, azar y predicción aparece en numerosos ámbitos. El clima, los sistemas biológicos, la actividad económica o los mercados financieros muestran comportamientos que combinan repetición, cambios bruscos, sensibilidad al contexto y episodios difíciles de anticipar. En ellos puede haber patrones parciales, pero esos patrones no garantizan una predicción exacta. Esta idea es especialmente relevante cuando se trabaja con datos reales: encontrar una regularidad no implica haber descubierto una ley completa, y obtener una mejora predictiva no significa comprender todo el mecanismo que genera los datos.

Los mercados financieros constituyen un ejemplo actual de este problema. Los precios incorporan información de forma continua y reaccionan a expectativas, liquidez, noticias, decisiones de los participantes y movimientos agregados del mercado. Como consecuencia, su comportamiento resulta difícil de modelar. A veces parece haber continuidad y memoria; otras veces aparecen saltos, cambios de intensidad o episodios extremos. Esta mezcla convierte al análisis financiero en un terreno adecuado para estudiar hasta qué punto los datos contienen estructura aprovechable y hasta qué punto esa estructura queda limitada por el ruido y la incertidumbre.

Bitcoin añade un interés particular dentro de este contexto. A diferencia de otros mercados con horarios de apertura y cierre, cotiza de forma continua y genera datos intradía durante todos los días del año. Además, su comportamiento suele presentar variaciones intensas en periodos cortos, lo que lo convierte en un activo adecuado para estudiar la variabilidad del mercado. El objetivo de este trabajo no es anticipar si el precio subirá o bajará, sino analizar la intensidad esperada de sus movimientos a corto plazo.

Desde el punto de vista de la Ingeniería de Software, este problema no se reduce a elegir un modelo y calcular una predicción. Es necesario construir un procedimiento completo: obtener datos, validarlos, transformarlos, comparar alternativas, guardar resultados, documentar decisiones

y ofrecer una forma de uso comprensible. La dificultad no está solo en el cálculo, sino en diseñar un flujo reproducible y trazable que permita saber de dónde sale cada resultado y qué alcance tiene.

Este Trabajo Fin de Grado se sitúa en ese punto intermedio. Por un lado, estudia un problema financiero real: la predicción de la variabilidad futura de Bitcoin a partir de datos intradía. Por otro lado, desarrolla una solución software organizada, capaz de ejecutar un procedimiento de análisis, generar artefactos reutilizables y trasladar una parte del trabajo a un prototipo web. La motivación principal es, por tanto, doble: analizar si los datos contienen información útil sobre el comportamiento futuro inmediato y construir una herramienta que permita hacerlo de forma ordenada y verificable.

1.2– Planteamiento del problema

El problema abordado consiste en estudiar la variabilidad futura de Bitcoin a corto plazo a partir de datos de mercado registrados cada cinco minutos. En lugar de trabajar directamente con el precio como variable final de interés, el trabajo se centra en medir la intensidad de los movimientos recientes y estimar la intensidad de los movimientos posteriores. Esta elección permite separar el estudio de la volatilidad del problema, mucho más ambicioso, de predecir la dirección del precio.

La pregunta principal puede formularse de forma sencilla: dado el comportamiento reciente del mercado, ¿es posible estimar con cierta utilidad la variabilidad de la próxima hora? La respuesta no puede reducirse a aplicar un único modelo. En datos financieros reales es fácil obtener patrones aparentes que después no se mantienen, o mejoras pequeñas que no justifican una interpretación fuerte. Por ello, el trabajo se plantea como una comparación entre distintas formas de resumir el pasado y distintas formas de generar predicciones.

El estudio incorpora también un caso sintético conocido. Su función es actuar como punto de control: antes de aplicar el procedimiento a una serie financiera real, conviene comprobar cómo se comporta ante datos generados por una regla conocida. Esta parte no pretende representar el mercado de Bitcoin, sino ayudar a interpretar después las diferencias entre un sistema controlado y un activo financiero real.

La aplicación a Bitcoin introduce dificultades propias. Los datos financieros son ruidosos, cambian con el tiempo y pueden estar afectados por eventos externos que no aparecen directamente en la base de datos. Por ese motivo, el análisis debe ser prudente. El trabajo no busca demostrar que Bitcoin siga una regla determinista simple ni afirmar que sus movimientos puedan anticiparse de forma precisa. El objetivo es más limitado y, precisamente por ello, más defendible: construir un procedimiento que permita detectar estructura útil, compararla con referencias razonables y evaluar su capacidad predictiva fuera de muestra.

El problema tiene también una dimensión práctica. Una vez desarrollado el procedimiento, una parte de sus resultados se integra en un MVP web. Esta aplicación permite cargar datos recientes o de ejemplo, realizar validaciones básicas, calcular las variables necesarias y mostrar predicciones de volatilidad. El prototipo no sustituye al análisis completo, pero permite comprobar que el trabajo no queda únicamente como un conjunto de scripts experimentales, sino que puede organizarse como una herramienta utilizable.

1.3– Naturaleza del trabajo realizado

El trabajo combina tres dimensiones: metodológica, experimental y software.

La dimensión metodológica consiste en definir un flujo de análisis para datos financieros ordenados en el tiempo. Este flujo incluye la preparación de los datos, la construcción de variables, la caracterización del comportamiento observado, la comparación de modelos y la interpretación de los resultados. La finalidad no es aplicar técnicas de forma aislada, sino establecer una secuencia coherente que permita pasar desde datos brutos hasta conclusiones justificadas.

La dimensión experimental aparece en dos contextos. El primero es sintético y controlado. En él se utiliza una regla conocida para comprobar que el procedimiento responde de forma razonable cuando los datos proceden de un sistema con estructura definida. El segundo es empírico y se centra en Bitcoin. En este caso, el procedimiento se enfrenta a datos reales, con todas las dificultades que eso implica: ruido, cambios en el comportamiento del mercado y ausencia de una regla generadora observable.

La dimensión software se materializa en la organización del proyecto y en el desarrollo del MVP web. El procedimiento técnico genera datos procesados, tablas, figuras, métricas y modelos exportables. El MVP consume una parte de esos artefactos y ofrece una interfaz para ejecutar predicciones de volatilidad sobre datos cargados por el usuario o descargados desde una fuente externa. Esta separación entre análisis experimental y herramienta web permite mantener el estudio reproducible y, al mismo tiempo, mostrar una aplicación práctica de sus resultados.

El trabajo se ha diseñado con una restricción interpretativa clara: no convertir indicios en afirmaciones más fuertes de lo que permiten los datos. En una serie financiera real, una mejora predictiva no implica necesariamente una explicación completa del fenómeno. Del mismo modo, la presencia de patrones o irregularidades no permite concluir por sí sola que exista una estructura profunda y estable. Por ello, la memoria insiste en comparar modelos, usar controles y distinguir entre resultados observados, interpretaciones plausibles y conclusiones no defendibles.

Esta orientación encaja con el carácter del TFG. No se trata solo de estudiar un problema financiero, ni solo de construir una aplicación, sino de integrar ambas partes dentro de un proyecto de Ingeniería de Software. La aportación está en diseñar un sistema reproducible, aplicar una metodología de análisis a datos reales y desarrollar una herramienta que permita utilizar una parte de lo construido.

1.4– Estructura de la memoria

La memoria se organiza siguiendo la evolución natural del trabajo realizado.

Tras esta introducción, el Capítulo 2 define el objetivo general del proyecto y lo descompone en objetivos específicos. También delimita el alcance del trabajo, sus principales limitaciones y la aportación esperada desde el punto de vista metodológico y software.

El Capítulo 3 presenta los antecedentes y el marco teórico necesarios para comprender el resto de la memoria. En él se introducen formalmente los conceptos técnicos, la finalidad de ese capítulo es concentrar la teoría que no se desarrolla en la introducción ni en los objetivos.

El Capítulo 4 describe la planificación del proyecto, la metodología de trabajo seguida y la estimación de costes. Esta parte sitúa el desarrollo dentro de un contexto propio de Ingeniería de Software, atendiendo a fases de trabajo, esfuerzo dedicado, herramientas utilizadas y organización temporal.

El Capítulo 5 recoge los requisitos del sistema. Estos requisitos no se limitan al MVP web, sino que incluyen también condiciones sobre datos, metodología, separación temporal, comunicación de resultados, reproducibilidad y validación.

El Capítulo 6 presenta el diseño de la solución. En él se explica la arquitectura general del sistema, la separación entre el procedimiento de análisis y la aplicación web, el flujo de datos y las decisiones estructurales que permiten mantener el proyecto organizado.

El Capítulo 7 describe la implementación del procedimiento de análisis. Se detallan las fases técnicas ejecutadas, desde la preparación de los datos hasta la comparación predictiva y la exportación de artefactos. Este capítulo se centra en qué se implementa y cómo se organiza la ejecución.

El Capítulo ?? presenta los resultados y su discusión. Primero se analiza el comportamiento del procedimiento en el caso sintético y después se estudia la aplicación empírica a Bitcoin. En esta parte se comparan modelos, se interpretan los indicadores obtenidos y se separan las conclusiones defendibles de las interpretaciones que requieren cautela.

El Capítulo 8 describe la implementación del MVP web. Se explican sus funcionalidades principales, su arquitectura, la integración de modelos exportados, la interfaz de usuario y las limitaciones propias del prototipo.

El Capítulo 9 actúa como manual de usuario. En él se explica cómo ejecutar el sistema, cómo utilizar el MVP web, cómo interpretar sus salidas principales y cómo generar figuras, tablas e informes.

Finalmente, el Capítulo 10 resume las conclusiones generales del trabajo y plantea posibles desarrollos futuros. La memoria se completa con anexos y bibliografía, donde se recogen detalles complementarios, referencias técnicas y explicaciones adicionales que no forman parte del flujo principal de la memoria.

Definición de objetivos

2.1– Objetivo general

El objetivo general de este Trabajo de Fin de Grado es diseñar, implementar y evaluar un sistema software para analizar y predecir la volatilidad de datos financieros intradía. El trabajo no pretende construir una herramienta de inversión ni anticipar la dirección del precio, sino desarrollar un procedimiento reproducible que permita transformar datos de mercado en información útil sobre la intensidad esperada de los movimientos futuros.

El proyecto combina tres elementos. En primer lugar, un procedimiento de análisis que parte de datos brutos, los valida, construye variables de trabajo y compara distintos modelos predictivos. En segundo lugar, una comprobación con un caso sintético conocido, que permite observar el comportamiento del procedimiento en un entorno controlado antes de aplicarlo a datos reales. En tercer lugar, una aplicación web que utiliza una parte de los artefactos generados durante el estudio y ofrece una interfaz sencilla para cargar datos, ejecutar predicciones y visualizar resultados.

La aplicación empírica se centra en Bitcoin, utilizando datos de mercado con frecuencia de cinco minutos. A partir de ellos se estudia la volatilidad futura a corto plazo. Esta elección permite abordar un problema financiero real sin convertir el trabajo en una estrategia de trading. El interés principal está en la construcción del sistema, la comparación entre modelos y la interpretación prudente de los resultados obtenidos.

2.2– Objetivos específicos

Para alcanzar el objetivo general se plantean los siguientes objetivos específicos:

1. Establecer el marco conceptual necesario para interpretar el problema, diferenciando entre predicción de precio, predicción de volatilidad, análisis de patrones y límites de interpretación en datos financieros reales.
2. Diseñar un procedimiento reproducible de análisis de datos financieros intradía, desde la preparación de los datos hasta la comparación de modelos predictivos y la generación de resultados documentables.

3. Comprobar el comportamiento del procedimiento en un caso sintético conocido, de forma que exista un punto de referencia controlado antes de aplicarlo a datos financieros reales.
4. Aplicar el procedimiento a Bitcoin, construyendo una medida de volatilidad futura a corto plazo y evaluando si los datos recientes contienen información útil para estimarla.
5. Comparar modelos predictivos de distinta complejidad bajo una separación temporal adecuada, evitando que la evaluación dependa de una única técnica o de una referencia demasiado débil.
6. Implementar un MVP web que integre una parte de los resultados del estudio y permita cargar datos, calcular variables, obtener predicciones de volatilidad y visualizar las salidas principales de forma comprensible.

2.3– Alcance del trabajo

El alcance del trabajo incluye el desarrollo de un procedimiento completo de análisis y predicción de volatilidad a corto plazo. Este procedimiento cubre la obtención de datos, su validación, la construcción de variables, la comparación de modelos, la generación de resultados y la exportación de artefactos reutilizables.

El estudio se aplica a dos contextos. El primero es un caso sintético conocido, usado como control del procedimiento. El segundo es una aplicación empírica sobre Bitcoin, utilizando datos intradía del par BTCUSD. Otros activos, otras frecuencias y otros horizontes de predicción quedan fuera del alcance principal y se consideran extensiones futuras.

El trabajo incluye también el desarrollo de un MVP web. Este prototipo permite utilizar una parte del sistema en un entorno interactivo, pero no reproduce todo el análisis experimental. Su finalidad es demostrar que los modelos y artefactos generados pueden integrarse en una herramienta software utilizable.

Quedan fuera del alcance del proyecto la construcción de una estrategia de inversión, la ejecución de operaciones reales, la optimización de carteras, la gestión de riesgo financiero y la recomendación de compra o venta de activos. El sistema se limita a estimar volatilidad futura y a mostrar los resultados de forma interpretable.

2.4– Limitaciones del trabajo

El trabajo presenta limitaciones que deben tenerse en cuenta desde el inicio. La primera es que los datos financieros reales son ruidosos y cambian con el tiempo. Por tanto, cualquier patrón observado debe interpretarse con cautela y compararse con modelos de referencia antes de extraer conclusiones.

La segunda limitación afecta a la predicción. El sistema estima volatilidad futura, no rentabilidad ni dirección del precio. Una predicción de mayor volatilidad no indica por sí sola si el precio subirá o bajará. En consecuencia, los resultados no deben entenderse como señales operativas de mercado.

La tercera limitación está relacionada con el alcance empírico. La aplicación principal se realiza sobre Bitcoin, con frecuencia de cinco minutos y horizonte de una hora. Los resultados obtenidos no pueden generalizarse automáticamente a otros activos, periodos o frecuencias sin repetir el análisis.

La cuarta limitación corresponde al MVP web. El prototipo demuestra la integración software de una parte del estudio, pero no incorpora todas las características de una plataforma profesional. No incluye ejecución automática de operaciones, gestión de usuarios, despliegue productivo, monitorización continua ni estimación formal de incertidumbre.

Por último, el trabajo distingue entre indicios y conclusiones. Que un modelo mejore a otro en una muestra concreta no implica que exista una explicación completa del comportamiento del mercado. La memoria debe mantener esta diferencia durante todo el análisis.

2.5– Aportación principal del TFG

La aportación principal del TFG consiste en integrar análisis de datos financieros, comparación predictiva e implementación software dentro de un mismo sistema reproducible. El trabajo no se limita a probar un modelo aislado, sino que diseña una cadena completa desde los datos brutos hasta una aplicación web funcional.

Desde el punto de vista metodológico, la aportación está en construir un procedimiento ordenado para estudiar la volatilidad futura a corto plazo. Este procedimiento incluye un caso sintético de control, una aplicación empírica sobre Bitcoin y una comparación entre modelos con distinta complejidad.

Desde el punto de vista de Ingeniería de Software, la aportación consiste en organizar el proyecto en componentes diferenciados: repositorio de análisis, artefactos exportables, MVP web, pruebas y documentación. Esta separación permite mantener trazabilidad entre los resultados generados y la herramienta que los utiliza.

En conjunto, el TFG aporta una solución que conecta investigación aplicada y desarrollo software. El resultado no es una plataforma financiera profesional, sino un sistema académico reproducible que permite analizar la volatilidad de Bitcoin, comparar modelos y mostrar una parte de los resultados mediante una interfaz web.

Antecedentes y marco teórico

3.1– Conceptos básicos para un lector no especialista

Una serie temporal es una sucesión de observaciones ordenadas según una variable temporal. Si se denota por x_t el valor observado en el instante t , una serie de longitud T puede escribirse como

$$\{x_t\}_{t=1}^T = x_1, x_2, \dots, x_T. \quad (3.1)$$

El subíndice t indica la posición de cada observación dentro de la secuencia. Cuando la serie está muestreada de forma regular, dos observaciones consecutivas están separadas por un intervalo fijo Δt . En ese caso, el tiempo físico asociado a la observación t puede expresarse como

$$s_t = s_0 + t\Delta t, \quad (3.2)$$

donde s_0 representa el instante inicial. En este trabajo los datos proceden de velas de cinco minutos, por lo que $\Delta t = 5$ minutos. Así, 12 observaciones corresponden a una hora, 288 observaciones a un día y 2016 observaciones a una semana.

El orden de los datos forma parte de la información de una serie temporal. En una tabla de observaciones independientes, intercambiar dos filas puede no alterar el análisis. En una serie temporal, esa operación modifica la estructura del problema, ya que el valor observado en un instante puede depender de valores anteriores. Esta dependencia se representa mediante retardos. Para un entero positivo k , el valor x_{t-k} es la observación situada k pasos antes de x_t . Por ejemplo, con datos de cinco minutos, x_{t-12} corresponde al valor observado una hora antes de x_t .

El conjunto de información disponible en un instante t puede escribirse como

$$\mathcal{F}_t = \{x_t, x_{t-1}, x_{t-2}, \dots\}. \quad (3.3)$$

Esta notación permite separar la información pasada y presente de la información futura. Cualquier predicción realizada en t debe construirse únicamente a partir de $\mathcal{F}_t$. Si durante el ajuste o la evaluación de un modelo se utiliza información posterior a t , aparece fuga de información. Este problema conduce a estimaciones artificialmente favorables, porque el modelo se beneficia de datos que no estarían disponibles en una situación real de predicción.

En el análisis de series temporales conviene distinguir entre observaciones directas y variables derivadas. En una serie financiera pueden observarse precios de apertura, máximo, mínimo, cierre, volumen o número de operaciones. A partir de estas magnitudes se calculan otras variables

diseñadas para estudiar propiedades concretas. Si P_t representa un precio observado, una variable derivada puede definirse mediante una transformación

$$z_t = g(P_t, P_{t-1}, \dots, P_{t-h}), \quad (3.4)$$

donde g es una función elegida por el investigador. Los retornos, la volatilidad realizada, las medias móviles o las variables retardadas son ejemplos de este tipo de construcción. La elección de la variable analizada forma parte de la metodología, ya que distintas transformaciones resaltan propiedades distintas de la serie.

También debe distinguirse entre variables explicativas y variable objetivo. Una variable explicativa disponible en t se construye con información pasada o presente. Una variable objetivo asociada a un horizonte futuro utiliza observaciones posteriores y sirve para entrenar o evaluar el modelo. De forma general, si h es el horizonte de predicción, puede escribirse

$$y_t^{(h)} = q(x_{t+1}, x_{t+2}, \dots, x_{t+h}), \quad (3.5)$$

donde q resume el comportamiento futuro que se desea estimar. En este TFG el horizonte principal es de 12 velas, equivalente a una hora.

El análisis de una serie temporal suele organizarse en tres tareas. La primera es la descripción de la serie. Consiste en representar sus valores y resumir propiedades básicas como nivel medio, dispersión, asimetría, colas y episodios extremos. Dos magnitudes elementales son la media muestral,

$$\bar{x} = \frac{1}{T} \sum_{t=1}^T x_t, \quad (3.6)$$

y la varianza muestral,

$$s^2 = \frac{1}{T-1} \sum_{t=1}^T (x_t - \bar{x})^2. \quad (3.7)$$

Estas medidas ofrecen una primera caracterización, aunque resultan insuficientes cuando la serie cambia de comportamiento a lo largo del tiempo o presenta periodos de variabilidad muy distintos.

La segunda tarea es la modelización. Un modelo proporciona una representación simplificada de la relación entre observaciones. En forma general, un modelo temporal puede escribirse como

$$x_t = f(x_{t-1}, x_{t-2}, \dots, x_{t-p}) + \varepsilon_t, \quad (3.8)$$

donde f describe la parte sistemática de la relación temporal, p es el número de retardos utilizados y ε_t recoge la parte no explicada por el modelo. Esta formulación no exige que el modelo reproduzca todos los mecanismos reales que generan los datos. Su valor depende de su capacidad para capturar regularidades medibles y contrastables.

La tercera tarea es la predicción. Si se desea estimar un valor futuro $y_t^{(h)}$ usando la información disponible en t , una predicción puede representarse como

$$\hat{y}_t^{(h)} = M(\mathcal{F}_t), \quad (3.9)$$

donde M es el modelo utilizado. La calidad de la predicción se evalúa comparando el valor estimado con el valor observado posteriormente. El error asociado se define como

$$e_t^{(h)} = y_t^{(h)} - \hat{y}_t^{(h)}. \quad (3.10)$$

A partir de estos errores se calculan métricas de evaluación que permiten comparar modelos sobre datos no utilizados durante el ajuste.

La dependencia temporal aparece cuando las observaciones de una serie contienen información sobre otras observaciones de la misma serie. Una forma sencilla de expresarlo es mediante la esperanza condicional:

$$\mathbb{E}[x_t \mid x_{t-1}, x_{t-2}, \dots] \neq \mathbb{E}[x_t]. \quad (3.11)$$

Esta desigualdad indica que conocer el pasado modifica la estimación esperada del presente. En series financieras, la dependencia temporal puede ser débil en los retornos y mucho más visible en medidas de variabilidad. Por esta razón, el análisis de volatilidad suele ser más informativo que el análisis directo de la dirección del precio.

La separación entre señal y ruido ayuda a interpretar los modelos. Una forma esquemática de escribir una observación es

$$x_t = s_t + \eta_t, \quad (3.12)$$

donde s_t representa una componente regular o aprovechable y η_t representa una perturbación. Esta descomposición tiene valor conceptual, pero en datos reales no suele conocerse de antemano. Una oscilación puede parecer ruido bajo un modelo simple y adquirir estructura bajo una representación más adecuada. Por ello, la distinción entre señal y ruido depende del modelo, de la escala temporal y de la información disponible.

Los modelos temporales pueden clasificarse, de forma básica, en deterministas y estocásticos. En un modelo determinista, la regla de evolución fija el estado siguiente a partir del estado actual:

$$x_{t+1} = f(x_t). \quad (3.13)$$

En un modelo estocástico interviene una perturbación aleatoria:

$$x_{t+1} = f(x_t) + \varepsilon_{t+1}. \quad (3.14)$$

La presencia de irregularidad visual en una serie no permite distinguir por sí sola entre ambas posibilidades. Un sistema determinista no lineal puede producir trayectorias muy irregulares, mientras que un modelo estocástico puede generar patrones persistentes. Esta distinción es relevante para interpretar con cautela los resultados de técnicas de dinámica no lineal.

En predicción de series temporales se trabaja normalmente con modelos de referencia. Un modelo de referencia es una alternativa sencilla que fija un nivel mínimo de comparación. En series persistentes, una referencia habitual es la persistencia:

$$\hat{y}_t^{(h)} = x_t. \quad (3.15)$$

Otra posibilidad es utilizar una media histórica o un modelo autorregresivo lineal. La comparación con estas referencias es necesaria porque un modelo más complejo solo aporta valor predictivo si mejora de forma verificable a alternativas simples.

La evaluación debe respetar el orden temporal. Si el conjunto de observaciones se divide en entrenamiento, validación y prueba, una partición coherente cumple

$$1 \leq t \leq T_{\text{train}} < T_{\text{val}} < T_{\text{test}} \leq T. \quad (3.16)$$

Los datos de entrenamiento se utilizan para ajustar el modelo, los de validación para seleccionar parámetros y los de prueba para medir el rendimiento final. Esta separación reproduce la situación real de predicción: al estimar el futuro, las observaciones posteriores al instante de decisión todavía no están disponibles.

En este TFG la predicción se entiende como una estimación a corto plazo bajo esta restricción temporal. La aplicación financiera se centra en la volatilidad realizada futura, es decir, en la intensidad de variación esperada durante un horizonte breve. La dirección del precio queda fuera del objetivo del trabajo. Esta delimitación permite estudiar una magnitud con mayor persistencia temporal y reduce la ambigüedad entre predicción de variabilidad y predicción de rentabilidad.

3.2– Complejidad, no linealidad y caos determinista

En este trabajo el término complejidad se utiliza en un sentido técnico. Una serie no se considera compleja únicamente por tener muchos datos o por presentar una gráfica difícil de interpretar. La complejidad aparece cuando el comportamiento observado no queda descrito de forma satisfactoria mediante una regla simple, estable y lineal. En series económicas y financieras esto puede manifestarse mediante dependencia del pasado, cambios de régimen, agrupamiento de episodios extremos, variabilidad no constante o relaciones que cambian según el estado del sistema.

La idea de linealidad puede formularse de manera precisa. Un operador L es lineal si, para cualesquiera valores x, y y escalares α, β , se cumple

$$L(\alpha x + \beta y) = \alpha L(x) + \beta L(y). \quad (3.17)$$

Esta propiedad recoge dos principios: proporcionalidad y superposición. La proporcionalidad indica que multiplicar la entrada por una constante multiplica la salida por esa misma constante. La superposición indica que el efecto conjunto de dos entradas puede obtenerse sumando los efectos que produciría cada una por separado.

En tiempo discreto, una evolución lineal elemental puede escribirse como

$$x_{t+1} = ax_t, \quad (3.18)$$

donde el valor futuro depende proporcionalmente del valor actual. En la práctica también se utilizan modelos afines, de la forma

$$x_{t+1} = ax_t + b, \quad (3.19)$$

que incorporan un término constante b . Aunque estrictamente una relación afín no cumple la propiedad de linealidad si $b \neq 0$, en el contexto de series temporales suele incluirse dentro de la familia de modelos lineales con constante.

Una relación no lineal no cumple la propiedad de superposición. En forma general puede escribirse como

$$x_{t+1} = f(x_t), \quad (3.20)$$

donde f no es una función lineal. Un ejemplo clásico es el mapa logístico,

$$x_{t+1} = rx_t(1 - x_t), \quad (3.21)$$

en el que el término $x_t(1 - x_t)$ introduce una interacción multiplicativa del estado consigo mismo. Esta clase de relaciones puede generar comportamientos muy distintos según el valor de los parámetros y la región del espacio en la que se encuentre el sistema.

La no linealidad puede aparecer en distintos aspectos de una serie temporal. Una posibilidad es que afecte a la media condicional, es decir, al valor esperado de la serie dado su pasado:

$$\mathbb{E}[x_t \mid \mathcal{F}_{t-1}]. \quad (3.22)$$

Otra posibilidad, especialmente relevante en series financieras, es que afecte a la varianza condicional:

$$\text{Var}(x_t \mid \mathcal{F}_{t-1}). \quad (3.23)$$

En este segundo caso, el nivel esperado de la serie puede no mostrar una dependencia clara, mientras que la intensidad de sus fluctuaciones sí depende del pasado. Este fenómeno está relacionado con la heterocedasticidad y con el agrupamiento de volatilidad: periodos de alta variabilidad tienden a estar próximos a otros periodos de alta variabilidad, y lo mismo ocurre con los periodos de baja variabilidad.

La distinción entre irregularidad, aleatoriedad y caos es esencial. Una serie irregular es aquella cuya evolución no sigue un patrón visual sencillo. Esa irregularidad puede modelizarse mediante un componente aleatorio, como ocurre en muchos modelos estocásticos. Sin embargo, también puede surgir de una regla determinista no lineal. En ese caso, el sistema no contiene ruido en su definición, pero sus trayectorias pueden parecer desordenadas.

El caos determinista describe esta situación particular: un sistema gobernado por reglas deterministas genera trayectorias irregulares, acotadas y sensibles a las condiciones iniciales. La sensibilidad a las condiciones iniciales significa que dos estados iniciales muy próximos pueden evolucionar hacia trayectorias muy diferentes. Si δ_0 representa una pequeña separación inicial entre dos trayectorias y δ_t la separación tras t pasos, una forma habitual de expresar esta divergencia es

$$|\delta_t| \approx e^{\lambda t} |\delta_0|, \quad (3.24)$$

donde λ representa una tasa media de separación. Cuando $\lambda > 0$, la distancia entre trayectorias cercanas crece exponencialmente durante el intervalo en el que la aproximación es válida. Esta propiedad limita la predicción a largo plazo, aunque puede permitir predicción local a corto plazo.

En algunos sistemas dinámicos, las trayectorias no recorren todo el espacio posible, sino que quedan confinadas en una región hacia la que el sistema tiende. Esa región se denomina atractor. En sistemas simples, el atractor puede ser un punto fijo o un ciclo periódico. En sistemas caóticos, la geometría puede ser más irregular y dar lugar a lo que se conoce como atractor extraño. En series observadas reales, sin embargo, la existencia de una nube de puntos estructurada o de patrones recurrentes no basta para afirmar que se ha identificado un atractor determinista.

La no linealidad es una condición necesaria para el caos determinista, pero no es suficiente. Muchos sistemas no lineales convergen a un equilibrio, oscilan de forma periódica o presentan comportamientos regulares. Por tanto, un contraste que detecte no linealidad no demuestra caos. Del mismo modo, una estimación positiva de divergencia local, una dimensión de correlación aparentemente baja o una mejora de predicción local deben interpretarse como indicios parciales, no como pruebas aisladas.

Esta cautela es especialmente necesaria en series financieras. Los datos de mercado combinan ruido, cambios de régimen, heterocedasticidad, dependencia temporal, efectos de liquidez y posibles errores de medición. Además, el proceso generador no se observa directamente. Por ello, al estudiar una serie como la volatilidad realizada de Bitcoin, resulta más adecuado hablar de dependencia temporal, no linealidad, estructura compatible con dinámica compleja y utilidad predictiva parcial.

La tesis de Olmedo Fernández utiliza esta distinción como punto de partida para estudiar series económicas desde la dinámica no lineal (Olmedo Fernández, 2001). Este TFG adopta esa línea metodológica, pero mantiene una interpretación conservadora. Los métodos de análisis permiten buscar indicios de comportamiento complejo y comparar modelos de predicción, pero esos indicios no autorizan, por sí solos, una afirmación de caos determinista en Bitcoin.

3.3– Indicios de caos en series económicas

En una serie económica real la identificación de caos determinista plantea dificultades que no aparecen en un sistema sintético. En un sistema construido por el investigador, como el mapa logístico, la regla de evolución es conocida y puede analizarse directamente. En una serie observada, en cambio, solo se dispone de una secuencia de datos generada por un mecanismo no observable. Si la serie se denota por $\{x_t\}_{t=1}^T$, el proceso que ha producido cada observación no se conoce de forma explícita. Por tanto, el problema consiste en inferir propiedades del posible

mecanismo generador a partir de una única realización finita.

Esta diferencia obliga a utilizar un lenguaje prudente. En datos económicos y financieros no suele ser posible demostrar caos en sentido estricto. Las observaciones pueden estar afectadas por ruido, errores de medida, agregación temporal, cambios institucionales, cambios de régimen, heterocedasticidad y factores externos no incluidos en la serie. Además, muchos métodos de dinámica no lineal fueron formulados bajo hipótesis más limpias que las que se cumplen en datos financieros reales: muestras largas, dinámica relativamente estable, bajo ruido y observación adecuada del sistema. Cuando estas condiciones no se cumplen plenamente, los resultados deben interpretarse como indicios.

Un indicio es una señal parcial compatible con una determinada hipótesis, pero insuficiente para establecerla por sí sola. En este contexto, pueden considerarse indicios la aparición de sensibilidad local, la existencia de estructura al reconstruir el espacio de estados, una dimensión efectiva relativamente baja en algún rango de escalas, una entropía menor que la obtenida en series barajadas o una mejora de la predicción local a corto plazo frente a modelos de referencia. Estas evidencias no tienen el mismo significado ni la misma fuerza. Su utilidad procede de la lectura conjunta y de la comparación con controles adecuados (Abarbanel, 1996; Kantz y Schreiber, 2003; Theiler, Eubank, Longtin, Galdrikian, y Farmer, 1992). La sensibilidad local se asocia a la separación de trayectorias inicialmente próximas. Si dos estados reconstruidos X_i y X_j son cercanos, puede estudiarse cómo evoluciona su distancia tras k pasos:

$$d_{ij}(k) = \|X_{i+k} - X_{j+k}\|. \quad (3.25)$$

Un crecimiento sistemático de esta distancia puede ser compatible con sensibilidad a las condiciones iniciales. Sin embargo, en una serie financiera también puede deberse a ruido, cambios de régimen o heterocedasticidad. Por ello, una pendiente positiva en una estimación tipo Lyapunov no constituye una prueba aislada de caos.

La reconstrucción del espacio de estados proporciona otro tipo de evidencia. A partir de una única serie escalar se construyen vectores retardados de la forma

$$X_t = (x_t, x_{t-\tau}, x_{t-2\tau}, \dots, x_{t-(m-1)\tau}), \quad (3.26)$$

donde τ es el retardo y m la dimensión de reconstrucción. Si la nube de puntos reconstruida muestra regiones diferenciadas, recurrencias o cierta organización geométrica, puede pensarse que la serie conserva información temporal relevante. Aun así, una nube estructurada no implica necesariamente la existencia de un atractor determinista. También puede aparecer por persistencia lineal, por cambios de volatilidad o por una distribución marginal no uniforme.

La dimensión de correlación se utiliza para estudiar cómo crece el número de puntos vecinos al aumentar una escala de distancia r . De forma simplificada, la suma de correlación puede escribirse como

$$C(r) = \frac{2}{N(N-1)} \sum_{i < j} \mathbf{1}\{\|X_i - X_j\| < r\}, \quad (3.27)$$

donde $\mathbf{1}\{\cdot\}$ vale 1 si la condición se cumple y 0 en caso contrario. Si existe una región en la que $C(r)$ sigue aproximadamente una ley de potencia,

$$C(r) \propto r^{D_2}, \quad (3.28)$$

entonces D_2 puede interpretarse como una estimación de dimensión efectiva. En datos reales, esta estimación es sensible al tamaño muestral, al ruido, a la elección de escalas y a la correlación temporal. Por tanto, una dimensión aparentemente baja solo tiene valor si se compara con controles adecuados.

La entropía proporciona una forma distinta de analizar la irregularidad. En términos generales, si una serie genera patrones π con probabilidades $p(\pi)$, una medida de entropía puede escribirse como

$$H = - \sum_{\pi} p(\pi) \log p(\pi). \quad (3.29)$$

Valores altos indican mayor diversidad de patrones; valores más bajos indican mayor regularidad. En una serie económica, una entropía menor que la de una serie barajada puede sugerir que el orden temporal contiene información. No obstante, esa diferencia no separa por sí sola no linealidad, dependencia lineal, estacionalidad o cambios de régimen.

Por este motivo son necesarios controles empíricos. Una serie barajada conserva los valores observados, pero destruye su orden temporal. Si un estadístico distingue claramente la serie original de sus versiones barajadas, puede afirmarse que el orden temporal contiene información. Este control descarta que el resultado se deba únicamente a la distribución marginal de los datos.

Las series sustitutas permiten contrastes más exigentes. Algunas se construyen para conservar aproximadamente la estructura lineal o espectral de la serie original y destruir otros aspectos de su organización temporal. Si un estadístico separa la serie original de series sustitutas que conservan parte de la dependencia lineal, la evidencia a favor de una estructura adicional es más fuerte. Si no existe separación clara, parte del comportamiento observado puede explicarse mediante dependencia lineal, heterocedasticidad o propiedades de segundo orden.

La dificultad aumenta en series financieras. La varianza suele cambiar con el tiempo y los episodios extremos tienden a concentrarse. En este caso, una señal compatible con no linealidad puede estar asociada a heterocedasticidad condicional. De forma general, esto significa que

$$\text{Var}(x_t \mid \mathcal{F}_{t-1}) \quad (3.30)$$

depende de la información pasada. Así, aunque la media condicional de los retornos sea difícil de predecir, la variabilidad futura puede contener dependencia temporal. Esta distinción es central en el estudio de volatilidad financiera.

En esta memoria se separan cuatro niveles de interpretación. El primero es la dependencia temporal: el pasado contiene información sobre el presente o sobre el futuro próximo. El segundo es la no linealidad: esa dependencia no queda explicada completamente por relaciones lineales en media. El tercero es la dinámica compleja: la serie muestra patrones compatibles con una organización temporal más rica que la de una sucesión independiente. El cuarto es el caos determinista: una propiedad fuerte de ciertos sistemas dinámicos deterministas, asociada a sensibilidad a condiciones iniciales, trayectorias acotadas e irregularidad persistente.

El análisis realizado en este TFG se mueve deliberadamente en los tres primeros niveles. Se estudia si la volatilidad realizada de Bitcoin presenta dependencia temporal, si aparecen indicios de no linealidad y si esa información tiene utilidad predictiva parcial. El cuarto nivel se reserva para sistemas donde la evidencia sea suficiente. En el caso de Bitcoin, los resultados deben interpretarse como indicios y comparaciones, no como demostración de caos determinista.

3.4– Sistemas dinámicos y mapa logístico

Un sistema dinámico es una representación matemática de la evolución de un estado a lo largo del tiempo. El estado contiene las variables necesarias para describir la situación del sistema en un instante determinado. El conjunto de todos los estados posibles recibe el nombre de espacio de estados. Cuando el sistema evoluciona, genera una sucesión de estados; esa sucesión se denomina trayectoria u órbita.

En un sistema dinámico continuo, el tiempo se considera una variable continua y la evolución suele describirse mediante ecuaciones diferenciales. Si $x(t)$ representa el estado del sistema en el instante t , una formulación general puede escribirse como

$$\frac{dx(t)}{dt} = F(x(t)), \quad (3.31)$$

donde F determina la velocidad de cambio del estado. Esta forma es habitual en modelos físicos, biológicos o económicos formulados en tiempo continuo.

En un sistema dinámico discreto, el tiempo avanza por pasos. Si x_t representa el estado en el paso t , la evolución se expresa mediante una regla de iteración:

$$x_{t+1} = f(x_t). \quad (3.32)$$

La función f transforma el estado actual en el estado siguiente. Esta formulación es adecuada cuando los datos se observan en instantes separados o cuando el propio modelo se define por iteraciones. En este TFG se trabaja con series discretas, ya que los datos financieros se registran en intervalos de cinco minutos y el sistema sintético utilizado para comprobación también se genera mediante iteraciones.

En dimensión mayor que uno, el estado deja de ser un escalar y pasa a ser un vector. Si el sistema tiene m componentes, puede escribirse como

$$X_t = (x_{1,t}, x_{2,t}, \dots, x_{m,t}) \in \mathbb{R}^m, \quad (3.33)$$

y la evolución adopta la forma

$$X_{t+1} = F(X_t). \quad (3.34)$$

Esta notación permite interpretar la dinámica como movimiento dentro de un espacio de estados. En cada instante, el sistema ocupa un punto de ese espacio; al evolucionar, va trazando una trayectoria.

El mapa logístico es uno de los ejemplos más conocidos de sistema dinámico discreto no lineal. Se define mediante la ecuación

$$x_{t+1} = rx_t(1 - x_t), \quad (3.35)$$

donde normalmente $x_t \in [0, 1]$ y r es un parámetro positivo que controla la dinámica. A pesar de su forma sencilla, este sistema puede mostrar comportamientos muy distintos según el valor de r . Para ciertos valores, la sucesión converge a un punto fijo. Para otros, oscila entre varios valores de forma periódica. Para valores suficientemente altos, como $r = 4$, puede generar una trayectoria irregular y sensible a las condiciones iniciales (May, 1976).

La no linealidad del mapa logístico procede del término $x_t(1 - x_t)$. Este término hace que el efecto de x_t sobre x_{t+1} no sea proporcional en todo el intervalo. Cuando x_t es pequeño, el producto puede aumentar en la siguiente iteración; cuando x_t se acerca a 1, el factor $1 - x_t$ reduce el valor siguiente. Esta interacción produce una dinámica que no puede entenderse como una simple relación lineal entre el estado actual y el futuro.

La sensibilidad a las condiciones iniciales puede ilustrarse comparando dos trayectorias del mapa logístico que parten de valores casi iguales. Si x_0 y $\tilde{x}_0$ son dos condiciones iniciales próximas, la separación inicial puede escribirse como

$$\delta_0 = |x_0 - \tilde{x}_0|. \quad (3.36)$$

Tras varias iteraciones, la separación será

$$\delta_t = |x_t - \tilde{x}_t|. \quad (3.37)$$

En un régimen sensible a las condiciones iniciales, una diferencia inicial muy pequeña puede crecer hasta producir trayectorias claramente distintas. Esta propiedad no implica ausencia de regla: ambas trayectorias siguen la misma ecuación. La dificultad aparece porque un pequeño error en la condición inicial se amplifica con el tiempo.

El mapa logístico tiene una función concreta dentro de esta memoria. No se utiliza como modelo de Bitcoin ni como representación de un mercado financiero. Su papel es servir como sistema de control. Al conocerse la regla generadora, puede comprobarse si el procedimiento de análisis responde de forma razonable cuando se aplica a una dinámica determinista no lineal conocida. Esta comprobación es útil antes de aplicar las mismas herramientas a una serie financiera real, donde la regla generadora no se observa y los datos contienen ruido, cambios de régimen y heterocedasticidad.

La comparación entre el mapa logístico y la serie de Bitcoin también ayuda a delimitar el alcance de las conclusiones. En el mapa logístico, el investigador conoce la ecuación que produce la serie. En Bitcoin, solo se dispone de observaciones de mercado. Por tanto, un resultado claro en el sistema sintético no autoriza a trasladar automáticamente la misma interpretación al caso financiero. La utilidad del ejemplo sintético está en validar el comportamiento del procedimiento, no en demostrar que la serie real tenga la misma naturaleza.

Esta forma de usar sistemas dinámicos sencillos como referencia conceptual está alineada con el papel que la tesis de Olmedo Fernández concede a los ejemplos deterministas antes de pasar al estudio de series económicas reales (Olmedo Fernández, 2001). En este TFG se conserva esa lógica, pero se separa explícitamente el caso controlado del caso empírico.

3.5– Series temporales financieras y volatilidad realizada

Una serie financiera de precios puede representarse como $\{P_t\}_{t=1}^T$, donde P_t es el precio observado en el instante t . En datos de velas, cada intervalo temporal contiene varias magnitudes: precio de apertura, máximo, mínimo, cierre, volumen negociado y número de operaciones. En este trabajo se utiliza principalmente el precio de cierre, ya que resume el último precio negociado dentro de cada intervalo de cinco minutos.

Aunque el precio es la variable más visible, no suele ser la más adecuada para el análisis temporal directo. Una serie de precios puede presentar tendencias, cambios de escala y niveles muy distintos a lo largo de la muestra. Además, comparar una variación de 100 unidades cuando el precio está en 10.000 con la misma variación cuando el precio está en 60.000 no tiene la misma interpretación relativa. Por este motivo, en análisis financiero se trabaja habitualmente con retornos.

El retorno logarítmico entre dos instantes consecutivos se define como

$$r_t = \log(P_t) - \log(P_{t-1}). \quad (3.38)$$

Esta expresión también puede escribirse como

$$r_t = \log\left(\frac{P_t}{P_{t-1}}\right). \quad (3.39)$$

El retorno logarítmico mide la variación relativa del precio entre dos instantes consecutivos. Para variaciones pequeñas, r_t se aproxima al retorno simple $(P_t - P_{t-1})/P_{t-1}$, pero presenta ventajas algebraicas, ya que los retornos logarítmicos acumulados en varios periodos pueden sumarse.

La volatilidad mide la intensidad de las variaciones del precio. No indica si el precio sube o baja, sino cuánto fluctúa. Dos periodos pueden tener retornos medios similares y, sin embargo,

mostrar niveles de volatilidad muy distintos. En mercados financieros esta distinción es relevante porque la incertidumbre, el riesgo y la actividad de mercado suelen estar más relacionados con la magnitud de los movimientos que con su signo.

Una forma empírica de medir la variabilidad observada es la volatilidad realizada. Para una ventana de h observaciones, se construye acumulando retornos al cuadrado. En el caso de una ventana pasada, disponible en el instante t , se define como

$$RV_t^{(h)} = \sum_{i=0}^{h-1} r_{t-i}^2. \quad (3.40)$$

Esta magnitud resume la variación acumulada durante las últimas h observaciones. En la implementación de este proyecto se utiliza la suma de retornos al cuadrado, no una media normalizada. Esta elección conserva la interpretación de variación acumulada dentro de la ventana.

La volatilidad realizada está relacionada con la idea de variación cuadrática de un proceso de precios y se utiliza de forma habitual cuando se dispone de datos de alta frecuencia (Barndorff-Nielsen y Shephard, 2002; Andersen, Bollerslev, Diebold, y Labys, 2003). En términos prácticos, permite construir una medida observable de volatilidad a partir de los retornos registrados, sin tener que tratar la volatilidad únicamente como una variable latente.

Para estabilizar la escala de la serie se aplica una transformación logarítmica. En este trabajo se define

$$v_t^{(h)} = \log \left(RV_t^{(h)} + \varepsilon \right), \quad (3.41)$$

donde $\varepsilon > 0$ es una constante pequeña introducida por estabilidad numérica. Su función es evitar problemas cuando la suma de cuadrados es muy próxima a cero. Esta transformación reduce la asimetría de la variable y suaviza la influencia de episodios extremos, lo que facilita su tratamiento estadístico.

La aplicación empírica de esta memoria se centra en ventanas de doce velas de cinco minutos. Por tanto, la ventana principal equivale a una hora:

$$h = 12. \quad (3.42)$$

La serie principal del estudio es la volatilidad realizada pasada transformada logarítmicamente, $v_t^{(12)}$. Esta variable se construye únicamente con información disponible hasta el instante t , por lo que puede utilizarse como entrada de modelos predictivos.

El objetivo predictivo es la volatilidad realizada futura sobre la hora siguiente. De forma análoga, para un horizonte de h observaciones puede definirse como

$$RV_{t,+}^{(h)} = \sum_{i=1}^h r_{t+i}^2. \quad (3.43)$$

Su transformación logarítmica viene dada por

$$y_t^{(h)} = \log \left(RV_{t,+}^{(h)} + \varepsilon \right). \quad (3.44)$$

En el caso principal del TFG, $h = 12$, de modo que $y_t^{(12)}$ representa la volatilidad realizada de la hora posterior a t . Esta variable no está disponible en el instante de predicción; se utiliza como objetivo para entrenar y evaluar los modelos.

La separación entre volatilidad pasada y volatilidad futura es metodológicamente importante. La variable $v_t^{(h)}$ resume información contenida en $\mathcal{F}_t$, mientras que $y_t^{(h)}$ depende de observaciones

posteriores. Si un modelo utilizara directa o indirectamente $y_t^{(h)}$ para predecir en t , se produciría fuga de información. Por ello, la construcción de variables debe respetar estrictamente la dirección temporal de la serie.

Una propiedad habitual de las series financieras es el agrupamiento de volatilidad. Los periodos de alta variabilidad tienden a estar próximos a otros periodos de alta variabilidad, y los periodos tranquilos tienden a agruparse. Esta propiedad puede expresarse de forma intuitiva mediante la dependencia temporal de los cuadrados de los retornos:

$$\text{Cov}(r_t^2, r_{t-k}^2) \neq 0 \quad (3.45)$$

para algunos retardos $k > 0$. Aunque los retornos r_t pueden mostrar poca autocorrelación lineal, sus cuadrados o valores absolutos suelen presentar una dependencia más marcada. Esta diferencia justifica estudiar volatilidad en lugar de dirección del precio.

El carácter persistente de la volatilidad realizada permite plantear problemas de predicción a corto plazo. La pregunta no es si el precio subirá o bajará, sino si la variabilidad futura será alta o baja en relación con el comportamiento reciente. Esta formulación es más adecuada para el objetivo del TFG, ya que la volatilidad suele contener más estructura temporal que los retornos direccionales.

La principal limitación de estas variables es el solapamiento de ventanas. Dos valores consecutivos de $RV_t^{(h)}$ comparten $h - 1$ retornos cuando la ventana se desplaza solo una observación. Por ejemplo, con $h = 12$, las ventanas de volatilidad pasada en t y $t + 1$ comparten once de los doce retornos utilizados. Este solapamiento introduce dependencia mecánica entre observaciones próximas y debe tenerse en cuenta al interpretar autocorrelaciones, contrastes y métricas predictivas.

Además de la ventana principal de una hora, en el trabajo se consideran escalas temporales más amplias para capturar memoria multiescala. Con datos de cinco minutos, las ventanas $h = 12$, $h = 48$ y $h = 288$ corresponden aproximadamente a una hora, cuatro horas y un día. Esta idea conecta con los modelos HAR de volatilidad realizada, que representan la persistencia de la volatilidad mediante componentes de distintas escalas temporales (Corsi, 2009). La formulación concreta de HAR-logRV se presenta más adelante.

3.6– Caracterización lineal de series temporales

La caracterización lineal estudia una serie temporal mediante medidas que describen su nivel, su dispersión y la relación lineal entre observaciones separadas por distintos retardos. Este análisis debe realizarse antes de aplicar herramientas no lineales, ya que una parte importante de la estructura observada en una serie puede deberse a autocorrelación, persistencia o ciclos simples (Brockwell y Davis, 2002; Box, Jenkins, Reinsel, y Ljung, 2015).

Sea $\{x_t\}_{t=1}^T$ una serie temporal. La media muestral resume el nivel promedio de la serie:

$$\bar{x} = \frac{1}{T} \sum_{t=1}^T x_t. \quad (3.46)$$

La varianza muestral mide la dispersión de los valores alrededor de esa media:

$$s^2 = \frac{1}{T-1} \sum_{t=1}^T (x_t - \bar{x})^2. \quad (3.47)$$

Estas dos magnitudes no utilizan todavía el orden temporal de las observaciones. Una serie y una permutación aleatoria de esa misma serie tendrían la misma media y la misma varianza. Para estudiar la dependencia temporal es necesario introducir medidas que comparen la serie consigo misma en distintos retardos.

La autocovarianza a retardo k mide la relación lineal entre x_t y x_{t-k} . En términos poblacionales se define como

$$\gamma(k) = \text{Cov}(x_t, x_{t-k}) = \mathbb{E}[(x_t - \mu)(x_{t-k} - \mu)], \quad (3.48)$$

donde $\mu = \mathbb{E}[x_t]$, suponiendo que la media no depende del tiempo. En una muestra finita, una estimación habitual es

$$\hat{\gamma}(k) = \frac{1}{T} \sum_{t=k+1}^T (x_t - \bar{x})(x_{t-k} - \bar{x}). \quad (3.49)$$

Si $\hat{\gamma}(k) > 0$, los valores altos tienden a aparecer junto a valores altos k pasos antes, y los valores bajos junto a valores bajos. Si $\hat{\gamma}(k) < 0$, los valores altos tienden a aparecer junto a valores bajos separados por ese retardo. Si $\hat{\gamma}(k)$ está cerca de cero, no se aprecia una relación lineal clara a ese retardo.

La autocorrelación normaliza la autocovarianza para expresarla en una escala comparable:

$$\rho(k) = \frac{\gamma(k)}{\gamma(0)}. \quad (3.50)$$

Como $\gamma(0)$ es la varianza de la serie, $\rho(k)$ mide la intensidad de la dependencia lineal a retardo k en una escala adimensional. Su estimador muestral es

$$\hat{\rho}(k) = \frac{\hat{\gamma}(k)}{\hat{\gamma}(0)}. \quad (3.51)$$

Los valores de $\hat{\rho}(k)$ se encuentran entre -1 y 1 . Valores próximos a 1 indican fuerte persistencia lineal positiva; valores próximos a -1 indican alternancia lineal; valores próximos a cero indican ausencia de dependencia lineal apreciable a ese retardo.

La función de autocorrelación, ACF, es la representación de $\hat{\rho}(k)$ para una colección de retardos $k = 1, 2, \dots, K$. Su objetivo es mostrar cómo se relaciona la serie con su propio pasado. Una ACF que decrece lentamente indica persistencia: los valores actuales conservan memoria de observaciones anteriores durante muchos retardos. Una ACF que cae rápidamente hacia cero sugiere que la dependencia lineal desaparece pronto. Si aparecen picos en retardos concretos, puede existir una periodicidad o repetición temporal asociada a esos retardos.

En datos de cinco minutos, la interpretación de los retardos debe traducirse a tiempo físico. Un retardo $k = 12$ equivale aproximadamente a una hora; un retardo $k = 288$, a un día; y un retardo $k = 2016$, a una semana. Por tanto, un pico de autocorrelación en $k = 288$ no es solo un rasgo numérico del gráfico: puede indicar una regularidad diaria. Esta traducción es necesaria para interpretar correlogramas en series intradía.

La ACF mide dependencia lineal total entre x_t y x_{t-k} . Sin embargo, una autocorrelación alta en un retardo grande no implica necesariamente que exista una relación directa entre esos dos valores. Puede ocurrir que la dependencia aparezca inducida por retardos intermedios. Por ejemplo, si x_t depende de x_{t-1} , y x_{t-1} depende de x_{t-2} , entonces puede observarse relación entre x_t y x_{t-2} aunque el efecto directo de x_{t-2} desaparezca al tener en cuenta x_{t-1} .

La función de autocorrelación parcial, PACF, intenta separar estos efectos. La autocorrelación parcial a retardo k mide la relación lineal entre x_t y x_{t-k} después de eliminar el efecto lineal de

los retardos intermedios $x_{t-1}, x_{t-2}, \dots, x_{t-k+1}$. Una forma de definirla es mediante la regresión autorregresiva de orden k :

$$x_t = c + \phi_{k,1}x_{t-1} + \phi_{k,2}x_{t-2} + \dots + \phi_{k,k}x_{t-k} + u_t. \quad (3.52)$$

La autocorrelación parcial a retardo k es el coeficiente asociado al último retardo:

$$\alpha(k) = \phi_{k,k}. \quad (3.53)$$

Este coeficiente mide la contribución lineal adicional de x_{t-k} una vez considerados los retardos anteriores. Por eso la PACF es útil para estudiar modelos autorregresivos: ayuda a valorar cuántos retardos aportan información directa sobre el valor actual.

La diferencia entre ACF y PACF es importante. La ACF responde a la pregunta: “¿se parece la serie a sí misma k pasos antes?”. La PACF responde a una pregunta más restrictiva: “¿aporta el retardo k información lineal adicional una vez incluidos los retardos más recientes?”. En modelos autorregresivos lineales, esta distinción permite identificar dependencias directas y evitar interpretar como independientes relaciones que se explican por la propagación de retardos más cortos.

Un modelo autorregresivo lineal de orden p , denotado $AR(p)$, se escribe como

$$x_t = c + \sum_{i=1}^p \phi_i x_{t-i} + \varepsilon_t. \quad (3.54)$$

En este modelo, el valor actual se explica como combinación lineal de sus p valores anteriores. La ACF y la PACF proporcionan información preliminar sobre la memoria lineal de la serie y sobre la posible elección del orden p . No sustituyen a criterios de selección formal, como AIC o BIC, pero ayudan a interpretar la estructura temporal antes del ajuste del modelo.

Otra herramienta de caracterización lineal es el análisis espectral. Mientras la ACF estudia la serie en el dominio temporal, el análisis espectral estudia cómo se reparte la variabilidad de la serie entre distintas frecuencias. Una frecuencia alta corresponde a oscilaciones rápidas; una frecuencia baja corresponde a movimientos más lentos. Si una serie contiene ciclos aproximadamente repetidos, parte de su variabilidad se concentrará en las frecuencias asociadas a esos ciclos.

El periodograma es una estimación muestral de esa distribución de potencia por frecuencias (Brockwell y Davis, 2002; Priestley, 1981).

$$I(\omega) = \frac{1}{T} \left| \sum_{t=1}^T x_t e^{-i\omega t} \right|^2. \quad (3.55)$$

Esta expresión mide la intensidad con la que la serie contiene una oscilación asociada a la frecuencia ω . En datos financieros intradía, el periodograma puede sugerir patrones horarios, diarios o semanales. Estos patrones no deben interpretarse como periodicidad exacta, ya que los mercados no son sistemas cerrados ni perfectamente periódicos, pero sí pueden indicar regularidades temporales relevantes.

ACF, PACF y periodograma son herramientas de caracterización lineal o espectral. Permiten identificar persistencia, memoria lineal, posibles ciclos y escalas temporales relevantes. También sirven para construir modelos de referencia y para comprobar qué parte de la estructura observada puede explicarse antes de recurrir a técnicas no lineales. Esta es la razón por la que aparecen antes de la reconstrucción del espacio de estados y de los contrastes de no linealidad (Brockwell y Davis, 2002; Olmedo Fernández, 2001).

3.7– Estacionariedad, heterocedasticidad y no linealidad

Una serie temporal es débilmente estacionaria cuando sus propiedades estadísticas de primer y segundo orden no cambian con el tiempo. De forma habitual, esto se expresa exigiendo media constante, varianza constante y autocovarianza dependiente solo del retardo:

$$\mathbb{E}[x_t] = \mu, \quad \text{Var}(x_t) = \sigma^2, \quad \text{Cov}(x_t, x_{t-k}) = \gamma(k). \quad (3.56)$$

Esta definición no implica que la serie sea constante ni que sus valores sean fácilmente predecibles. Una serie estacionaria puede fluctuar de forma intensa; lo relevante es que esas fluctuaciones se produzcan alrededor de una estructura estadística estable.

La estacionariedad es importante porque muchas herramientas de series temporales se interpretan bajo la hipótesis de que la serie mantiene un comportamiento relativamente homogéneo durante la muestra. Si una serie presenta tendencia, cambios de escala o rupturas de régimen, algunas medidas pueden ser engañosas. Por ejemplo, una autocorrelación elevada puede deberse a una tendencia común y no a una relación dinámica estable entre observaciones. Del mismo modo, un contraste de no linealidad puede detectar cambios de régimen o variabilidad no constante en lugar de una estructura dinámica no lineal.

En la práctica no existe una única prueba definitiva de estacionariedad. Por este motivo suelen utilizarse contrastes con hipótesis nulas distintas. El contraste aumentado de Dickey-Fuller, ADF, parte de una regresión auxiliar en la que se estudia si la serie contiene una raíz unitaria (Dickey y Fuller, 1979). Una forma simplificada de la regresión es

$$\Delta x_t = \alpha + \beta t + \rho x_{t-1} + \sum_{i=1}^p \psi_i \Delta x_{t-i} + u_t, \quad (3.57)$$

donde $\Delta x_t = x_t - x_{t-1}$. La hipótesis de raíz unitaria se asocia al caso $\rho = 0$. Si se rechaza, se obtiene evidencia contra una forma persistente de no estacionariedad.

El contraste KPSS adopta el punto de vista contrario: su hipótesis nula es la estacionariedad alrededor de un nivel o de una tendencia (Kwiatkowski, Phillips, Schmidt, y Shin, 1992). Esta diferencia es útil. Si ADF no rechaza raíz unitaria y KPSS rechaza estacionariedad, la evidencia apunta a no estacionariedad. Si ADF rechaza raíz unitaria y KPSS no rechaza estacionariedad, la serie parece más compatible con estacionariedad. En series financieras reales, los resultados pueden ser mixtos, especialmente cuando hay cambios de régimen, episodios extremos o varianza variable. Por ello, estos contrastes deben interpretarse junto con gráficos, momentos móviles y conocimiento del problema.

La heterocedasticidad aparece cuando la varianza de una serie no es constante en el tiempo. En un modelo con residuos ε_t , la homocedasticidad exigiría

$$\text{Var}(\varepsilon_t \mid \mathcal{F}_{t-1}) = \sigma^2. \quad (3.58)$$

En cambio, si la varianza condicional depende de la información pasada, se tiene

$$\text{Var}(\varepsilon_t \mid \mathcal{F}_{t-1}) = \sigma_t^2, \quad (3.59)$$

donde σ_t^2 puede variar con t . Esta situación es habitual en series financieras: los retornos pueden tener media cercana a cero y, al mismo tiempo, alternar periodos de baja y alta variabilidad.

Los modelos ARCH formalizan esta dependencia temporal en la varianza (Engle, 1982). En su forma básica, un modelo ARCH(q) puede escribirse como

$$\varepsilon_t = \sigma_t z_t, \quad z_t \sim iid(0, 1), \quad (3.60)$$

con

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \varepsilon_{t-i}^2. \quad (3.61)$$

La idea central es que errores grandes en valor absoluto tienden a ir seguidos de periodos de mayor varianza condicional. En este contexto, no se estudia solo la dependencia de x_t , sino también la dependencia de x_t^2 , $|x_t|$ o de los residuos al cuadrado.

Para separar dependencia lineal en media de otros efectos, puede ajustarse un modelo autorregresivo de orden p :

$$x_t = c + \sum_{i=1}^p \phi_i x_{t-i} + \varepsilon_t. \quad (3.62)$$

Este modelo expresa el valor actual como combinación lineal de valores pasados más un residuo. Si después del ajuste los residuos ε_t siguen mostrando estructura, esa estructura no queda explicada por la dependencia lineal incluida en el modelo. Esta idea es importante antes de aplicar contrastes de no linealidad, porque una serie con fuerte autocorrelación lineal puede provocar rechazos que no necesariamente indican dinámica no lineal.

El contraste de Ljung-Box evalúa si un conjunto de autocorrelaciones hasta cierto retardo puede considerarse conjuntamente nulo (Ljung y Box, 1978). Para un número máximo de retardos m , el estadístico se define como

$$Q = T(T+2) \sum_{k=1}^m \frac{\hat{\rho}_k^2}{T-k}, \quad (3.63)$$

donde T es el tamaño muestral y $\hat{\rho}_k$ es la autocorrelación muestral a retardo k . Aplicado a residuos, el contraste permite analizar si queda dependencia lineal en media. Aplicado a residuos al cuadrado, ayuda a detectar dependencia en varianza. Esta segunda aplicación es especialmente relevante en series financieras, donde la autocorrelación de los retornos puede ser reducida mientras que la de los cuadrados resulta significativa.

La no linealidad, en este contexto, se refiere a dependencias que no quedan descritas adecuadamente por una combinación lineal de retardos. Puede aparecer en la media condicional, en la varianza condicional o en relaciones más generales entre observaciones. Por ello, rechazar un modelo lineal no identifica automáticamente un mecanismo caótico. Puede indicar heterocedasticidad, cambios de régimen, asimetrías, efectos umbral o mezcla de procesos.

El contraste BDS, introducido por Brock, Dechert y Scheinkman y desarrollado posteriormente con LeBaron, se utiliza como contraste de independencia basado en la correlación espacial de vectores contruidos a partir de la serie (Brock, Dechert, Scheinkman, y LeBaron, 1996). Su lógica consiste en comparar la frecuencia con la que aparecen observaciones cercanas en dimensión m con lo que cabría esperar si la serie fuera independiente e idénticamente distribuida. De forma simplificada, se construyen vectores

$$X_t^{(m)} = (x_t, x_{t-1}, \dots, x_{t-m+1}) \quad (3.64)$$

y se calcula una suma de correlación $C_m(\varepsilon)$, que mide la proporción de pares de vectores situados a distancia menor que ε . Bajo independencia, se espera aproximadamente

$$C_m(\varepsilon) \approx C_1(\varepsilon)^m. \quad (3.65)$$

El estadístico BDS mide la desviación entre ambos términos. Si la desviación es grande, se rechaza la hipótesis de independencia.

La interpretación del BDS requiere cautela. Un rechazo puede indicar dependencia no lineal, pero también puede deberse a dependencia lineal no eliminada, heterocedasticidad, no estacionariedad o cambios de régimen. Por este motivo, en aplicaciones empíricas se suele aplicar tanto a la serie original como a residuos de un modelo lineal. Si el rechazo persiste tras el filtrado lineal, la evidencia de estructura no explicada por el modelo lineal es más fuerte. Aun así, el contraste BDS no prueba caos determinista.

En conjunto, estacionariedad, heterocedasticidad y no linealidad deben analizarse como problemas relacionados. La estacionariedad controla si la estructura estadística de la serie es razonablemente estable; la heterocedasticidad examina si la varianza cambia de forma dependiente del pasado; y los contrastes de no linealidad estudian si queda estructura más allá de relaciones lineales simples. Esta secuencia evita interpretar como caos lo que puede explicarse por persistencia lineal, varianza condicional o cambios de régimen. Esta precaución está en línea con la metodología de análisis de series económicas no lineales expuesta por Olmedo Fernández (Olmedo Fernández, 2001).

3.8– Reconstrucción del espacio de estados

La reconstrucción del espacio de estados intenta resolver una dificultad habitual en el análisis de series temporales observadas: se dispone de una sola variable medida, pero el sistema que genera la serie puede depender de más componentes. En un mercado financiero, por ejemplo, el precio o la volatilidad observada reflejan la interacción de liquidez, expectativas, volumen, información externa, actividad de distintos agentes y cambios de régimen. La mayor parte de esos factores no se observa directamente. La reconstrucción por retardos propone construir una representación aproximada del estado del sistema utilizando valores pasados de la propia serie.

Sea $\{x_t\}_{t=1}^T$ una serie escalar observada. En lugar de estudiar cada valor x_t de forma aislada, se construye un vector formado por valores retardados:

$$X_t = (x_t, x_{t-\tau}, x_{t-2\tau}, \dots, x_{t-(m-1)\tau}) . \quad (3.66)$$

El parámetro τ es el tiempo de retardo y m es la dimensión de reconstrucción. Cada vector X_t pertenece a $\mathbb{R}^m$ y representa un estado reconstruido a partir de la historia de la serie. Al variar t , los vectores X_t forman una nube de puntos en el espacio reconstruido.

La idea geométrica es sencilla. Si el valor actual de la serie no contiene por sí solo suficiente información sobre la situación del sistema, se añaden valores pasados como coordenadas adicionales. De este modo, dos instantes se consideran parecidos no solo porque tengan un valor x_t similar, sino porque presentan una configuración histórica parecida. Esta representación permite estudiar recurrencias, vecindades, trayectorias locales y predicción en un espacio de estados aproximado.

La base teórica de esta construcción procede del teorema de Takens, que establece condiciones bajo las cuales la dinámica de un sistema determinista puede reconstruirse mediante coordenadas retardadas de una variable observada (Takens, 1981). En términos intuitivos, el teorema justifica que, bajo hipótesis adecuadas, una única medición escalar puede contener información suficiente para representar la geometría de la dinámica subyacente. La tesis de Olmedo Fernández recoge esta idea dentro del estudio de reconstrucción de series temporales económicas (Olmedo Fernández, 2001).

Las condiciones ideales del teorema no se cumplen plenamente en una serie financiera. En los datos de mercado hay ruido, muestra finita, cambios de régimen, heterocedasticidad, variables omitidas y posibles errores de medición. Por esta razón, en este TFG la reconstrucción no se interpreta como recuperación exacta del verdadero espacio de estados del mercado. Se utiliza como

una representación operativa para estudiar dependencia temporal, estructura local y capacidad predictiva.

La elección del retardo τ es importante. Si τ es demasiado pequeño, las coordenadas $x_t, x_{t-\tau}, x_{t-2\tau}$ serán muy parecidas entre sí. En ese caso, el vector reconstruido contiene información redundante y la nube de puntos queda cerca de una diagonal. Si τ es demasiado grande, las coordenadas pueden perder relación dinámica y comportarse como valores casi desconectados. El objetivo es elegir un retardo que reduzca redundancia sin destruir la dependencia temporal útil.

Un criterio habitual para orientar esta elección es la información mutua media. Para dos variables discretizadas X e Y , la información mutua se define como

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}. \quad (3.67)$$

Esta cantidad mide cuánta información aporta una variable sobre la otra. Si X e Y son independientes, entonces $p(x, y) = p(x)p(y)$ y la información mutua es nula. Si existe dependencia, la información mutua toma valores positivos. En reconstrucción por retardos se calcula la información mutua entre x_t y $x_{t-\tau}$ para distintos valores de τ . Fraser y Swinney proponen usar el primer mínimo de esta función como criterio práctico para seleccionar el retardo, porque busca un compromiso entre redundancia y pérdida de relación dinámica (Fraser y Swinney, 1986).

La elección de la dimensión m plantea un problema distinto. Si m es demasiado pequeño, la reconstrucción puede comprimir la geometría del sistema y hacer que puntos alejados parezcan cercanos. Si m es demasiado grande, el espacio se vuelve más disperso, aumenta el coste computacional y las distancias entre vecinos pueden perder significado práctico. Esta dificultad es especialmente relevante en datos financieros, donde el ruido y los cambios de régimen pueden degradar la noción de vecindad.

El método de falsos vecinos se basa en una idea geométrica. Dos puntos pueden parecer cercanos en una reconstrucción de baja dimensión porque la proyección ha plegado la geometría. Al aumentar la dimensión, esos puntos dejan de ser vecinos. Si $X_t^{(m)}$ representa el vector reconstruido en dimensión m , y $X_j^{(m)}$ es uno de sus vecinos próximos, se compara la distancia en dimensión m con la distancia tras añadir una coordenada adicional:

$$X_t^{(m+1)} = (x_t, x_{t-\tau}, \dots, x_{t-m\tau}). \quad (3.68)$$

Si la distancia aumenta de forma desproporcionada al pasar de m a $m + 1$, el vecino se considera falso. A medida que m crece, el porcentaje de falsos vecinos debería disminuir si la reconstrucción despliega adecuadamente la geometría. Este método fue propuesto por Kennel, Brown y Abarbanel como un criterio práctico para determinar una dimensión mínima de reconstrucción (Kennel, Brown, y Abarbanel, 1992).

El método de Cao proporciona un contraste adicional para estudiar la dimensión de reconstrucción (Cao, 1997). Su idea es analizar cómo cambian las relaciones entre vecinos al aumentar m , utilizando cocientes de distancias entre puntos próximos. Si al incrementar la dimensión la estructura de vecindad deja de cambiar de forma relevante, se interpreta que la dimensión es suficiente para la reconstrucción. Este método resulta útil como complemento de los falsos vecinos, aunque tampoco debe interpretarse como una prueba absoluta de dimensión real en datos ruidosos.

En la práctica, también se introduce una ventana de Theiler. Esta ventana impide que puntos demasiado próximos en el tiempo sean considerados vecinos válidos. La razón es que dos observaciones consecutivas suelen parecerse por continuidad temporal o por solapamiento de información, no necesariamente porque representen estados dinámicamente equivalentes. Si se permitiesen esos vecinos inmediatos, las estimaciones de vecindad, dimensión o divergencia local podrían resultar

artificialmente optimistas. Theiler mostró que los algoritmos de dimensión pueden producir estimaciones espurias cuando se aplican sin controlar la correlación temporal en series finitas (Theiler, 1986).

Formalmente, si se busca un vecino de X_t , se excluyen candidatos X_j tales que

$$|t - j| \leq W, \quad (3.69)$$

donde W es la ventana de Theiler. Así se evita comparar un punto con observaciones casi consecutivas. Esta precaución es relevante cuando se trabaja con series de alta frecuencia y con variables construidas mediante ventanas solapadas, como ocurre con la volatilidad realizada.

La reconstrucción por retardos se usa después como base para distintos análisis. En primer lugar, permite visualizar la serie en dos o tres coordenadas retardadas. En segundo lugar, permite estudiar medidas de complejidad dinámica, como dimensión de correlación, divergencia local o entropía. En tercer lugar, permite formular predicción local: si dos estados reconstruidos son próximos, se puede comparar qué ocurrió después de estados parecidos en el pasado.

La interpretación de esta reconstrucción debe mantenerse dentro de sus límites. En un sistema sintético, como el mapa logístico, la regla generadora es conocida y la reconstrucción puede evaluarse frente a una dinámica controlada. En una serie financiera como la volatilidad realizada de Bitcoin, los parámetros τ y m deben leerse como elecciones prácticas de representación. Su selección puede estar apoyada por información mutua, falsos vecinos y el método de Cao, pero no demuestra que se haya identificado el verdadero atractor de un sistema financiero. En este trabajo, la reconstrucción se utiliza como herramienta de análisis y predicción, no como prueba directa de caos determinista.

3.9– Cuantificación de dinámicas complejas

Una vez reconstruido el espacio de estados, pueden calcularse medidas que intentan describir la geometría, la sensibilidad local o la irregularidad de las trayectorias. Estas medidas proceden de la teoría de sistemas dinámicos y aparecen en la tesis de referencia como parte de la cuantificación de dinámicas complejas (Olmedo Fernández, 2001). En este TFG se utilizan como indicadores exploratorios. Su lectura aislada no permite afirmar caos determinista en una serie financiera, pero sí ayuda a comparar la serie original con versiones de control y a estudiar si existe estructura temporal aprovechable.

La primera familia de medidas describe la geometría de la nube reconstruida. Si se dispone de N puntos $X_1, \dots, X_N$ en el espacio reconstruido, una pregunta natural es cuántos puntos quedan cerca unos de otros al variar una escala de distancia r . La suma de correlación se define como

$$C(r) = \frac{2}{N(N-1)} \sum_{i < j} \mathbf{1} \{ \|X_i - X_j\| < r \}, \quad (3.70)$$

donde $\mathbf{1}\{\cdot\}$ vale 1 si la condición se cumple y 0 en caso contrario. La función $C(r)$ mide la proporción de pares de puntos cuya distancia es menor que r . Si r es pequeño, solo se cuentan vecinos muy próximos; si r aumenta, se incorporan pares cada vez más alejados.

La dimensión de correlación se basa en estudiar cómo crece $C(r)$ cuando aumenta r . Si en un rango de escalas se observa una relación aproximada de tipo potencia,

$$C(r) \propto r^{D_2}, \quad (3.71)$$

entonces el exponente D_2 puede estimarse como la pendiente de la relación entre $\log C(r)$ y $\log r$:

$$D_2 \approx \frac{d \log C(r)}{d \log r}. \quad (3.72)$$

Esta idea fue introducida por Grassberger y Procaccia como una forma de estimar dimensiones asociadas a atractores reconstruidos (Grassberger y Procaccia, 1983). Intuitivamente, si al aumentar el radio aparecen vecinos muy rápido, la nube ocupa un espacio de mayor dimensión efectiva; si aparecen más lentamente, la estructura puede estar más concentrada.

En una aplicación empírica, la estimación de D_2 debe interpretarse con cautela. Para que la pendiente tenga sentido, debería existir una zona de escala en la que la relación logarítmica sea aproximadamente lineal. Si esa zona no aparece con claridad, no es razonable hablar de una dimensión bien determinada. Además, la estimación depende del tamaño muestral, del ruido, de la métrica de distancia, de la elección de escalas y de la correlación temporal entre puntos. Esta última cuestión es relevante: puntos muy cercanos en el tiempo pueden parecer vecinos por continuidad temporal, no por proximidad dinámica. Por ello se suele emplear una ventana de Theiler al calcular vecinos o pares de puntos (Theiler, 1986).

La segunda familia de medidas estudia la sensibilidad local. En un sistema sensible a las condiciones iniciales, dos estados inicialmente próximos pueden separarse de forma rápida. En un espacio reconstruido, si X_i y X_j son dos estados cercanos, puede observarse cómo evoluciona su distancia tras k pasos:

$$d_{ij}(k) = \|X_{i+k} - X_{j+k}\|. \quad (3.73)$$

Si la separación media crece aproximadamente de forma exponencial durante un tramo inicial, puede escribirse

$$d(k) \approx d(0)e^{\lambda k}, \quad (3.74)$$

donde λ representa una tasa media de divergencia. Tomando logaritmos se obtiene

$$\log d(k) \approx \log d(0) + \lambda k. \quad (3.75)$$

Por tanto, λ puede estimarse como la pendiente de la curva media $\log d(k)$ frente a k en una región inicial aproximadamente lineal.

El método de Rosenstein proporciona un procedimiento práctico para estimar el mayor exponente de Lyapunov a partir de series temporales reconstruidas (Rosenstein, Collins, y De Luca, 1993). La idea consiste en buscar vecinos próximos, excluir vecinos temporalmente triviales mediante una ventana de Theiler, seguir la separación entre trayectorias durante varios pasos y ajustar una pendiente en la zona inicial de divergencia. Un valor positivo de la pendiente es compatible con sensibilidad local, pero no constituye por sí solo una prueba de caos en datos financieros. El ruido, los cambios de régimen, la heterocedasticidad o una reconstrucción imperfecta también pueden producir separación rápida entre trayectorias cercanas.

La tercera familia de medidas estudia la irregularidad de la serie. La entropía de permutación mide la diversidad de patrones ordinales observados en una serie temporal (Bandt y Pompe, 2002). En lugar de utilizar directamente los valores numéricos, analiza el orden relativo de grupos de observaciones. Por ejemplo, para tres valores consecutivos (x_t, x_{t+1}, x_{t+2}) , se observa si aparecen en orden creciente, decreciente o en alguna de las restantes ordenaciones posibles. Cada ordenación define un patrón ordinal.

Si $p(\pi)$ es la frecuencia relativa del patrón ordinal π , la entropía de permutación se define como

$$H = - \sum_{\pi} p(\pi) \log p(\pi). \quad (3.76)$$

Para comparar configuraciones con distinto número de patrones posibles, se utiliza la versión normalizada:

$$H_{\text{norm}} = \frac{H}{\log(m!)}, \quad (3.77)$$

donde m es el orden de los patrones ordinales. Valores cercanos a 1 indican que los patrones aparecen con frecuencias similares, lo que corresponde a mayor irregularidad ordinal. Valores más bajos indican que ciertos patrones son más frecuentes que otros y, por tanto, que existe mayor regularidad relativa.

La entropía de permutación tiene varias ventajas prácticas. Es sencilla de calcular, no requiere estimar distancias en espacios de alta dimensión y es relativamente robusta frente a transformaciones monótonas de la serie. Sin embargo, tampoco permite identificar caos por sí sola. Una entropía menor que la de una serie aleatoria puede deberse a dependencia lineal, periodicidad aproximada, cambios de régimen o persistencia de volatilidad. Su valor debe compararse con controles adecuados.

Las series barajadas y las series sustitutas cumplen precisamente esa función de control. Una serie barajada se obtiene permutando aleatoriamente los valores observados. Este procedimiento conserva la distribución marginal de la serie, pero destruye el orden temporal. Si una métrica cambia de forma clara al barajar, puede concluirse que el orden temporal contiene información relevante. Este control es simple, pero no distingue entre dependencia lineal y no lineal.

Las series sustitutas permiten contrastes más exigentes. El método de datos sustitutos fue propuesto como una forma de contrastar si una serie contiene estructura más allá de una hipótesis nula, por ejemplo un proceso lineal transformado estáticamente (Theiler y cols., 1992). Algunas series sustitutas se generan aleatorizando fases en el dominio de Fourier, lo que conserva aproximadamente la estructura espectral lineal y destruye relaciones de fase. Otras, como AAFT, intentan conservar tanto la distribución marginal como una aproximación a la estructura lineal de la serie. Revisiones posteriores, como la de Schreiber y Schmitz, insisten en que estos contrastes deben interpretarse atendiendo a la hipótesis nula que conserva cada tipo de sustituto (Schreiber y Schmitz, 2000).

La lógica de estos controles puede resumirse de forma escalonada. Si la serie original se diferencia de sus versiones barajadas, hay evidencia de que el orden temporal importa. Si además se diferencia de series sustitutas que conservan parte de la estructura lineal, la evidencia apunta a una organización temporal que no se explica solo por la distribución marginal ni por autocorrelación lineal. Si no se observa esa separación, parte de los resultados puede estar explicada por dependencia lineal, heterocedasticidad o propiedades de segundo orden.

En conjunto, dimensión de correlación, divergencia local, entropía de permutación y contrastes con series de control permiten caracterizar la serie reconstruida desde varias perspectivas. La dimensión de correlación estudia geometría; el exponente de Lyapunov aproximado estudia sensibilidad local; la entropía de permutación estudia regularidad ordinal; y las series barajadas y sustitutas ayudan a interpretar si los valores obtenidos dependen del orden temporal. En una serie financiera, estas herramientas deben leerse como indicios complementarios, no como pruebas concluyentes de caos determinista.

3.10– Predicción local y modelos de referencia

La reconstrucción del espacio de estados también puede utilizarse con fines predictivos. La idea procede de la interpretación de la predicción como una forma indirecta de caracterizar una serie temporal: si la representación reconstruida conserva información útil sobre la dinámica, entonces estados cercanos deberían tener, al menos a corto plazo, evoluciones futuras parcialmente parecidas (Olmedo Fernández, 2001; Kantz y Schreiber, 2003; Abarbanel, 1996; Farmer y Sidorowich, 1987). Esta hipótesis no exige afirmar que el sistema sea caótico. Basta con que el espacio reconstruido agrupe situaciones con comportamiento posterior similar.

Sea X_t el estado reconstruido en el instante t , construido a partir de retardos de la serie observada. Si se desea predecir una variable futura y_t , el enfoque local busca en el conjunto de entrenamiento estados X_j próximos a X_t . La proximidad suele medirse mediante una distancia, por ejemplo la distancia euclídea:

$$d(X_t, X_j) = \|X_t - X_j\|_2. \quad (3.78)$$

A partir de esa distancia se selecciona el conjunto $N_k(X_t)$, formado por los k vecinos más cercanos de X_t dentro de los datos disponibles para entrenamiento.

El predictor local más sencillo asigna como predicción la media de los valores futuros observados después de esos vecinos. Si y_j representa el valor futuro asociado al estado vecino X_j , la predicción viene dada por

$$\hat{y}_t = \frac{1}{k} \sum_{j \in N_k(X_t)} y_j. \quad (3.79)$$

Esta formulación tiene una interpretación directa: para estimar qué ocurrirá después del estado actual, se buscan estados históricos parecidos y se promedia lo que ocurrió después de ellos. También existen variantes ponderadas, en las que los vecinos más cercanos reciben más peso:

$$\hat{y}_t = \sum_{j \in N_k(X_t)} w_j y_j, \quad \sum_{j \in N_k(X_t)} w_j = 1. \quad (3.80)$$

En este trabajo el enfoque principal se basa en medias locales simples.

El parámetro k controla el compromiso entre localidad y estabilidad. Con un valor pequeño de k , la predicción se apoya en pocos estados muy próximos. Esto puede captar relaciones locales finas, pero también aumenta la sensibilidad al ruido o a observaciones atípicas. Con un valor grande de k , la predicción se vuelve más estable, ya que promedia más casos, pero pierde especificidad local. Este comportamiento refleja el equilibrio entre varianza y sesgo: usar pocos vecinos reduce el sesgo local, pero puede aumentar la variabilidad de la estimación; usar muchos vecinos reduce la variabilidad, pero puede mezclar estados que no son realmente equivalentes.

La predicción local depende de la calidad de la reconstrucción. Si los parámetros τ y m producen una representación informativa, los vecinos en el espacio reconstruido pueden tener valor predictivo. Si la reconstrucción está dominada por ruido, por una dimensión excesiva o por cambios de régimen, la noción de vecino pierde utilidad. En espacios de dimensión alta aparece además el problema de la dispersión de los datos: las distancias tienden a ser menos informativas porque los puntos quedan más separados entre sí. Por este motivo, la dimensión de reconstrucción debe interpretarse también desde el punto de vista predictivo, no solo geométrico.

Para evitar comparaciones triviales, en predicción local se puede aplicar una ventana de Theiler o una exclusión temporal equivalente. Si se permite elegir como vecino un punto inmediatamente anterior o posterior al instante evaluado, la predicción puede beneficiarse de observaciones casi idénticas por continuidad temporal o por solapamiento de ventanas. Para evitarlo, se excluyen candidatos X_j tales que

$$|t - j| \leq W, \quad (3.81)$$

donde W es el tamaño de la ventana de exclusión. Esta precaución es especialmente relevante en series de alta frecuencia y en variables como la volatilidad realizada, donde las ventanas consecutivas comparten gran parte de sus observaciones.

La evaluación de un predictor local debe realizarse fuera de muestra. El conjunto de entrenamiento se utiliza para almacenar los estados históricos y sus valores futuros asociados. El conjunto de validación puede emplearse para seleccionar parámetros como k , τ o m . El conjunto de prueba debe reservarse para medir el rendimiento final. Esta separación temporal evita que el modelo utilice información futura de forma directa o indirecta.

Para valorar si la predicción local aporta información, debe compararse con modelos de referencia. La referencia más simple es la media histórica del entrenamiento:

$$\hat{y}_t = \bar{y}_{\text{train}}. \quad (3.82)$$

Este modelo no utiliza el estado actual; predice siempre el nivel medio observado en el tramo de entrenamiento. Aunque es muy simple, resulta útil como punto mínimo de comparación.

Otra referencia habitual en series persistentes es la persistencia. En su forma básica, asume que el valor futuro será igual al último valor disponible:

$$\hat{y}_{t+h} = x_t. \quad (3.83)$$

En la aplicación a volatilidad realizada, esta referencia equivale a usar la volatilidad realizada reciente como predicción de la volatilidad futura. Si una serie tiene fuerte persistencia, esta referencia puede ser difícil de superar.

También se emplean modelos autorregresivos lineales como referencia. Un modelo $AR(p)$ tiene la forma

$$x_t = c + \sum_{i=1}^p \phi_i x_{t-i} + \varepsilon_t. \quad (3.84)$$

Este modelo captura dependencia lineal en media mediante una combinación de retardos. Su papel en este trabajo es servir como comparación frente a los métodos locales en el espacio reconstruido. Si un método no lineal no mejora a una referencia autorregresiva, su ventaja predictiva no queda justificada aunque pueda ser útil como herramienta de análisis.

El rendimiento predictivo se mide comparando los valores observados y_i con las predicciones $\hat{y}_i$. Una métrica habitual es el error absoluto medio:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (3.85)$$

El MAE mide el tamaño medio del error en la misma escala que la variable objetivo. Al usar valor absoluto, penaliza de forma lineal las desviaciones.

Otra métrica frecuente es la raíz del error cuadrático medio:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (3.86)$$

El $RMSE$ penaliza más los errores grandes porque eleva las diferencias al cuadrado antes de promediarlas. Por esta razón puede ser más sensible a episodios extremos que el MAE .

También puede utilizarse un coeficiente de determinación fuera de muestra. En este trabajo se expresa como

$$R_{\text{OOS}}^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_{\text{train}})^2}. \quad (3.87)$$

El denominador compara contra una predicción constante basada en la media del entrenamiento. Si $R_{\text{OOS}}^2 > 0$, el modelo mejora esa referencia constante. Si $R_{\text{OOS}}^2 < 0$, el modelo predice peor que usar la media histórica del entrenamiento.

Una mejora predictiva moderada no prueba caos determinista. Indica que el modelo ha capturado alguna regularidad útil bajo una partición temporal concreta. En datos financieros, esa regularidad puede proceder de persistencia, heterocedasticidad, memoria multiescala o cambios de régimen. Por ello, la predicción local se interpreta junto con los modelos de referencia y con el resto de indicadores del análisis. Su valor principal es comprobar si la reconstrucción del espacio de estados contiene información práctica para anticipar parcialmente la volatilidad futura.

3.11– Modelo HAR-logRV para volatilidad realizada

El modelo HAR, acrónimo de *Heterogeneous Autoregressive model*, es un modelo diseñado para describir y predecir volatilidad realizada a partir de información agregada en varias escalas temporales. Fue propuesto originalmente para capturar la persistencia de la volatilidad mediante una estructura sencilla e interpretable (Corsi, 2009). Su motivación está relacionada con la heterogeneidad de horizontes en los mercados financieros: distintos participantes operan y reaccionan en escalas temporales diferentes (Müller y cols., 1997). Algunos agentes reaccionan a movimientos muy recientes, mientras que otros toman decisiones considerando horizontes más amplios. Como consecuencia, la volatilidad futura puede depender simultáneamente de medidas de volatilidad de corto, medio y mayor plazo. El modelo HAR no pertenece a la familia de métodos de dinámica caótica. Tampoco reconstruye un espacio de estados ni busca identificar un atractor. Su función en este trabajo es distinta: proporcionar un modelo de referencia multiescala para volatilidad realizada. Esta diferencia es relevante porque permite separar dos objetivos. Por un lado, el análisis de reconstrucción y cuantificación estudia si la serie presenta estructura compatible con dinámica compleja. Por otro, HAR-logRV ofrece una formulación práctica para explotar la persistencia temporal de la volatilidad.

En su forma general, un modelo HAR para volatilidad realizada expresa la volatilidad futura como combinación lineal de volatilidades realizadas calculadas en distintas ventanas. En este TFG se trabaja con la transformación logarítmica de la volatilidad realizada. Siguiendo la notación introducida anteriormente, sea

$$v_t^{(h)} = \log \left(RV_t^{(h)} + \varepsilon \right) \quad (3.88)$$

la volatilidad realizada pasada transformada logarítmicamente para una ventana de h observaciones. El uso de volatilidad realizada se apoya en la literatura de datos financieros de alta frecuencia, donde esta magnitud se emplea como medida observable de variabilidad y como base para modelos de predicción (Andersen y cols., 2003). La variable objetivo para el horizonte principal se denota por

$$y_t^{(12)} = \log \left(RV_{t,+}^{(12)} + \varepsilon \right), \quad (3.89)$$

donde $RV_{t,+}^{(12)}$ representa la volatilidad realizada de las doce velas posteriores al instante t .

Adaptado a datos de cinco minutos, el modelo HAR-logRV utilizado en este trabajo puede escribirse como

$$y_t^{(12)} = \beta_0 + \beta_1 v_t^{(12)} + \beta_2 v_t^{(48)} + \beta_3 v_t^{(288)} + u_t. \quad (3.90)$$

En esta ecuación, $v_t^{(12)}$ resume la volatilidad realizada de la última hora, $v_t^{(48)}$ resume la volatilidad realizada de las últimas cuatro horas y $v_t^{(288)}$ resume la volatilidad realizada del último día completo. El término u_t recoge la parte de la volatilidad futura que no queda explicada por esas tres escalas.

La elección de estas ventanas responde a la frecuencia de los datos. Al trabajar con velas de cinco minutos, doce observaciones equivalen a una hora:

$$12 \times 5 \text{ minutos} = 60 \text{ minutos}. \quad (3.91)$$

Del mismo modo, cuarenta y ocho observaciones equivalen a cuatro horas, y 288 observaciones equivalen a un día:

$$48 \times 5 \text{ minutos} = 240 \text{ minutos}, \quad 288 \times 5 \text{ minutos} = 1440 \text{ minutos}. \quad (3.92)$$

Estas escalas permiten representar memoria reciente, memoria intradía más amplia y memoria diaria dentro de un modelo lineal compacto.

Los coeficientes del modelo tienen una interpretación directa. El parámetro β_1 mide la relación entre la volatilidad realizada más reciente y la volatilidad futura. El coeficiente β_2 recoge la contribución de una ventana intradía más amplia. El coeficiente β_3 incorpora información del último día. Si estos coeficientes son positivos, la interpretación habitual es que niveles altos de volatilidad pasada en esas escalas se asocian con niveles altos de volatilidad futura.

El ajuste del modelo puede realizarse mediante mínimos cuadrados ordinarios. Dado un conjunto de observaciones de entrenamiento, los coeficientes se eligen minimizando la suma de errores cuadráticos:

$$\min_{\beta_0, \beta_1, \beta_2, \beta_3} \sum_{t \in \mathcal{T}_{\text{train}}} \left(y_t^{(12)} - \beta_0 - \beta_1 v_t^{(12)} - \beta_2 v_t^{(48)} - \beta_3 v_t^{(288)} \right)^2. \quad (3.93)$$

La estimación debe hacerse únicamente con datos de entrenamiento. Una vez ajustados los coeficientes, el modelo se evalúa sobre tramos posteriores sin utilizar información futura en el ajuste.

El modelo HAR-logRV se diferencia de un modelo autorregresivo clásico en la forma de resumir el pasado. Un modelo AR(p) utiliza valores retardados concretos:

$$x_t = c + \sum_{i=1}^p \phi_i x_{t-i} + \varepsilon_t. \quad (3.94)$$

En cambio, HAR utiliza agregados de volatilidad sobre ventanas de distinta longitud. Esta decisión reduce el número de parámetros y facilita la interpretación. En lugar de introducir muchos retardos individuales, el modelo resume la información pasada en escalas temporales relevantes.

Esta simplicidad es una de las razones por las que HAR-logRV resulta adecuado como cierre predictivo práctico. El modelo es rápido de ajustar, sus coeficientes son interpretables y su implementación en una aplicación web es directa. Además, al trabajar con variables ya construidas a partir de volatilidad realizada, mantiene una conexión clara con el fenómeno financiero estudiado: la persistencia de la variabilidad.

El papel de HAR-logRV en esta memoria debe entenderse con precisión. No se utiliza como evidencia de caos ni como sustituto del análisis no lineal. Se utiliza como modelo práctico de comparación frente a persistencia, modelos autorregresivos y predicción local en el espacio reconstruido. Sus resultados empíricos se discuten en capítulos posteriores. En este marco teórico basta con fijar su significado: un modelo lineal multiescala para volatilidad realizada, útil por su equilibrio entre sencillez, interpretabilidad y capacidad predictiva.

3.12– Relación con la tesis de referencia

La tesis de Olmedo Fernández proporciona la referencia metodológica principal de este trabajo (Olmedo Fernández, 2001). Su estructura recorre los conceptos de determinismo, azar, complejidad y caos; introduce los sistemas dinámicos; desarrolla herramientas de caracterización de series temporales; presenta la reconstrucción del espacio de estados; estudia estacionariedad y no linealidad; y aborda la cuantificación y la predicción en una aplicación económica. Esta secuencia sirve como guía para ordenar el marco teórico y el procedimiento de análisis desarrollado en esta memoria.

La relación con la tesis no debe entenderse como una reproducción directa. El objeto empírico, la frecuencia de los datos y el enfoque computacional son distintos. La tesis aplica las herramientas a una serie mensual de desempleo en España. Este TFG toma esa lógica metodológica y la aplica primero a una comprobación sintética mediante el mapa logístico, donde la dinámica generadora

es conocida. A partir de esa validación controlada, el procedimiento se adapta después al estudio de la volatilidad realizada intradía de BTCUSDT, construida a partir de velas de cinco minutos.

También cambia el grado de cautela en la interpretación. En la tesis de referencia, los resultados empíricos permiten formular conclusiones relativamente fuertes sobre la existencia de indicios de dinámica caótica en la serie estudiada. En este TFG, la serie de Bitcoin se trata de forma más conservadora. Se analizan dependencia temporal, no linealidad, heterocedasticidad, estructura reconstruida y utilidad predictiva parcial, pero no se afirma que la volatilidad realizada de Bitcoin esté generada por un sistema caótico determinista.

Elemento en Olmedo	Adaptación en este TFG
Marco de complejidad, no linealidad y caos	Lectura prudente orientada a indicios, sin afirmar caos determinista en BTC
Caracterización de series temporales	Validación de datos, visualización, estadísticos, ACF, PACF, periodograma y recurrencia sobre volatilidad realizada
Estacionariedad y no linealidad	ADF, KPSS, filtrado autorregresivo, Ljung-Box sobre residuos y cuadrados, ARCH-LM y BDS
Reconstrucción por retardos	Selección operativa de τ y m con información mutua, falsos vecinos y método de Cao
Cuantificación dinámica	Dimensión de correlación, divergencia local y entropía de permutación comparadas con series barajadas y series sustitutas
Predicción local	kNN en el espacio reconstruido, con separación temporal, validación de parámetros y exclusión de vecinos triviales
Aplicación empírica económica	Aplicación financiera de alta frecuencia sobre volatilidad realizada de Bitcoin
Extensión propia	Validación sintética con mapa logístico, modelo HAR-logRV y MVP web autónomo

Cuadro 3.1: Relación entre la tesis de referencia y la adaptación realizada en este trabajo.

La adaptación introduce elementos propios de un proyecto de ingeniería del software. El análisis se organiza como un procedimiento reproducible, con validación explícita de datos, separación temporal entre entrenamiento, validación y prueba, controles para evitar fuga de información, generación de artefactos y comparación sistemática de modelos. A esto se añade un MVP web independiente del repositorio de investigación, diseñado para ejecutar predicciones de volatilidad y visualizar resultados de forma interactiva.

Por último, se incorpora un cierre predictivo con HAR-logRV. Este modelo no procede de la tesis de Olmedo, sino de la literatura específica sobre volatilidad realizada. Su papel en el TFG es práctico: ofrecer una referencia multiescala sencilla, interpretable y exportable al prototipo. De este modo, el trabajo mantiene la lógica metodológica de la tesis, pero la adapta a un caso financiero de alta frecuencia y a una implementación software autónoma.

3.13– Aportación realizada respecto a los antecedentes

La aportación de este TFG no consiste en proponer una nueva teoría de caos ni en establecer una propiedad fuerte sobre Bitcoin. Su contribución está en adaptar una metodología de análisis de series temporales mediante herramientas de dinámica no lineal a un caso financiero de alta frecuencia, manteniendo una interpretación prudente de los resultados.

Desde el punto de vista metodológico, el trabajo integra en un único procedimiento las fases necesarias para estudiar una serie temporal real: validación de datos, construcción de variables, caracterización lineal, análisis de estacionariedad, contrastes de dependencia, reconstrucción por retardos, cuantificación exploratoria, comparación con series barajadas y sustitutas, y evaluación predictiva. Esta secuencia evita aplicar directamente indicadores de caos sin haber controlado antes problemas básicos como calidad del dato, dependencia lineal, heterocedasticidad o fuga de información.

Una aportación relevante es la comprobación sintética con el mapa logístico. La tesis de referencia utiliza este sistema como ejemplo teórico dentro del marco de sistemas dinámicos; en este trabajo se convierte en una fase experimental separada. Al aplicar el mismo procedimiento a una serie generada por una dinámica conocida, se comprueba si las herramientas responden de forma coherente en un entorno controlado. La comparación posterior con Bitcoin ayuda a distinguir entre limitaciones del procedimiento y dificultades propias de los datos financieros reales.

La aplicación a Bitcoin introduce una diferencia sustancial respecto a la tesis de referencia. La serie analizada no es una magnitud macroeconómica mensual, sino una variable financiera intradía construida a partir de datos de cinco minutos. La variable principal es la volatilidad realizada logarítmica, no el precio. Esta elección desplaza el problema desde la predicción direccional hacia el estudio de la intensidad de las fluctuaciones, una magnitud que suele presentar mayor persistencia temporal.

Otro elemento diferencial es la comparación explícita con modelos de referencia. La predicción local en el espacio reconstruido se evalúa frente a persistencia, media histórica, modelos autorregresivos y HAR-logRV. Esta comparación es necesaria para no atribuir valor predictivo a un método complejo si modelos más simples explican una parte similar de la estructura. En particular, HAR-logRV actúa como cierre práctico del bloque predictivo: no forma parte del análisis caótico, pero resume de forma eficiente la memoria multiescala de la volatilidad realizada.

El trabajo también aporta una dimensión software que no está presente en la tesis de referencia. El MVP web convierte parte del estudio en una herramienta demostrable. La aplicación integra carga de datos, cálculo de variables, ejecución de modelos y visualización de predicciones de volatilidad. Su diseño mantiene independencia respecto al repositorio de investigación y utiliza artefactos exportados, lo que facilita separar el análisis experimental de la ejecución operativa.

Las limitaciones de esta aportación también forman parte del resultado. El prototipo no proporciona asesoramiento financiero, no predice la dirección del mercado, no incorpora intervalos de confianza y no demuestra caos. Su función es mostrar que el procedimiento desarrollado puede trasladarse a una aplicación autónoma, comprensible y verificable. Con ello, el TFG combina tres planos: marco matemático, validación empírica y construcción de software.

En conjunto, la aportación respecto a los antecedentes consiste en una adaptación prudente y reproducible de herramientas de dinámica no lineal a volatilidad financiera de alta frecuencia. La memoria queda así preparada para los capítulos posteriores: primero se presentará el procedimiento implementado, después los resultados obtenidos y finalmente la aplicación web que materializa la parte predictiva del estudio.

Planificación, metodología de trabajo y costes

4.1– Organización general del proyecto

El proyecto se organizó en torno a cuatro fases principales. La primera correspondió a la formación inicial y al estudio de la tesis de referencia, necesaria para entender el enfoque metodológico del trabajo. La segunda agrupó el estudio técnico y experimental, desarrollado en el repositorio `btc-volatility`. La tercera consistió en la implementación del MVP web, ubicado en el repositorio `mvp-web`. La cuarta fase correspondió a la redacción, revisión y preparación de la memoria en $\text{\LaTeX}$, mantenida en el repositorio `tfg-memoria`.

Esta división permitió separar el aprendizaje inicial, el trabajo experimental, la aplicación software y la documentación académica. El repositorio técnico concentró los scripts, informes, tablas y figuras generados durante el análisis. El MVP web consumió una parte de los artefactos exportados desde el estudio, manteniéndose desacoplado del repositorio experimental. Finalmente, la memoria integró la motivación, los objetivos, el marco teórico, la planificación, el diseño, la implementación, los resultados y las conclusiones del trabajo.

4.2– Fases principales del trabajo

La primera fase fue de formación y lectura. En ella se estudió la tesis de referencia y se revisaron los conceptos necesarios para adaptar su enfoque al contexto del TFG. Esta fase permitió delimitar el problema, entender las herramientas principales y decidir cómo trasladar una metodología de análisis a un caso financiero actual.

La segunda fase fue el estudio técnico. En ella se construyó el procedimiento experimental sobre datos de Bitcoin. También se incorporó la comprobación sintética con el mapa logístico, utilizada como control metodológico del procedimiento.

La tercera fase fue la implementación del MVP web. Su objetivo fue trasladar una parte del estudio técnico a una aplicación interactiva. Para ello se diseñó una herramienta capaz de cargar datos, validarlos, calcular variables, ejecutar modelos disponibles y mostrar predicciones de volatilidad junto con información de diagnóstico.

La cuarta fase fue la escritura y revisión de la memoria. Esta fase incluyó la redacción de capítulos, integración de tablas y figuras, revisión de coherencia entre secciones, depuración de

repeticiones y preparación final del documento. Se considera una fase propia porque la memoria es uno de los entregables principales del proyecto y requiere una dedicación significativa.

4.3– Planificación temporal

La planificación inicial se realizó tomando como referencia una dedicación total de 300 horas. La estimación se agrupó en las cuatro fases principales del proyecto, como se muestra en la Tabla 4.1.

Fase de trabajo	Horas planificadas
Formación y lectura de la tesis de referencia	60
Estudio técnico y experimental	160
Implementación del MVP web	50
Escritura, revisión y preparación de la memoria	30
Total	300

Cuadro 4.1: Distribución inicial de horas planificadas.

La fase de estudio técnico concentra la mayor parte de la planificación, ya que incluye la construcción del procedimiento experimental completo. En ella se agrupan la preparación de datos, el análisis de la serie, la comprobación sintética, la comparación de modelos y la lectura técnica de resultados. La implementación del MVP se planificó como una fase posterior, una vez disponibles los artefactos principales del estudio.

4.4– Distribución del esfuerzo

Durante el desarrollo se registró el tiempo dedicado a las distintas tareas mediante Clockify. Para comparar la planificación inicial con el esfuerzo real, las tareas registradas se agruparon en las mismas cuatro fases principales. Esta agrupación evita una fragmentación excesiva y permite interpretar mejor dónde se concentró la dedicación del proyecto.

Fase de trabajo	Planificado	Real	Desviación
Formación y lectura de la tesis de referencia	60	–	–
Estudio técnico y experimental	160	–	–
Implementación del MVP web	50	–	–
Escritura, revisión y preparación de la memoria	30	–	–
Total	300	–	–

Cuadro 4.2: Comparación entre esfuerzo planificado y esfuerzo real registrado.

Los valores reales se incorporan a partir de la exportación final de Clockify. La comparación permite identificar desviaciones respecto a la planificación inicial y valorar qué partes del proyecto requirieron mayor esfuerzo del previsto.

4.5– Herramientas de seguimiento y desarrollo

El seguimiento temporal se realizó mediante Clockify, registrando las tareas según las fases principales del proyecto. Esta información permitió mantener una visión cuantitativa del esfuerzo dedicado y preparar la comparación entre planificación inicial y dedicación real.

El desarrollo técnico se organizó en repositorios separados. El procedimiento experimental se implementó principalmente en Python, generando salidas reutilizables en formatos como CSV, JSON, NPZ y figuras. El MVP web se desarrolló como aplicación independiente. La memoria se redactó en $\text{\LaTeX}$, lo que facilitó la organización por capítulos, la inclusión de ecuaciones, tablas y figuras, y la gestión de referencias.

4.6– Estimación de costes

La estimación de costes considera principalmente el coste de personal, dado que las herramientas utilizadas fueron mayoritariamente de código abierto y no requirieron licencias comerciales. El coste personal se modela mediante la expresión:

$$C_{\text{personal}} = H \cdot c_h, \quad (4.1)$$

donde H representa el número de horas dedicadas y c_h el coste horario estimado. Para la planificación inicial se consideraron 300 horas de trabajo. Tomando como referencia un coste horario de 15 €/h, el coste planificado del proyecto es:

$$C_{\text{planificado}} = 300 \cdot 15 = 4500 \text{ €}. \quad (4.2)$$

El coste real se obtiene sustituyendo H por el número final de horas registradas:

$$C_{\text{real}} = H_{\text{real}} \cdot 15. \quad (4.3)$$

Concepto	Coste estimado
Coste de personal planificado	4500 €
Software y librerías de desarrollo	0 €
Servicios externos y APIs	0 €
Infraestructura adicional	0 €
Total planificado	4500 €

Cuadro 4.3: Estimación inicial de costes del proyecto.

4.7– Desviaciones respecto a la planificación inicial

La dedicación real del proyecto se comparará con la planificación inicial una vez cerrado el registro final de horas. Las desviaciones se analizarán agrupando el esfuerzo en las cuatro fases principales: formación, estudio técnico, implementación del MVP y escritura de la memoria.

En caso de desviación positiva, esta se interpretará atendiendo al alcance real del trabajo. En particular, el estudio técnico pudo requerir más esfuerzo por la necesidad de validar datos, contrastar resultados, comparar modelos y generar artefactos reproducibles. La implementación del MVP también pudo incrementar la dedicación al exigir integración de modelos, validación de entradas y diseño de una interfaz utilizable.

La escritura de la memoria se considera parte del esfuerzo del proyecto porque no se limita a transcribir resultados. Incluye ordenar el trabajo realizado, seleccionar qué información debe aparecer en cada capítulo, integrar figuras y tablas, revisar la coherencia interna y asegurar que las conclusiones respeten el alcance real del análisis.

Análisis de requisitos

Este capítulo recoge las condiciones que debe cumplir el sistema desarrollado. En este trabajo, los requisitos no se limitan a la aplicación web: afectan también al procedimiento de análisis, a la comprobación sintética, a la aplicación empírica sobre Bitcoin y a la forma en que se comunican los resultados. La finalidad del capítulo es fijar qué debe garantizarse para que el estudio sea reproducible, interpretable y coherente con los objetivos planteados.

5.1– Requisitos generales del sistema

El sistema debe permitir analizar una serie financiera de volatilidad mediante un procedimiento reproducible. Para ello debe cubrir el flujo completo de trabajo: validación de datos, construcción de variables, caracterización de la serie, comparación de modelos, generación de resultados y exportación de artefactos reutilizables.

El sistema debe mantener separadas sus piezas principales: el procedimiento experimental, la comprobación sintética, la aplicación empírica a Bitcoin y el MVP web. Esta separación es necesaria porque cada parte tiene un alcance distinto.

La variable central del estudio debe representar intensidad de movimiento, no dirección de precio. Por ello, el sistema trabaja con volatilidad realizada construida a partir de retornos logarítmicos. El objetivo predictivo principal es la volatilidad realizada futura de una hora, expresada en escala logarítmica.

5.2– Requisitos conceptuales y de comunicación

La interpretación de los resultados debe distinguir entre no linealidad, complejidad, caos y predictibilidad. La presencia de dependencia temporal, heterocedasticidad, estructura ordinal o divergencia local de trayectorias no permite afirmar automáticamente que exista caos determinista.

Las salidas del procedimiento deben leerse como estimaciones de volatilidad realizada, no como señales direccionales de mercado. El sistema no predice si el precio de Bitcoin subirá o bajará, y no debe presentarse como herramienta de inversión.

El MVP debe mantener visibles estas limitaciones. La interfaz debe indicar que la predicción

se refiere a intensidad esperada de movimiento en una ventana temporal concreta y no a rentabilidad, dirección del precio ni demostración de caos en Bitcoin.

5.3– Requisitos de datos

El análisis requiere una serie temporal limpia, ordenada y con frecuencia homogénea. Los datos de entrada deben contener, como mínimo, marca temporal y precio de cierre. Para el estudio empírico se utilizan velas de cinco minutos de BTCUSD; cualquier ventana usada por el MVP debe poder normalizarse al mismo esquema básico.

El procedimiento debe validar frecuencia temporal, orden cronológico, duplicados, huecos, nulos, conversiones numéricas y valores de mercado no válidos. Si estos requisitos no se cumplen, el análisis debe detenerse o marcar la ventana como no fiable, en lugar de generar predicciones sin base suficiente.

Las variables pasadas y futuras deben construirse de forma separada. Las variables pasadas solo pueden usar información disponible hasta el instante t . Las variables futuras se reservan para evaluar errores. Esta separación es imprescindible para evitar fuga de información.

Toda normalización debe ajustarse con datos de entrenamiento. En particular, la estandarización de la serie principal no debe utilizar información de validación ni de test.

5.4– Requisitos metodológicos

La metodología debe avanzar de herramientas simples a herramientas más específicas, siguiendo un orden progresivo: primero caracterizar la serie y después aplicar herramientas más específicas. Esta progresión evita interpretar patrones complejos sin haber comprobado antes el comportamiento básico de los datos.

La elección de la serie principal debe quedar justificada frente a alternativas más directas, como precio, log-precio, retorno firmado o retorno absoluto. El procedimiento debe mostrar por qué la volatilidad realizada logarítmica es la magnitud adecuada para estudiar persistencia, estructura temporal y utilidad predictiva.

El análisis debe incluir referencias lineales y persistentes antes de interpretar cualquier posible componente no lineal. Los modelos locales en espacio reconstruido deben compararse con referencias básicas, como media histórica, persistencia y modelos autorregresivos.

Los indicadores dinámicos deben contrastarse con controles. La comparación con series barajadas y subrogadas es necesaria para no atribuir a dinámica no lineal lo que pueda deberse a estructura lineal, propiedades espectrales, distribución marginal o cambios de régimen.

5.5– Requisitos de la comprobación sintética

La comprobación sintética debe utilizar un sistema conocido, independiente de Bitcoin, que permita evaluar el comportamiento del procedimiento en condiciones controladas. El mapa logístico en régimen caótico cumple esta función porque ofrece una dinámica determinista, no lineal y explícita.

El procedimiento aplicado al sistema sintético debe ser comparable al aplicado a la serie financiera: selección de retardo, selección de dimensión, reconstrucción, cuantificación y predicción

local. También debe incluir referencias simples o lineales para comprobar si el método local aporta ventaja cuando la relación subyacente es no lineal.

Los resultados del mapa logístico no deben trasladarse como conclusión sobre Bitcoin. Su función es comprobar que el procedimiento responde ante una dinámica conocida y proporcionar un punto de comparación para interpretar después la serie financiera real.

5.6– Requisitos funcionales del procedimiento de análisis

El procedimiento de análisis debe cubrir el flujo completo desde los datos brutos hasta los artefactos finales. Sus funciones principales son:

- cargar y validar los datos de mercado;
- construir retornos, volatilidades realizadas, variables pasadas, targets futuros y transformaciones logarítmicas;
- generar tablas, figuras e informes de caracterización;
- aplicar filtros lineales y contrastes de dependencia residual;
- reconstruir el espacio de estados con orientación causal;
- calcular indicadores dinámicos y compararlos con controles;
- evaluar modelos predictivos con particiones temporales;
- exportar artefactos reutilizables para la memoria y MVP.

Cada fase debe dejar salidas persistidas en formatos reutilizables. Las tablas, figuras, informes y artefactos permiten vincular las afirmaciones del trabajo con una ejecución concreta del código.

En las fases predictivas, el procedimiento debe impedir cualquier uso indebido del futuro. La selección de configuraciones debe realizarse en validación y la evaluación final debe reservarse para test. En métodos de vecinos, los candidatos deben pertenecer únicamente al histórico permitido y respetar la ventana de exclusión temporal definida.

5.7– Requisitos funcionales del MVP web

El MVP web debe ofrecer una interfaz para aplicar los modelos principales sobre datos recientes, un dataset de ejemplo o un CSV cargado por el usuario.

La aplicación debe aceptar tres fuentes de datos: Binance API, dataset de ejemplo y CSV de usuario. En todos los casos debe normalizar los datos, validar la ventana temporal, calcular las variables necesarias y determinar qué modelos pueden ejecutarse con la información disponible.

El MVP debe generar predicciones de volatilidad futura a una hora. También debe indicar el horizonte temporal, la disponibilidad de cada modelo, diagnósticos básicos de fiabilidad y opciones de descarga de resultados.

La aplicación debe distinguir entre modelos basados en artefactos históricos exportados y modelos ligeros recalculados sobre la ventana cargada. En particular, los modelos históricos no

deben reentrenarse desde la interfaz web, mientras que el Mini-HAR local puede recalibrarse con los datos disponibles en tiempo de uso como estimación experimental.

El MVP debe permitir una evaluación reciente sobre la ventana cargada cuando existan datos suficientes. Esta evaluación debe utilizar el objetivo futuro solo para medir errores pasados y no para generar la predicción actual. Además, debe presentarse como diagnóstico local de la ventana cargada, no como sustituto de la validación histórica formal.

La aplicación debe funcionar de forma autónoma, para ello debe consumir artefactos previamente exportados, sin importar directamente código del repositorio de investigación.

5.8– Requisitos no funcionales

El requisito no funcional principal es la reproducibilidad. El análisis debe poder rehacerse a partir de los datos, scripts y parámetros guardados, generando de nuevo tablas, figuras y métricas.

El sistema debe ser modular. El repositorio de investigación, el MVP y la memoria cumplen funciones distintas, por lo que no deben depender unos de otros de forma innecesaria. Esta separación reduce acoplamiento y evita que la aplicación web altere la validez del estudio experimental.

La eficiencia computacional debe abordarse de forma práctica. Cuando una herramienta no sea viable sobre toda la muestra, el procedimiento puede utilizar ventanas representativas, submuestras equiespaciadas o bloques temporales, siempre que la decisión quede documentada.

El MVP debe ser robusto frente a datos insuficientes o mal formados. Si no hay información suficiente para calcular una variable o ejecutar un modelo, la aplicación debe indicarlo explícitamente en lugar de producir una salida forzada.

5.9– Criterios de validación

Los requisitos se consideran satisfechos si se cumplen los siguientes criterios:

- Los datos de entrada están ordenados, tienen frecuencia válida y no contienen duplicados.
- Las variables pasadas y futuras se construyen sin fuga de información.
- La normalización y el ajuste de modelos respetan la separación entre entrenamiento y test.
- La serie principal queda justificada frente a alternativas más simples.
- Los indicadores dinámicos se comparan con controles adecuados.
- La predicción se evalúa fuera de muestra frente a referencias básicas, lineales y multiescala.
- La comprobación sintética muestra el comportamiento del procedimiento en una dinámica conocida sin trasladar sus conclusiones a Bitcoin.
- El MVP carga datos, valida entradas, calcula variables, ejecuta modelos disponibles, muestra predicciones, informa de limitaciones y permite descargar resultados.
- El MVP distingue entre modelos históricos exportados y modelos recalculados localmente.

Diseño de la solución

Este capítulo describe la solución desde el punto de vista del diseño. El objetivo es fijar la estructura lógica del sistema: sus componentes principales, la relación entre ellos, el flujo de datos y las restricciones que permiten separar el procedimiento metodológico, la comprobación sintética, la aplicación empírica y el MVP web.

6.1– Visión general de la solución propuesta

La solución se plantea como un sistema de análisis reproducible de series temporales financieras. Su núcleo es un procedimiento metodológico para construir una serie de volatilidad realizada, caracterizar su estructura temporal, compararla con referencias adecuadas y evaluar su utilidad predictiva. A partir de este núcleo se integran dos elementos adicionales: una comprobación sintética con el mapa logístico, utilizada como control metodológico, y un MVP web que consume artefactos generados por el estudio.

El objeto de estudio es la volatilidad realizada intradía de Bitcoin. Los datos de entrada son velas de cinco minutos del par BTCUSDT. A partir del precio de cierre se calculan retornos logarítmicos y ventanas de volatilidad realizada. La serie principal es `log_rv_past_12`, que resume la volatilidad realizada de la última hora en escala logarítmica. El objetivo predictivo principal es `log_rv_future_12`, correspondiente a la volatilidad realizada de la siguiente hora.

El diseño distingue tres niveles. El primero es metodológico y contiene el procedimiento de análisis. El segundo es experimental y permite contrastar el comportamiento del procedimiento en un sistema sintético conocido y en una serie financiera real. El tercero es operativo y se materializa en el MVP web. Esta separación evita atribuir a cada parte un alcance que no tiene: la comprobación sintética no demuestra nada sobre Bitcoin, la aplicación empírica no prueba por sí sola caos determinista y el MVP no sustituye al estudio completo.

6.2– Arquitectura global del sistema

La arquitectura global se diseña como una arquitectura desacoplada en tres bloques: repositorio de investigación, MVP web y memoria académica. El primer bloque contiene el procedimiento experimental y genera los artefactos técnicos. El segundo bloque proporciona una interfaz de uso

sobre una selección de esos artefactos. El tercero documenta el planteamiento, las decisiones de diseño, la implementación, los resultados y las limitaciones.

Bloque	Responsabilidad de diseño
Repositorio de investigación	Ejecutar el procedimiento de análisis, generar datasets procesados, parámetros, métricas, figuras y artefactos reutilizables.
MVP web	Consumir artefactos exportados y ofrecer una interfaz para cargar datos, validar entradas, construir variables y obtener predicciones de volatilidad.
Memoria académica	Integrar el trabajo realizado, justificar las decisiones metodológicas y separar los resultados defendibles de las interpretaciones no justificadas.

Cuadro 6.1: Bloques principales de la arquitectura global del proyecto.

La comunicación entre el repositorio de investigación y el MVP se realiza mediante ficheros intermedios. El diseño evita que la aplicación web dependa directamente del código experimental. En lugar de importar funciones internas del repositorio de análisis, el MVP lee parámetros, coeficientes, referencias de entrenamiento, métricas históricas y modelos exportados. Con ello, el entrenamiento y la validación histórica quedan fuera de la ejecución interactiva.

Esta frontera tiene dos ventajas. Primero, mejora la trazabilidad: un artefacto usado por el MVP procede de una ejecución concreta del procedimiento. Segundo, reduce el acoplamiento: el MVP puede ejecutarse como aplicación autónoma siempre que disponga de los ficheros necesarios. La estructura física completa de los repositorios se documenta en el anexo correspondiente; en este capítulo solo interesa la separación lógica entre responsabilidades.

6.3– Diseño del procedimiento metodológico

El procedimiento metodológico se diseña como una cadena de análisis con entradas, salidas y controles definidos. No se plantea como una colección de gráficos aislados ni como un único modelo predictivo, sino como un flujo que transforma datos brutos en resultados interpretables. Esta estructura es necesaria porque, en series financieras reales, una mejora puntual de error o un patrón visual no bastan para defender una interpretación dinámica fuerte.

El procedimiento se estructura en fases, desarrolladas con detalle en el Capítulo 7. De forma resumida, estas fases cubren la validación de datos, la construcción de variables de volatilidad realizada, la caracterización estadística de la serie, el filtrado lineal, la reconstrucción del espacio de estados, la cuantificación dinámica, la comparación predictiva y la exportación de artefactos para el MVP.

Este diseño permite que cada resultado posterior tenga un contexto metodológico previo. La predicción se evalúa después de construir y caracterizar la serie; la reconstrucción no se introduce sin estudiar antes la dependencia temporal; y los modelos se comparan frente a referencias simples y lineales antes de extraer conclusiones sobre utilidad predictiva.

6.4– Diseño de la comprobación sintética

La comprobación sintética se diseña como un control interno del procedimiento. Su función no es representar el comportamiento de Bitcoin, sino ofrecer un caso en el que la estructura de los

datos está definida de antemano. De esta forma, antes de aplicar el sistema a una serie financiera real, se dispone de un entorno controlado para observar si el flujo de análisis responde de manera coherente.

El caso elegido es el mapa logístico, introducido formalmente en el marco teórico. Dentro del diseño del proyecto, su papel es servir como referencia sintética: permite ejecutar el mismo tipo de procesamiento, reconstrucción, cuantificación y predicción local sobre una serie que no procede del mercado. Así se evita evaluar el procedimiento únicamente sobre datos financieros reales, donde intervienen ruido, cambios de comportamiento y factores externos no observados.

La comprobación se plantea con una versión limpia de la serie y con variantes perturbadas por ruido. Esta decisión permite observar cómo cambia el comportamiento del procedimiento cuando la señal disponible se degrada. No se busca obtener una equivalencia con Bitcoin, sino disponer de un punto de comparación entre un entorno controlado y una aplicación empírica más incierta.

Por tanto, la comprobación sintética queda integrada como una pieza de diseño del sistema. Sirve para apoyar la interpretación posterior del estudio, pero sus resultados no se trasladan directamente a Bitcoin ni se utilizan como prueba de comportamiento financiero.

6.5– Diseño del flujo de datos y variables principales

El flujo de datos se diseña para mantener una separación clara entre entrada, transformación, objetivo predictivo y salida. La entrada original son velas de mercado con información temporal y precio de cierre. A partir de esos datos se construyen variables internas orientadas al estudio de la volatilidad, no a la predicción directa del nivel de precio.

La idea central del diseño es trabajar con información disponible hasta cada instante y reservar la información posterior únicamente para evaluación. Por ello, el sistema distingue entre variables construidas con datos pasados y variables objetivo asociadas a una ventana futura. Esta separación permite que los modelos se evalúen de forma coherente y reduce el riesgo de fuga de información.

Las variables principales se resumen en la Tabla 6.2. La tabla recoge solo las magnitudes necesarias para entender el flujo general del sistema; las definiciones formales y transformaciones matemáticas se desarrollan en el marco teórico y en la implementación del procedimiento.

Variable	Papel en el diseño
log_rv_past_12	Serie principal del análisis; resume la volatilidad reciente en una ventana de una hora.
log_rv_future_12	Objetivo predictivo; representa la volatilidad de la siguiente hora.
log_rv_past_48	Variable auxiliar que incorpora información de una ventana temporal más amplia.
log_rv_past_288	Variable auxiliar que incorpora información de escala diaria.
z_log_rv_past_12	Versión normalizada de la serie principal, utilizada cuando el método requiere trabajar en una escala comparable.

Cuadro 6.2: Variables principales consideradas en el diseño del sistema.

El flujo completo se organiza como una transformación secuencial: datos de mercado, variables de volatilidad, modelos y salidas interpretables. En el repositorio técnico, este flujo permite generar resultados, tablas y artefactos. En el MVP web, permite recibir datos recientes o de ejem-

plo, validar su estructura, construir las variables necesarias y mostrar predicciones de volatilidad de forma comprensible.

6.6– Diseño experimental y separación temporal

La separación temporal es una condición estructural del diseño experimental. En un problema basado en datos ordenados en el tiempo, las observaciones futuras no pueden intervenir en decisiones que simulan realizarse en el pasado. Por ello, el sistema debe distinguir de forma explícita entre los datos usados para ajustar modelos, los datos usados para elegir configuraciones y los datos reservados para evaluación final.

El diseño establece tres usos diferenciados de los datos. El tramo de entrenamiento se utiliza para ajustar parámetros y modelos. El tramo de validación permite elegir configuraciones cuando existe más de una alternativa razonable. El tramo de test se reserva para evaluar el comportamiento final sin intervenir en las decisiones anteriores.

La construcción del objetivo predictivo sigue la misma lógica. Las variables de entrada deben estar disponibles en el instante en el que se formula la predicción, mientras que la variable objetivo se calcula con información posterior y solo se utiliza para entrenamiento o evaluación según la partición temporal correspondiente.

Los métodos que comparan estados pasados con estados actuales también deben respetar esta separación. En esos casos, los candidatos disponibles para una predicción deben pertenecer únicamente al histórico permitido. Además, se excluyen observaciones demasiado próximas en el tiempo cuando existe riesgo de comparación trivial por solapamiento de ventanas.

Con este diseño, el procedimiento queda orientado a evitar fugas de información. Las configuraciones se seleccionan sin usar test, las variables explicativas se construyen con información pasada o presente, y las salidas finales se interpretan como resultados fuera de muestra.

6.7– Diseño de la arquitectura del MVP web

Desde el punto de vista arquitectónico, la aplicación se organiza en tres niveles. El primer nivel corresponde a la interfaz web, implementada sobre Streamlit, que actúa como punto de interacción con el usuario. El segundo nivel agrupa la lógica reutilizable de la aplicación: carga de datos, validación, construcción de variables, diagnóstico, predicción y generación de salidas. El tercer nivel está formado por los datos auxiliares y artefactos locales que permiten ejecutar el MVP sin depender en tiempo de ejecución del repositorio técnico.

La separación entre estos niveles permite que la interfaz no contenga toda la lógica del sistema y que los modelos históricos no tengan que recalcularse desde la aplicación.

El diseño contempla también un caso distinto: algunos modelos ligeros pueden ajustarse sobre la ventana cargada en tiempo de uso. En particular, el Mini-HAR se considera una estimación local y experimental, diferenciada de los modelos históricos exportados. Esta distinción es importante porque el ajuste local puede adaptarse a la información reciente disponible, pero no sustituye a la validación histórica formal del procedimiento.

Implementación del procedimiento de análisis

7.1– Estructura general del procedimiento

El procedimiento de análisis se implementa como una secuencia de fases reproducibles. Cada fase recibe una entrada concreta, ejecuta un conjunto delimitado de operaciones y exporta los resultados en formatos CSV o JSON, así como en elementos visuales como imágenes, gráficos o tablas. Esta organización permite repetir partes del estudio sin rehacer todo el proyecto y facilita que las cifras, tablas y figuras usadas posteriormente en la memoria puedan rastrearse hasta un script y un fichero de salida.

La cadena principal trabaja sobre datos de BTCUSDT con frecuencia de cinco minutos. Primero se obtiene y valida el dataset limpio; después se construyen retornos, medidas de volatilidad realizada y objetivos futuros; a continuación se aplican herramientas exploratorias, contrastes estadísticos, análisis lineal, reconstrucción del espacio de estados y comparación predictiva. Las fases finales incorporan una comprobación sintética con el mapa logístico y un modelo HAR-logRV compacto, exportable al MVP web.

La estructura del repositorio distingue entre datos brutos, datos procesados, resultados y código. Los ficheros originales se conservan en `data/raw/`; los datasets generados se guardan en `data/processed/`; las tablas, informes y figuras se escriben en `reports/`, `reports/tables/` y `reports/figures/`. Esta separación evita mezclar entradas, resultados intermedios y productos finales del análisis.

En este capítulo se describe la implementación del procedimiento, no la discusión completa de los resultados. Por ello, las secciones siguientes explican qué calcula cada fase, qué criterios se han fijado y qué artefactos produce. La interpretación conjunta de las salidas se reserva para el capítulo de resultados y discusión.

7.2– Entorno de desarrollo y herramientas utilizadas

El procedimiento técnico se implementa en Python. La decisión de usar scripts independientes por fase responde a una necesidad práctica: el análisis combina procesos de naturaleza distinta, desde validación de datos hasta cálculo de correlogramas, gráficos de recurrencia, reconstrucción y predicción. Separar cada bloque en un script reduce dependencias internas y permite revisar una fase sin alterar el resto de la cadena.

La construcción inicial del dataset limpio utiliza `pandas`, porque los datos proceden de ficheros mensuales comprimidos y es necesario leer, concatenar, ordenar y exportar grandes tablas de velas. En cambio, varias fases posteriores se apoyan en módulos propios y, cuando es viable, en biblioteca estándar. Por ejemplo, la validación integral del CSV se implementa sin `pandas`; los gráficos SVG se generan mediante funciones propias; y los cálculos de ACF, PACF, periodograma, ADF y KPSS se implementan en módulos auxiliares del repositorio.

Los módulos de apoyo separan operaciones comunes. `data_loading.py` centraliza lectura y escritura de CSV; `preprocessing.py` construye las variables de Fase 1; `volatility.py` contiene las sumas móviles pasadas y futuras; `stationarity.py` implementa herramientas de estacionariedad; `spectral.py` calcula correlogramas y espectro; `recurrence.py` genera gráficos de recurrencia; y `plotting.py` agrupa utilidades ligeras para figuras SVG.

La salida de cada fase se guarda en formatos simples. Las tablas se escriben en CSV, los resúmenes de parámetros en JSON cuando procede, las matrices o referencias numéricas en NPZ y las figuras en SVG o PNG. Esta elección simplifica la reutilización de resultados por la memoria y por el MVP web, sin depender de objetos internos de Python.

7.3– Obtención y validación inicial de los datos

El dato bruto procede de los ficheros mensuales de *klines* de Binance Spot para el par BTCUSDT con intervalo de cinco minutos. Los ficheros se almacenan en `data/raw/` con nombres del tipo `BTCUSDT-5m-2024-01.zip`. Cada ZIP contiene un CSV sin cabecera con las columnas originales de Binance.

El script `build_btc_5m_clean.py` lee todos los ZIP disponibles en `data/raw/`, extrae el CSV interno de cada archivo y concatena las tablas resultantes. Durante la limpieza se convierte `open_time` a fecha UTC, se detecta automáticamente si la unidad temporal viene en milisegundos, microsegundos o nanosegundos, se convierten las columnas numéricas, se ordenan las filas por tiempo y se eliminan duplicados por `open_time`. La salida se reduce a las columnas necesarias para el análisis: `open_time`, `open`, `high`, `low`, `close`, `volume`, `trades` y `taker_buy_quote`.

La validación formal se realiza con `validate.py`. Este script comprueba que el CSV limpio tiene las columnas esperadas, que el rango temporal coincide con el intervalo declarado, que la frecuencia modal es de cinco minutos, que no existen diferencias temporales distintas de cinco minutos, que `open_time` es estrictamente creciente, que no hay duplicados, huecos temporales, nulos ni errores de conversión numérica, y que los precios son positivos y coherentes con la relación OHLC.

La Fase 0 escribe varias tablas auxiliares en `reports/tables/`: resumen de calidad, frecuencia temporal, nulos, comprobaciones numéricas y reporte de huecos. En la ejecución documentada, el dataset limpio contiene 245.088 observaciones, desde el 1 de enero de 2024 a las 00:00 hasta el 30 de abril de 2026 a las 23:55. Esta fase fija la base empírica sobre la que se construyen las variables del resto del trabajo.

7.4– Construcción de variables y medidas de volatilidad

La Fase 1 transforma el CSV limpio en el dataset de trabajo `data/processed/btc_5m_features.csv`. El script principal es `build_phase1_features.py`, que delega la construcción de variables en `preprocessing.py` y las sumas móviles de volatilidad en `volatility.py`.

Primero se calcula el logaritmo del cierre, `log_close`, y a partir de él el retorno logarítmico de cinco minutos,

$$r_t = \log(C_t) - \log(C_{t-1}).$$

Sobre el retorno se obtienen `r2`, `abs_r`, el rango logarítmico `hl_range`, y las transformaciones `log_volume` y `log_trades`. Estas variables permiten conservar información de precio, intensidad de movimiento y actividad de mercado.

La volatilidad realizada se implementa como suma móvil de retornos al cuadrado. Las variables `rv_past_12`, `rv_past_48` y `rv_past_288` agregan, respectivamente, 1 hora, 4 horas y 1 día de datos pasados. La convención es inclusiva hasta t : la ventana pasada de tamaño h usa los retornos desde $t - h + 1$ hasta t .

Los objetivos futuros se construyen con una convención estrictamente posterior. `rv_future_12` y `rv_future_48` agregan retornos desde $t + 1$ hasta el horizonte correspondiente. Esta definición evita que el retorno del instante t entre simultáneamente en las variables explicativas y en el objetivo futuro.

Sobre todas las volatilidades realizadas se aplica una transformación logarítmica con $\varepsilon = 10^{-12}$. Así se obtienen, entre otras, `log_rv_past_12` y `log_rv_future_12`. Tras eliminar las filas iniciales sin histórico suficiente y las filas finales sin futuro disponible, el dataset procesado queda con 244.752 observaciones, desde el 2 de enero de 2024 a las 00:00 hasta el 30 de abril de 2026 a las 19:55.

La serie central del procedimiento queda definida como

$$v_t = \log_rv_past_12,$$

mientras que el objetivo predictivo principal es `log_rv_future_12`. Esta elección no se discute aquí como resultado final; se fija como salida técnica de la construcción de variables y se evalúa en las fases posteriores.

7.5– Visualización inicial y estadísticos descriptivos

La Fase 2 aplica una primera batería de herramientas descriptivas sobre `btc_5m.features.csv`. El script `phase2_general_tools.py` lee el dataset procesado, convierte las columnas numéricas necesarias y genera figuras temporales, estadísticos descriptivos y una tabla de eventos extremos.

Las figuras generadas cubren el precio de cierre, el logaritmo del precio, los retornos logarítmicos, los retornos absolutos, `log_rv_past_12`, `log_rv_future_12`, volumen y número de operaciones. Las salidas se escriben como SVG en `reports/figures/`. El objetivo de estas figuras es inspeccionar la escala de las variables, localizar episodios extremos y comprobar visualmente la diferencia entre precio, retorno e intensidad de movimiento.

La fase también calcula estadísticos descriptivos para las variables principales: media, desviación típica, mínimo, percentiles, máximo, asimetría y exceso de curtosis. Además, identifica eventos extremos como el retorno más negativo, el retorno más positivo, el mayor retorno absoluto, la mayor volatilidad realizada pasada, la mayor volatilidad realizada futura, el mayor volumen y el mayor número de operaciones.

Esta fase no realiza contrastes formales ni extrae conclusiones sobre caos o predictibilidad. Su función dentro del procedimiento es preparar una lectura inicial del dataset procesado y documentar qué variables merecen análisis más detallado en las fases de estacionariedad, correlación, recurrencia y reconstrucción.

7.6– Análisis de estacionariedad

La Fase 3 estudia estacionariedad y transformaciones sobre cinco series: `close`, `log_close`, `r`, `abs_r` y `log_rv_past_12`. El script `phase3_stationarity.py` genera gráficos temporales, gráficos de media y desviación típica móvil, contrastes ADF y contrastes KPSS.

La ventana móvil se fija en 288 observaciones, equivalente a un día de datos de cinco minutos. Para el ADF se prueban retardos candidatos $\{0, 1, 2, 3, 6, 12\}$ y se selecciona el retardo mediante BIC. El KPSS se implementa con constante y varianza de largo plazo tipo Newey-West. En esta fase no se usa `statsmodels`; los módulos propios devuelven estadísticos y rangos aproximados de *p-value* usando valores críticos asintóticos habituales.

Las salidas de la fase incluyen figuras temporales por serie, figuras de momentos móviles y tablas CSV con resultados ADF, selección de retardos ADF, resultados KPSS y resumen de momentos móviles. Estas salidas permiten justificar qué transformaciones son adecuadas para continuar el análisis, pero la lectura completa de estacionariedad y cambios de régimen se desarrolla en el capítulo de resultados.

7.7– Análisis de autocorrelación y espectro

La Fase 4 analiza dependencia lineal y estructura espectral en tres series: `r`, `abs_r` y `log_rv_past_12`. El script `phase4_correlogram_spectrum.py` calcula ACF, PACF y periodograma para cada una de ellas.

La ACF se calcula hasta 2016 retardos, equivalente a una semana de datos de cinco minutos. También se genera una vista específica hasta 288 retardos, equivalente a un día. La PACF se calcula hasta 288 retardos mediante recursiones de Levinson-Durbin. El periodograma se obtiene tras restar la media y se representa frente al periodo equivalente en retardos de cinco minutos, utilizando escala logarítmica para poder comparar escalas de una hora, un día y una semana.

La fase guarda tablas con valores de ACF, valores de PACF, picos espectrales, potencia en periodos de referencia y resumen de correlograma. Las figuras correspondientes se escriben en SVG. Las bandas de ACF/PACF se tratan como referencias visuales, no como contraste exacto de ruido blanco, porque el tamaño muestral, las colas pesadas y la heterocedasticidad pueden hacer que retardos de magnitud pequeña resulten formalmente significativos.

El propósito de esta fase es documentar la diferencia entre retorno firmado e intensidad de movimiento. La interpretación final de los retardos relevantes y de los picos espectrales se reserva al capítulo de resultados; en la implementación basta con fijar el protocolo de cálculo y las salidas generadas.

7.8– Gráficos de recurrencia

La Fase 5 introduce gráficos de recurrencia iniciales sobre ventanas representativas. No se calcula una matriz de recurrencia completa sobre toda la muestra, porque el tamaño del dataset haría inviable una matriz $N \times N$ y, además, su interpretación visual sería poco útil. En su lugar se seleccionan ventanas locales de 2.000 observaciones.

El script `phase5_initial_recurrence.py` selecciona cuatro ventanas: una ventana tranquila, una ventana de alta volatilidad, una ventana reciente y una ventana central representativa. La ventana tranquila se elige mediante medias móviles de `log_rv_past_12`; la de alta volatilidad se centra en el máximo de esa misma serie; la reciente toma las últimas observaciones

disponibles; y la central se sitúa en la mitad del dataset procesado.

En cada ventana se analizan tres series: r , abs_r y log_rv_past_12 . Cada fragmento se normaliza mediante z-score calculado dentro de la propia ventana. La distancia utilizada es absoluta en una dimensión y el umbral ε se ajusta para alcanzar una tasa de recurrencia objetivo del 5 %. Los gráficos se guardan en PNG, donde el píxel negro representa un par recurrente y el píxel blanco un par no recurrente.

La fase escribe dos tablas principales: `phase5_selected_windows.csv`, con la descripción y límites temporales de las ventanas, y `phase5_recurrence_parameters.csv`, con los parámetros usados para cada serie y ventana. En este punto los gráficos de recurrencia se usan como herramienta exploratoria sobre series escalares; no constituyen todavía una reconstrucción del espacio de estados ni una prueba de caos. Su papel es preparar visualmente la transición hacia las fases posteriores de filtrado, contraste y reconstrucción.

7.9– Filtrado lineal mediante modelos autorregresivos

La Fase 6 introduce un filtrado lineal sobre la serie principal del estudio, $v_t = \text{log_rv_past_12}$. Antes del ajuste, la serie se estandariza utilizando exclusivamente la media y la desviación típica del tramo de entrenamiento. Esta decisión evita que información posterior al corte temporal intervenga en la escala usada por el modelo.

El script `phase6_linear_filtering.py` fija el final del entrenamiento en 2025-06-30 23:55:00. Sobre ese tramo ajusta modelos $\text{AR}(p)$ para $p = 1, \dots, 100$, mediante ecuaciones de Yule-Walker. Para cada orden se calcula la varianza de innovación, AIC y BIC. El orden seleccionado es el que minimiza BIC; en la ejecución documentada se obtiene un $\text{AR}(49)$.

Una vez seleccionado el orden, el modelo se aplica sobre la serie completa estandarizada para obtener valores ajustados *one-step* y residuos. Estos residuos se guardan en `reports/tables/phase6_residual_series.csv`, junto con el valor original de `log_rv_past_12`, su versión estandarizada, el ajuste AR y el residuo correspondiente. Esta tabla constituye la entrada principal de la fase siguiente.

La fase genera también tablas de selección del orden autorregresivo, coeficientes del AR seleccionado, estadísticos descriptivos de residuos, valores ACF/PACF de residuos, contrastes Ljung-Box y comparación de correlogramas entre la serie estandarizada y los residuos. Las figuras asociadas incluyen la serie temporal de residuos, su histograma, ACF/PACF y gráficos de recurrencia sobre las mismas ventanas seleccionadas en la Fase 5.

El papel de esta fase es doble. Por un lado, proporciona un benchmark lineal fuerte para las fases predictivas. Por otro, permite separar la dependencia lineal en media de la estructura que pueda permanecer en los residuos. La interpretación de si esa estructura residual es relevante se pospone a los contrastes de la Fase 7 y a la discusión de resultados.

7.10– Contrastes de no linealidad

La Fase 7 aplica contrastes de dependencia residual sobre dos series alineadas: la serie principal estandarizada, `z_log_rv_past_12`, y los residuos del $\text{AR}(49)$. El objetivo de la fase no es demostrar caos, sino comprobar si, tras retirar una componente lineal en media, siguen apareciendo dependencias compatibles con heterocedasticidad, estructura temporal en varianza o desviaciones frente a una secuencia i.i.d.

El script `phase7_nonlinearity_tests.py` toma como entrada `phase6_residual_series.csv`. En primer lugar calcula contrastes Ljung-Box sobre los cuadrados de ambas series, con retardos 12, 24, 48, 96, 288 y 2016. Esta prueba se usa para evaluar dependencia en segundo momento, especialmente relevante en series financieras donde la volatilidad tiende a agruparse.

A continuación se aplica un contraste ARCH-LM aproximado sobre los residuos del AR. La implementación evita construir una regresión masiva con matriz de diseño explícita y usa una aproximación mediante Yule-Walker sobre residuos cuadrados centrados. Los retardos considerados son 12, 24, 48, 96 y 288. La salida se guarda en `phase7_arch_lm.csv`, junto con el estadístico, el p -valor y una nota sobre la aproximación empleada.

La fase incorpora también el contraste BDS en ventanas representativas. Para mantener el coste computacional controlado, se trabaja sobre ventanas de 3.000 observaciones, reutilizando los centros de las ventanas seleccionadas previamente cuando están disponibles. Se evalúan dimensiones $m = 2, 3, 4$ y umbrales ϵ iguales a 0,5, 1,0 y 1,5 desviaciones típicas, dado que cada ventana se normaliza antes del contraste. Los resultados se escriben en `phase7_bds_results.csv` y se resumen mediante una figura de p -valores.

Por último, se realiza una comparación frente a series barajadas. Para cada ventana y serie se calcula la autocorrelación de cuadrados en retardos 1, 12 y 288, y se compara con 50 permutaciones aleatorias generadas con semilla fija. Esta prueba conserva la distribución marginal de la ventana, pero destruye el orden temporal. La comparación permite distinguir entre efectos puramente distribucionales y dependencias asociadas al orden de la serie.

Las salidas de la Fase 7 incluyen tablas de ventanas, Ljung-Box sobre cuadrados, ARCH-LM, resultados BDS, resúmenes por ventana y comparación con barajado. Las figuras principales son la ACF de cuadrados de la serie estandarizada, la ACF de cuadrados de los residuos, el resumen BDS y la comparación de autocorrelaciones frente a permutaciones. La lectura metodológica es prudente: la fase busca evidencia de dependencia residual, no una prueba directa de dinámica caótica determinista.

7.11– Reconstrucción del espacio de estados

La Fase 8 implementa la reconstrucción del espacio de estados de la serie principal. La serie de partida vuelve a ser `log_rv_past_12`, estandarizada con media y desviación típica calculadas solo en el tramo de entrenamiento. La reconstrucción se realiza mediante retardos temporales siguiendo la convención causal

$$X_t = [z_t, z_{t-\tau}, z_{t-2\tau}, \dots, z_{t-(m-1)\tau}]. \quad (7.1)$$

Esta orientación garantiza que cada vector reconstruido use únicamente información disponible en t o anterior a t , requisito necesario para su uso posterior en predicción.

El script `phase8_state_space_reconstruction.py` calcula primero el retardo τ mediante información mutua media. Para ello discretiza la serie de entrenamiento en 32 contenedores definidos por cuantiles y evalúa la información mutua para retardos entre 1 y 288. La regla de selección busca el primer mínimo local de la curva AMI; si no aparece, contempla un criterio alternativo por codo. En la ejecución documentada el retardo seleccionado es $\tau = 137$. La fase calcula además una curva AMI para una versión barajada de la serie, usada como referencia visual.

La dimensión de embedding se estudia con falsos vecinos cercanos y con el método de Cao. El cálculo se realiza sobre una submuestra equiespaciada de 3.000 puntos del entrenamiento, con dimensión máxima 15. Para FNN se usan los criterios $R_{tol} = 10$ y $A_{tol} = 2$, y se fija como referencia un umbral del 5 %. El método de Cao calcula las secuencias $E1$ y $E2$ para evaluar estabilización

de la dimensión. La regla final combina ambas fuentes: si FNN y Cao son compatibles, se adopta la mayor dimensión sugerida; si discrepan, se prioriza FNN y se conserva Cao como cautela metodológica. En la ejecución documentada se adopta $m = 5$.

Con los parámetros seleccionados se construyen embeddings separados para entrenamiento y prueba. La separación temporal es la misma que en las fases anteriores: el entrenamiento termina en 2025-06-30 23:55:00 y el tramo posterior queda reservado para evaluación. Los vectores se guardan en `data/processed/phase8_embedding_train.npz` y `data/processed/phase8_embedding_test.npz`, junto con índices y metadatos. Los parámetros finales se almacenan en `reports/tables/phase8_selected_embedding_params.json`.

La fase genera tablas de AMI, AMI barajada, FNN, Cao y una muestra del embedding. También produce figuras de AMI, comparación original frente a barajada, FNN, Cao, proyecciones 2D/3D globales y proyecciones 2D por ventanas representativas. Estas visualizaciones no se interpretan como prueba de atractor; se usan para documentar la representación de estados que alimenta las fases de cuantificación y predicción local.

7.12– Cuantificación de la dinámica reconstruida

La Fase 9 cuantifica la dinámica asociada al embedding seleccionado. El script `phase9_dynamics_quantification.py` carga los parámetros de la Fase 8, reconstruye los vectores de entrenamiento y compara la serie original con una versión barajada reproducible. Esta comparación es importante porque permite distinguir entre propiedades que dependen del orden temporal y propiedades que podrían aparecer por la distribución marginal de la serie.

La primera familia de indicadores corresponde a la dimensión de correlación. La implementación toma una submuestra equiespaciada de 2.500 vectores y calcula distancias euclídeas entre pares válidos, excluyendo vecinos temporales mediante una ventana de Theiler igual a $m\tau$. A partir de esas distancias se definen 32 radios entre cuantiles y se estima la curva $C(r)$, su representación log-log y la pendiente local. La salida incluye tanto los valores de la curva como un resumen prudente de D_2 , que registra si existe o no una meseta estable.

La segunda familia de indicadores es la divergencia local de trayectorias cercanas mediante el método de Rosenstein. Para reducir el coste computacional, se toma un bloque continuo de 3.000 vectores situado en la zona central del entrenamiento. Se identifican vecinos no triviales, se calcula la evolución de la distancia media durante 60 pasos de cinco minutos y se ajusta una recta en el intervalo $k = 1, \dots, 30$. La pendiente se guarda tanto por paso de cinco minutos como reescalada por hora.

La tercera familia de indicadores es la entropía de permutación. Se calcula para órdenes 3, 4, 5, 6 y 7, usando dos retardos: 1 y τ . El mismo cálculo se repite sobre la serie barajada. Esta medida resume la diversidad de patrones ordinales y complementa los indicadores basados en distancias, ya que no requiere una interpretación geométrica directa del embedding.

La fase guarda tablas para la curva de dimensión de correlación, la curva de Rosenstein y la entropía de permutación. Además, escribe resúmenes JSON con los parámetros y resultados agregados. Las figuras asociadas muestran la curva log-log de correlación, la pendiente local, la divergencia de Rosenstein, la entropía de permutación y una comparación agregada entre original y barajada. La interpretación de estos indicadores se desarrolla en el capítulo de resultados; aquí quedan definidos los cálculos y las salidas generadas.

7.13– Contrastes con barajado y datos subrogados

La Fase 10 amplía el control frente a series de referencia. La comparación con una serie barajada destruye todo orden temporal, pero no es suficiente para evaluar si ciertos resultados pueden explicarse por estructura lineal o por la distribución marginal. Por ello, el script `phase10_surrogate_tests.py` genera tres grupos de referencia: permutaciones aleatorias, series *phase-randomized* y series AAFT.

La fase trabaja sobre una ventana de entrenamiento de 8.192 observaciones. Este tamaño es una potencia de dos, requisito práctico para la generación eficiente de subrogadas *phase-randomized* mediante FFT. La ventana se toma del tramo de entrenamiento y se estandariza con los parámetros de train. Los parámetros τ , m y la convención de embedding proceden de la Fase 8, mientras que la exclusión temporal se fija de forma coherente con la ventana de Theiler usada en cuantificación.

Las series barajadas conservan los valores observados, pero eliminan su orden temporal. Las series *phase-randomized* se generan aleatorizando fases en el dominio frecuencial, manteniendo de forma aproximada el espectro lineal. Las series AAFT aplican una transformación por rangos, aleatorización de fases y reordenación final para aproximar simultáneamente distribución marginal y estructura lineal. En la ejecución se generan 50 series barajadas, 39 *phase-randomized* y 39 AAFT, todas con semilla fija para asegurar reproducibilidad.

Para cada réplica se calculan los mismos estadísticos principales: dimensión de correlación aproximada, pendiente de Rosenstein por paso y por hora, entropía de permutación con retardo 1 y entropía de permutación con retardo τ . Por coste computacional, esta fase usa una configuración más compacta que la Fase 9: muestra de 700 vectores para dimensión de correlación, 24 radios, bloque de 1.600 vectores para Rosenstein, 550 referencias, $k_{\text{máx}} = 40$ y ajuste en $k = 1, \dots, 20$.

Las salidas se guardan en `reports/tables/`. Incluyen el estadístico original, la configuración de la fase, las réplicas de cada grupo y un resumen por estadístico con media, desviación típica, percentiles, distancia estandarizada y p -valor empírico. Las figuras generadas muestran boxplots agregados, boxplots separados por tipo de estadístico, histogramas de Lyapunov y entropía, y ejemplos de serie original frente a subrogadas.

El objetivo de esta fase es someter los indicadores de la Fase 9 a referencias más exigentes. Si una diferencia frente a barajado desaparece frente a subrogadas que conservan espectro o distribución, la lectura debe limitarse. Por tanto, la Fase 10 actúa como mecanismo de control para que el capítulo de resultados pueda distinguir entre estructura temporal general, dependencia lineal, efectos distribucionales e indicios no lineales más robustos.

7.14– Predicción local de volatilidad

La Fase 11 evalúa la capacidad predictiva del espacio reconstruido sobre el objetivo `log_rv_future_12`. La prueba se plantea como una evaluación empírica fuera de muestra. No se utiliza para demostrar caos, sino para comprobar si la representación de estados construida en la Fase 8 permite mejorar referencias predictivas simples bajo una separación temporal estricta.

El script `phase11_local_state_space_prediction.py` carga los parámetros seleccionados en la reconstrucción, en particular $\tau = 137$ y $m = 5$, y prepara tres particiones cronológicas: entrenamiento hasta el 30 de junio de 2025, validación hasta el 31 de diciembre de 2025 y test desde el 1 de enero de 2026. La serie de entrada es `log_rv_past_12`; el objetivo es `log_rv_future_12`; y el horizonte es de 12 velas, equivalente a una hora.

Los modelos comparados son media histórica, persistencia, AR(49) recursivo a horizonte 12, vecino más próximo y media de k vecinos. Para el método local se construyen vectores de embedding en cada partición y se buscan vecinos mediante un KD-tree exacto. La rejilla inicial de k es

$$k \in \{2, 3, 5, 10, 20, 50\}.$$

La selección de k se realiza exclusivamente en validación, usando RMSE como criterio. El conjunto de test no interviene en esta selección.

La fase incorpora varias reglas anti-*leakage*. En validación, los vecinos se buscan solo dentro de entrenamiento. En test, la biblioteca de vecinos permitida es entrenamiento más validación. Además, un candidato solo es elegible si su etiqueta futura ya sería conocida en el instante de consulta, es decir, si cumple `candidate_index + 12 ≤ query_index`. También se aplica una ventana de Theiler de $\max(\text{theiler}, m\tau)$, que en esta configuración queda en 685 retardos, para evitar vecinos temporalmente triviales.

Las salidas de la fase se escriben en `reports/tables/`. Incluyen el resumen de particiones, la selección de k en validación, las métricas de test, una muestra de predicciones fuera de muestra y un resumen JSON con los parámetros, controles de fuga de información y métricas agregadas. Las figuras generadas muestran la curva de validación por k , la comparación real frente a predicho en una ventana de test, la evolución de errores, el histograma de errores y la comparación de métricas entre modelos.

7.15– Experimentos de robustez y configuraciones alternativas

La Fase 12 extiende la evaluación predictiva de la Fase 11 sin introducir una nueva familia de modelos. Su objetivo es comprobar si el comportamiento del predictor local depende de una rejilla de k demasiado limitada o de la elección operativa $m = 5$, frente a la dimensión más alta sugerida por el método de Cao.

El script `phase12_prediction_sensitivity.py` evalúa dos configuraciones:

$$(\tau, m) = (137, 5) \quad \text{y} \quad (\tau, m) = (137, 14).$$

En la primera, la memoria efectiva del embedding es $(5 - 1)137 = 548$ velas, aproximadamente 45,7 horas. En la segunda, la memoria efectiva es $(14 - 1)137 = 1781$ velas, aproximadamente 148,4 horas. La ventana de Theiler se fija como τm en cada configuración.

La rejilla de vecinos se amplía a

$$k \in \{2, 3, 5, 10, 20, 50, 100, 200\}.$$

La configuración $m = 5$ se evalúa sobre 5.000 puntos equiespaciados de validación y test. La configuración $m = 14$ se evalúa sobre 1.000 puntos, debido al mayor coste computacional asociado a la dimensión alta y a la menor densidad local del espacio reconstruido.

La lógica de evaluación mantiene las mismas restricciones temporales que la Fase 11: validación busca vecinos solo en entrenamiento; test busca vecinos en entrenamiento más validación; los candidatos deben tener etiqueta futura disponible; y test no participa en la selección de k . Además de kNN, se conservan las referencias de media histórica, persistencia, AR(49) y vecino más próximo para que la comparación sea homogénea.

La fase genera tablas de selección de k por configuración, métricas de test, comparación compacta entre configuraciones y un resumen JSON. Las figuras muestran el RMSE de validación por

k , el RMSE de test por configuración, la comparación de métricas y una ventana real frente a predicha para las mejores configuraciones. La lectura de estos resultados se reserva para el capítulo de resultados; en este capítulo la fase queda definida como prueba de sensibilidad del procedimiento local.

7.16– Comprobación sintética con mapa logístico

La Fase 13 replica una parte sustancial del procedimiento sobre un sistema sintético conocido: el mapa logístico en régimen caótico. Esta fase no utiliza datos de Bitcoin. Su función es actuar como control metodológico: comprobar que las herramientas de reconstrucción, cuantificación y predicción local responden de forma razonable cuando la dinámica subyacente sí procede de un sistema determinista no lineal.

La serie limpia se genera mediante

$$x_{t+1} = 4x_t(1 - x_t), \quad (7.2)$$

con $x_0 = 0,123456789$. Se simulan 12.000 iteraciones y se descartan las primeras 1.000 como transitorio, quedando 11.000 observaciones útiles. A partir de la serie limpia se construyen dos variantes con ruido observacional gaussiano: una con ruido pequeño y otra con ruido moderado. La desviación típica del ruido se fija como proporción de la desviación típica de la serie limpia, y los valores resultantes se recortan al intervalo $[0, 1]$.

El script `phase13_logistic_map_validation.py` aplica a cada serie sintética la selección de retardo mediante AMI, la selección de dimensión mediante FNN y Cao, la reconstrucción por retardos, visualizaciones 2D y 3D, una estimación tipo Rosenstein y una evaluación predictiva local. En este caso el horizonte predictivo es 1, porque el mapa logístico define directamente x_{t+1} a partir de x_t .

La partición temporal de cada serie sintética se realiza con un 60 % para entrenamiento, 20 % para validación y 20 % para test. La rejilla de vecinos es

$$k \in \{1, 2, 3, 5, 10, 20, 50\}.$$

La selección de k se realiza en validación y la evaluación final se calcula en test. Se mantienen controles temporales análogos a los usados en Bitcoin: los vecinos de validación se buscan en entrenamiento y los de test en entrenamiento más validación.

La fase incluye también una referencia lineal autorregresiva seleccionada por BIC. A diferencia de Bitcoin, aquí se contempla explícitamente $AR(0)$ como media histórica, además de $AR(p)$ para $p = 1, \dots, 100$. Esta referencia sirve para comprobar si una aproximación lineal simple puede capturar la dinámica sintética o si el predictor local aprovecha mejor la estructura no lineal del sistema.

Las salidas incluyen CSV con las series sintéticas, tablas de AMI, FNN, Cao, parámetros seleccionados, curva de Rosenstein, métricas predictivas, selección AR por BIC y resumen JSON. Las figuras generadas muestran ejemplos de las series, curvas de AMI/FNN/Cao, embeddings 2D/3D, divergencia de Rosenstein, selección de k , selección AR, mapa de transición de la serie limpia, predicciones real frente a predicho y comparación de métricas. La interpretación final de esta comprobación se desarrolla en el capítulo de resultados.

7.17– Modelo HAR-logRV y exportación al MVP

La Fase 14 introduce un modelo HAR-logRV compacto como cierre predictivo operativo. El objetivo no es abrir una búsqueda amplia de modelos de volatilidad, sino incorporar una referencia específica para volatilidad realizada que sea simple, interpretable y exportable al MVP web.

El modelo utiliza tres escalas pasadas de volatilidad realizada:

$$\log_rv_past_12, \quad \log_rv_past_48, \quad \log_rv_past_288.$$

La variable objetivo es $\log_rv_future_12$. La especificación implementada es

$$\log_rv_future_12 = \beta_0 + \beta_1 \log_rv_past_12 + \beta_2 \log_rv_past_48 + \beta_3 \log_rv_past_288 + \varepsilon_t. \quad (7.3)$$

La estimación se realiza mediante mínimos cuadrados ordinarios sobre el tramo de entrenamiento. La implementación resuelve las ecuaciones normales y solo recurre a una regularización ridge muy pequeña si la matriz resulta numéricamente singular. Las tres variables del modelo se fijan antes de la evaluación, por lo que validación no se usa para selección de variables y test no interviene en entrenamiento ni en ajuste.

El script `phase14.har_logrv_mvp.py` compara HAR-logRV con media histórica, persistencia, AR(49) a horizonte 12 y kNN local con $\tau = 137$, $m = 5$ y $k = 200$. Para que la comparación con kNN sea homogénea, se utiliza una muestra comparable de 5.000 puntos de test. Además, se calculan métricas sobre validación completa y test completo para los modelos que no requieren búsqueda de vecinos.

La fase mantiene una comprobación explícita de alineación temporal entre $\log_rv_future_12(t)$ y $\log_rv_past_12(t+12)$. También documenta los controles anti-*leakage*: HAR se entrena solo con train, las variables son pasadas, test no se usa para selección ni ajuste, $k = 200$ procede de la fase anterior y los vecinos de test del kNN se buscan solo en entrenamiento más validación.

Como salida principal, la fase exporta el artefacto `har_logrv_model.json` en `data/model_artifacts/`. El artefacto contiene nombre y versión del modelo, fechas de entrenamiento, objetivo, variables, intercepto, coeficientes, tamaño de entrenamiento, posible regularización usada y métricas principales. Esta exportación permite que el MVP ejecute el modelo sin importar código del repositorio técnico.

Además del artefacto exportado, la fase genera tablas con coeficientes HAR, métricas de validación y test, predicciones de muestra, comparación de modelos, simulación del modo MVP con las últimas 1.000 velas y resumen JSON. Las figuras asociadas muestran comparación de métricas, real frente a predicho, distribución de errores absolutos, errores temporales, coeficientes y contexto de predicción para el modo MVP.

7.18– Cierre técnico del procedimiento y artefactos generados

El procedimiento de análisis finaliza con un conjunto de artefactos persistidos que permiten separar la ejecución experimental de la interpretación de resultados y de la aplicación web. Las fases iniciales generan el dataset limpio y el dataset de variables; las fases intermedias producen tablas, figuras y parámetros de reconstrucción; y las fases predictivas generan métricas, predicciones fuera de muestra y modelos exportables.

Los principales artefactos del procedimiento son el dataset `btc_5m_features.csv`, los embeddings de entrenamiento y prueba de la Fase 8, los coeficientes del AR(49), las métricas de

predicción local, los resultados de sensibilidad, las salidas de la comprobación sintética y el modelo `har_logrv_model.json`. Este último se guarda en `data/model_artifacts/` para ser consumido por el MVP web sin importar código del repositorio técnico.

Con esta estructura, el capítulo de implementación deja cerrado qué se ha ejecutado y qué salidas produce cada fase. La comparación entre modelos, la lectura de los indicadores dinámicos y la discusión sobre la existencia de indicios no lineales se desarrollan en el capítulo de resultados.

Implementación del MVP web

8.1– Objetivo del prototipo web

El MVP web se implementa como una capa operativa sobre el procedimiento de análisis desarrollado en el repositorio técnico. Su finalidad no es reproducir todas las fases experimentales del trabajo, sino ofrecer una herramienta ligera que permita cargar datos recientes o de ejemplo, construir las variables necesarias y ejecutar los modelos principales de predicción de volatilidad a una hora.

La aplicación estima la volatilidad realizada esperada de BTCUSDT durante la próxima hora, trabajando con velas de cinco minutos y un horizonte de doce velas. La magnitud predicha es `log_rv_future_12`; por tanto, el MVP no predice dirección del precio, retorno esperado ni señales de compra o venta. Esta distinción se mantiene de forma explícita en la interfaz, ya que el objetivo del prototipo es mostrar la viabilidad operativa del modelo, no convertir el estudio en una herramienta de inversión.

El prototipo se diseña como una aplicación autónoma. En tiempo de ejecución no importa código del repositorio `btc_volatility`, sino que consume artefactos exportados previamente desde el estudio técnico. Esta decisión permite separar la validación histórica de los modelos del uso interactivo de la aplicación. El entrenamiento y la comparación formal pertenecen al procedimiento experimental; el MVP solo aplica los modelos ya cerrados sobre una ventana de datos cargada por el usuario.

El modelo práctico recomendado dentro del MVP es HAR-logRV global. La interfaz mantiene también persistencia, AR(49), kNN local y Mini-HAR local para comparación. De esta forma, el usuario puede observar la predicción principal y, al mismo tiempo, contrastarla con referencias simples, lineales, no lineales y recalibradas localmente.

8.2– Funcionalidades implementadas

La funcionalidad principal del MVP es la generación de predicciones actuales de volatilidad realizada. La aplicación permite cargar datos desde tres fuentes: API de Binance, dataset de ejemplo incluido en el repositorio y CSV proporcionado por el usuario. En todos los casos, los datos se normalizan a una estructura común formada por `timestamp` y `close`, que es suficiente para reconstruir las variables de volatilidad usadas por los modelos.

Una vez cargados los datos, la aplicación valida la frecuencia, los huecos temporales, los valores nulos, el orden cronológico y la positividad de los precios. Si la validación no supera los requisitos mínimos, el MVP no fuerza la predicción. En su lugar, muestra los errores o advertencias correspondientes. Esta decisión evita que una entrada defectuosa produzca salidas numéricas aparentemente válidas.

Cuando los datos son utilizables, la aplicación construye las variables de volatilidad realizada: `log_rv_past_12`, `log_rv_past_48`, `log_rv_past_288` y `z_log_rv_past_12`. Con ellas evalúa la disponibilidad de cada modelo. Esta disponibilidad depende del número de valores útiles: AR(49) requiere al menos 49 valores de la serie principal; kNN requiere la historia necesaria para construir el embedding con $\tau = 137$ y $m = 5$; HAR global requiere las tres variables de memoria; y Mini-HAR necesita una ventana suficiente para recalibrarse localmente.

El MVP implementa cinco modelos de predicción. Persistencia replica la volatilidad realizada actual. AR(49) aplica el modelo autorregresivo exportado desde el estudio técnico. kNN local utiliza el embedding $\tau = 137$, $m = 5$ y $k = 200$. HAR-logRV global aplica el modelo compacto exportado de la Fase 14. Mini-HAR local ajusta un HAR sobre la ventana cargada y se presenta como experimento local, no como sustituto de la validación histórica.

La salida de predicción se muestra en varias escalas. La aplicación presenta la predicción en escala logarítmica, su transformación inversa a `rv_future_12` y la raíz de la volatilidad realizada en porcentaje para la ventana de una hora. También muestra la última vela utilizada, el inicio y el final del horizonte previsto, siempre en UTC. Además, permite descargar las predicciones en CSV.

La aplicación incluye una evaluación *walk-forward* reciente sobre la ventana cargada. Esta evaluación genera predicciones en puntos pasados de la misma ventana y las compara con la volatilidad realizada observada una hora después. El target futuro se usa solo para medir error, no para producir la predicción. Las métricas calculadas son RMSE, MAE y sesgo, junto con tablas y gráficos por modelo. Esta funcionalidad es útil para inspeccionar la ventana cargada, pero no sustituye la validación histórica formal del estudio.

8.3– Arquitectura de la aplicación

La aplicación se implementa con Streamlit y se estructura en un punto de entrada, un conjunto de páginas y varios módulos auxiliares en `src/`. El fichero `app.py` configura la página, muestra el resumen inicial, verifica los artefactos locales y define la navegación entre páginas. La navegación incluye cinco vistas: Inicio, Predicción de volatilidad, Diagnóstico de datos, Comparación de modelos y Ayuda y limitaciones.

El módulo `config.py` centraliza constantes y rutas de la aplicación. Entre ellas se encuentran el símbolo BTCUSDT, el intervalo 5m, el límite de descarga desde Binance, el horizonte de 12 velas, el directorio de artefactos, el directorio del dataset de ejemplo y la lista de artefactos obligatorios. También implementa funciones de verificación que comprueban si los ficheros necesarios existen y no están vacíos.

La carga de datos se divide entre el cliente de Binance y el módulo `data_loader.py`. Para CSV y dataset de ejemplo, `data_loader.py` normaliza los datos a `timestamp, close`. El módulo acepta distintos nombres de columna temporal, como `timestamp`, `open_time`, `date`, `datetime` o `time`. También ordena los datos, elimina duplicados por `timestamp`, descarta precios no positivos y limita los CSV subidos a las últimas 2000 filas.

La validación se concentra en `data_validation.py`. Este módulo devuelve un resultado estructurado con estado de validez, errores, advertencias e información auxiliar. Las comprobacio-

nes incluyen columnas requeridas, timestamps inválidos, cierres no numéricos, tamaño mínimo, precios no positivos, duplicados, orden temporal, timestamps futuros, frecuencia modal, huecos y resumen básico de precios. La frecuencia esperada se fija en cinco minutos y todos los timestamps se interpretan en UTC.

El módulo `feature_engineering.py` construye las variables de volatilidad a partir de la serie de cierre. Calcula `log_close`, retornos logarítmicos, retornos al cuadrado, volatilidades realizadas pasadas de 12, 48 y 288 velas, transformaciones logarítmicas y la versión estandarizada de `log_rv_past_12`. Los parámetros de media, desviación típica, ε y horizonte proceden de `preprocessing_params.json`, de modo que la aplicación reutiliza la misma escala del estudio histórico.

La carga de modelos se centraliza en `model_registry.py`. Este módulo valida y carga coeficientes AR, parámetros de preprocesamiento, parámetros de embedding, matriz de referencia kNN y artefacto HAR-logRV. A partir de ellos construye instancias de los predictores disponibles. También implementa `run_available_predictions`, que ejecuta únicamente los modelos disponibles y devuelve predicciones junto con metadatos del horizonte temporal.

La lógica matemática de los modelos se implementa en `predictors.py`. Este módulo contiene las clases `PersistenceModel`, `AR49Model`, `KNNModel` y `HARLogRVGlobalModel`, además de funciones para ajustar y aplicar Mini-HAR. AR(49) realiza una predicción recursiva a 12 pasos sobre la serie estandarizada. kNN construye el vector retardado actual y promedia los targets de los k vecinos más cercanos. HAR-logRV calcula la predicción mediante una regresión lineal compacta sobre tres escalas de volatilidad realizada.

El diagnóstico de fiabilidad se implementa en `diagnostics.py`. La función principal combina la validación de datos, la disponibilidad de modelos y el dataset de features para producir un score entre 0 y 100, con estados *Fiable*, *Fiabilidad limitada* o *No fiable*. La evaluación *walk-forward* reciente se implementa en `walk_forward.py`, que genera predicciones retrospectivas, calcula errores y resume métricas por modelo.

8.4— Integración del modelo validado

La integración entre el estudio técnico y el MVP se realiza mediante artefactos locales. El directorio `data/model_artifacts/` contiene los ficheros necesarios para ejecutar los modelos sin depender del repositorio de investigación. Los artefactos obligatorios son `ar49_coefficients.csv`, `preprocessing_params.json`, `embedding_params.json`, `knn_reference_train.npz`, `historical_metrics.json` y `har_logrv_model.json`.

El script `export_model_artifacts.py` copia y valida los artefactos procedentes del estudio técnico. Exporta los coeficientes de AR(49), los parámetros de preprocesamiento usados en las fases de predicción, los parámetros de embedding, la referencia de entrenamiento para kNN, el modelo HAR-logRV, las métricas históricas y un dataset de ejemplo reciente. También exporta figuras históricas relevantes a `assets/historical_validation/`, de modo que la página de comparación puede contextualizar la predicción sin recomputar el estudio completo.

La validación de artefactos se refuerza mediante `validate_artifacts.py`. Este script comprueba la existencia de todos los ficheros requeridos, valida que los JSON no contengan rutas absolutas ni referencias de ejecución al repositorio técnico, revisa que los parámetros de embedding sean $\tau = 137$, $m = 5$ y horizonte de 12 velas, verifica que AR tenga 49 coeficientes, valida la estructura del NPZ de kNN y comprueba que el artefacto HAR tenga nombre, target, horizonte y variables correctas.

El artefacto HAR-logRV representa el cierre predictivo de la Fase 14. Contiene el modelo `har_logrv_compact`, `target log_rv_future_12`, horizonte de 12 velas y las variables `log_rv_past_12`, `log_rv_past_48` y `log_rv_past_288`. En ejecución, `model_registry.py` valida este artefacto y crea un `HARLogRVGlobalModel` con intercepto y coeficientes.

El modelo kNN se integra de forma distinta. No se reentrena en la aplicación. El MVP carga una matriz de vectores de entrenamiento y sus targets desde `knn_reference_train.npz`. El predictor actual construye el embedding de la última observación cargada usando los mismos parámetros de la Fase 8 y compara ese estado con la referencia histórica. Así se mantiene la coherencia con el estudio técnico y se evita recalibrar el modelo desde la interfaz.

AR(49) también se ejecuta a partir de parámetros exportados. El modelo recibe la historia reciente de `log_rv_past_12`, la estandariza con media y desviación típica de entrenamiento, aplica recursivamente los 49 coeficientes y devuelve la predicción en escala logarítmica original. Persistencia, en cambio, no requiere artefactos: replica el último valor disponible de `log_rv_past_12`.

8.5– Interfaz de usuario

La interfaz se organiza en cinco páginas. La página de inicio resume el propósito de la aplicación, muestra el símbolo, la frecuencia, el horizonte y el número de modelos, y verifica la disponibilidad de artefactos locales. Si falta algún artefacto obligatorio, la aplicación detiene la ejecución y muestra los comandos necesarios para regenerar y validar los ficheros.

La página *Predicción de volatilidad* concentra el flujo principal. El usuario selecciona la fuente de datos, carga la ventana correspondiente y revisa el resumen temporal. La página muestra primeras y últimas filas, validación de datos, features calculadas, disponibilidad de modelos, predicciones actuales, gráfico reciente y descarga CSV. En esta página también se encuentra la evaluación *walk-forward* reciente.

La página *Diagnóstico de datos* muestra una vista más compacta de la calidad de la ventana cargada. Incluye un score de fiabilidad, checks principales, resumen de filas y rango temporal, frecuencia modal, número de gaps y disponibilidad de modelos. Esta página no ejecuta nuevos modelos; interpreta el estado de los datos almacenados en la sesión y ayuda a entender por qué un modelo puede estar disponible, limitado o bloqueado.

La página *Comparación de modelos* presenta información estática de validación histórica. Incluye una tabla con el rol de cada modelo, los parámetros principales del kNN local, métricas históricas formales y figuras exportadas del estudio técnico. Las figuras siguen la lógica del trabajo: persistencia de volatilidad, reconstrucción del espacio de estados, comparación predictiva final y control sintético con mapa logístico. Esta página no pretende ser un visor de todas las fases del TFG; selecciona únicamente los elementos necesarios para justificar el uso operativo de los modelos.

La página *Ayuda y limitaciones* recoge el alcance de la aplicación, el formato CSV esperado, la fuente de datos de Binance y las restricciones de interpretación. En ella se explicita que el MVP predice volatilidad realizada esperada, no dirección del precio ni señales de trading. También se recuerda que todos los tiempos se muestran en UTC y que Mini-HAR es la única recalibración local.

La interfaz utiliza componentes comunes para cabeceras, tarjetas métricas, tablas de artefactos y disclaimers. Esta decisión evita duplicar mensajes críticos en cada página y mantiene una comunicación coherente: el usuario ve las funcionalidades disponibles, pero también los límites

técnicos del prototipo.

8.6– Flujo de uso de la aplicación

El flujo de uso comienza en la página de inicio. Allí se comprueba que los artefactos locales requeridos están presentes. Si la verificación falla, el MVP no continúa como si pudiera operar normalmente. Esta restricción es deliberada: sin coeficientes, parámetros o referencias históricas, las predicciones no serían reproducibles respecto al estudio técnico.

Después, el usuario accede a la página de predicción y selecciona la fuente de datos. Si elige Binance API, la aplicación descarga las últimas velas cerradas de BTCUSDT con intervalo de cinco minutos. Si elige dataset de ejemplo, carga el fichero local incluido en `data/sample/`. Si sube un CSV, el archivo se normaliza aceptando una columna temporal reconocida y una columna `close`.

Una vez cargados los datos, la aplicación valida la ventana y muestra el resumen. Si la validación es correcta, se construyen las variables de volatilidad realizada. En ese momento se evalúa la disponibilidad de cada modelo y se muestra una tabla con los valores disponibles, los mínimos requeridos y los mensajes correspondientes. Esta tabla es relevante porque no todos los modelos necesitan la misma longitud de historia.

Al pulsar *Generar predicciones*, el MVP carga los modelos mediante `ModelRegistry` y ejecuta solo aquellos que están disponibles. La salida incluye una tabla con estado, predicción en escala logarítmica, volatilidad realizada transformada, raíz de volatilidad en porcentaje y nota técnica de cada modelo. La aplicación también representa el contexto reciente y permite descargar los resultados en `btc_volatility_predictions.csv`.

De forma opcional, el usuario puede calcular la evaluación *walk-forward* reciente. En ese caso, la aplicación crea targets futuros dentro de la ventana cargada solo para puntos donde ya son observables, ejecuta predicciones retrospectivas y calcula métricas por modelo. Los resultados pueden visualizarse en pestañas separadas y descargarse en `btc_volatility_walk_forward_predictions.csv`.

El flujo termina con la interpretación de resultados. La comparación histórica debe leerse en la página de modelos; la evaluación reciente debe entenderse como diagnóstico de la ventana actual. Esta separación evita confundir validación formal del proyecto con comportamiento local de una muestra corta cargada en la interfaz.

8.7– Limitaciones del MVP

El MVP tiene un alcance deliberadamente acotado. No predice si el precio de Bitcoin subirá o bajará, no calcula retornos esperados y no genera señales de compra o venta. La magnitud estimada es volatilidad realizada esperada a una hora. Por tanto, una predicción alta indica mayor intensidad esperada de movimiento, no una dirección de mercado.

La aplicación tampoco constituye asesoramiento financiero. No promete rentabilidad, no evalúa costes de transacción, no incorpora gestión de riesgo, no ejecuta operaciones y no selecciona posiciones. Además, la salida `sqrt_rv_percent` no está anualizada; se presenta solo como escala aproximada de la ventana de una hora.

Los modelos históricos no se reentrenan desde la web. AR(49), kNN y HAR-logRV global proceden de artefactos generados por el estudio técnico. Esta restricción es necesaria para preser-

var la trazabilidad entre validación histórica y uso operativo. La única excepción es Mini-HAR local, que se recalibra con la ventana cargada y se marca explícitamente como experimental.

La evaluación *walk-forward* reciente no sustituye la validación histórica formal. Su función es inspeccionar cómo se comportan los modelos sobre la ventana que el usuario ha cargado. El tamaño de esa ventana puede ser corto, contener gaps o pertenecer a un régimen concreto de mercado. Por ello, sus métricas deben interpretarse como diagnóstico local, no como estimación general del rendimiento esperado.

El MVP tampoco demuestra caos determinista en Bitcoin. La aplicación incorpora un modelo kNN asociado a la reconstrucción del espacio de estados y muestra figuras históricas del análisis no lineal, pero su función operativa es predecir volatilidad realizada. La evidencia sobre complejidad, no linealidad y caos pertenece al procedimiento experimental y se interpreta con cautela en los capítulos de resultados.

Finalmente, la aplicación depende de la disponibilidad y calidad de los datos cargados. La fuente Binance está limitada a las últimas velas solicitadas; los CSV subidos se reducen a las últimas 2000 filas; y los modelos con mayor memoria histórica pueden no estar disponibles si la ventana no contiene suficientes observaciones. Por este motivo, el diagnóstico de datos y la tabla de disponibilidad forman parte esencial de la interfaz, no un añadido secundario.

Manual de usuario

Este capítulo describe el uso del sistema desarrollado, tanto desde el punto de vista del procedimiento de análisis como desde el punto de vista del MVP web. No se trata de repetir la implementación interna, sino de indicar qué necesita el usuario para ejecutar el proyecto, cómo se ponen en marcha sus componentes principales y cómo deben interpretarse las salidas generadas.

El sistema tiene dos modos de uso diferenciados. El primero corresponde al procedimiento técnico completo, orientado a reproducir las fases de análisis, generar tablas, figuras, informes y artefactos. El segundo corresponde al MVP web, orientado a cargar una ventana de datos reciente o de ejemplo y obtener predicciones de volatilidad realizada a una hora. Ambos modos comparten variables, modelos y artefactos, pero no tienen el mismo alcance.

9.1– Objetivo de la herramienta

La herramienta desarrollada tiene como objetivo estimar la volatilidad realizada esperada de BTCUSDT durante la próxima hora. La frecuencia de trabajo es de cinco minutos, por lo que el horizonte operativo equivale a doce velas. La magnitud principal que se predice es `log_rv_future_12`, es decir, la volatilidad realizada futura de una hora en escala logarítmica.

La herramienta no predice la dirección del precio. Una predicción de volatilidad elevada no significa que Bitcoin vaya a subir ni que vaya a bajar, sino que el modelo estima una mayor intensidad de movimiento durante la próxima ventana. Esta distinción es esencial para usar correctamente el sistema: el resultado debe leerse como una predicción de variabilidad, no como una señal direccional de mercado.

El procedimiento completo tiene un objetivo más amplio que el MVP. Permite reproducir el análisis técnico realizado en el trabajo: validación de datos, construcción de variables, caracterización, estacionariedad, autocorrelación, recurrencia, filtrado lineal, contrastes de no linealidad, reconstrucción, cuantificación dinámica, predicción local, comprobación sintética con el mapa logístico y cierre HAR-logRV. El MVP, en cambio, expone solo la parte operativa necesaria para aplicar los modelos principales sobre una ventana cargada por el usuario.

Por tanto, la herramienta debe entenderse en dos niveles. Como procedimiento de investigación, sirve para auditar y reproducir la metodología. Como prototipo web, sirve para demostrar que los modelos finales pueden utilizarse de forma interactiva. En ninguno de los dos casos debe

interpretarse como asesoramiento financiero ni como sistema automático de trading.

9.2– Requisitos para la ejecución

La ejecución del proyecto requiere un entorno de Python con las dependencias indicadas en los ficheros de requisitos de cada repositorio. El MVP web se ejecuta desde su propio directorio mediante Streamlit y utiliza el fichero `requirements.txt`. Para una instalación local básica del MVP basta con situarse en el directorio de la aplicación y ejecutar:

```
pip install -r requirements.txt
streamlit run app.py
```

El procedimiento técnico completo requiere, además, disponer de los datos históricos utilizados en el análisis. Los ficheros brutos de Binance deben encontrarse en `data/raw/`, con nombres mensuales del tipo `BTCUSDT-5m-2024-01.zip`. A partir de ellos se genera el dataset limpio y posteriormente el dataset con variables de volatilidad realizada. Si el usuario solo quiere utilizar el MVP con el dataset de ejemplo o con datos recientes descargados desde Binance, no necesita reproducir todo el procedimiento histórico.

El MVP necesita una serie de artefactos locales para poder ejecutarse de forma autónoma. Estos artefactos se almacenan en `data/model_artifacts/` y son:

- `ar49_coefficients.csv`;
- `preprocessing_params.json`;
- `embedding_params.json`;
- `knn_reference_train.npz`;
- `historical_metrics.json`;
- `har_logrv_model.json`.

También debe existir un dataset de ejemplo en `data/sample/btcusdt_5m_recent_sample.csv`. Al iniciar la aplicación, `app.py` comprueba la existencia de estos ficheros. Si falta alguno, el MVP muestra un error y detiene la ejecución, porque no puede garantizar predicciones coherentes con el estudio técnico.

Para usar la fuente Binance API es necesaria conexión a Internet. La aplicación descarga las últimas velas disponibles de BTCUSDT con intervalo de cinco minutos. Para usar el dataset de ejemplo o un CSV local, la conexión no es imprescindible una vez instaladas las dependencias y presentes los artefactos.

9.3– Puesta en marcha del sistema

La puesta en marcha depende del modo de uso. Si se desea reproducir el estudio técnico, el primer paso es preparar los datos brutos. Si se quiere replicar el estudio técnico con los mismos datos, los ficheros mensuales de Binance deben copiarse en `data/raw/`, para obtener datos diferentes basta con descargar los archivos `.zip` desde la api de biance desde el link

(<https://data.binance.vision/?prefix=data/spot/monthly/klines/BTCUSDT/5m/>). A continuación se ejecuta el script de construcción del dataset limpio, que lee los ZIP, normaliza las columnas y genera `btc_5m_clean.csv`. Sobre este fichero se ejecutan después las fases de construcción de variables y análisis.

Si el objetivo es utilizar el MVP, el usuario debe entrar en el directorio de `mvp_web`, instalar dependencias y lanzar la aplicación:

```
pip install -r requirements.txt
streamlit run app.py
```

Al abrirse la interfaz, la página de inicio muestra el resumen del prototipo: símbolo, frecuencia, horizonte y modelos disponibles. También verifica los artefactos locales. Si la verificación es correcta, la aplicación informa de que puede arrancar sin depender del repositorio técnico en tiempo de ejecución. Si la verificación falla, se deben regenerar los artefactos desde el estudio técnico o copiar los ficheros requeridos al directorio correspondiente.

Cuando se han modificado modelos, métricas históricas o artefactos en el repositorio técnico, debe repetirse la exportación al MVP. Desde el directorio del MVP puede ejecutarse el script de exportación indicando, si es necesario, la ruta al repositorio técnico:

```
python scripts/export_model_artifacts.py
python scripts/validate_artifacts.py
```

El primer comando copia y genera los artefactos necesarios para la aplicación. El segundo comprueba que los ficheros resultantes son válidos, que los parámetros esperados coinciden y que el MVP no mantiene dependencias de ejecución hacia el repositorio técnico.

9.4– Ejecución del procedimiento de análisis

El procedimiento de análisis debe ejecutarse en orden. Las fases iniciales producen ficheros que son necesarios para las fases posteriores. Saltarse una fase puede dejar artefactos obsoletos o incoherentes. En particular, no debe ejecutarse la predicción local sin haber validado antes los datos, construido las variables, estandarizado la serie y fijado los parámetros de reconstrucción.

El flujo comienza con la validación integral de datos. A partir de los ficheros mensuales de Binance, el script de limpieza genera `btc_5m_clean.csv`. Esta fase comprueba columnas, rango temporal, frecuencia de cinco minutos, duplicados, huecos, nulos y precios positivos. El resultado es la base empírica del resto del análisis.

Después se construyen las variables de volatilidad realizada. El dataset limpio se transforma en `btc_5m_features.csv`. Este fichero contiene retornos logarítmicos, retornos al cuadrado, volatilidades realizadas pasadas, volatilidades realizadas futuras y transformaciones logarítmicas. La variable principal es `log_rv_past_12`, y el target predictivo es `log_rv_future_12`.

Las fases de caracterización deben ejecutarse antes de reconstruir el espacio de estados. En ellas se generan gráficos temporales, estadísticos descriptivos, pruebas de estacionariedad, ACF, PACF, periodogramas y gráficos de recurrencia. Estas salidas permiten comprobar que la volatilidad realizada presenta persistencia temporal y que la serie seleccionada es más adecuada para el análisis que el precio o el retorno firmado.

A continuación se ejecuta el filtrado lineal AR(49), los contrastes de no linealidad y la reconstrucción del espacio de estados. La reconstrucción utiliza $\tau = 137$ y $m = 5$, con la convención causal $X_t = [z_t, z_{t-\tau}, \dots, z_{t-(m-1)\tau}]$. A partir de este embedding se calculan indicadores dinámicos, se comparan controles barajados y subrogados, y se evalúa la predicción local kNN.

Las últimas fases del procedimiento son la robustez de parámetros, la comprobación sintética con mapa logístico y el modelo HAR-logRV. La robustez comprueba configuraciones alternativas de k y m . La comprobación sintética valida el comportamiento del pipeline sobre una dinámica caótica conocida. HAR-logRV proporciona el modelo práctico exportable para el MVP.

Una ejecución completa genera tres tipos de salida. Las tablas se guardan en CSV o JSON, las figuras en SVG o PNG, y los artefactos de modelos en CSV, JSON o NPZ. Estos ficheros son los que alimentan la memoria y, en el caso de los modelos finales, también el MVP web.

9.5– Uso del MVP web

El uso del MVP se realiza desde la interfaz Streamlit. Tras lanzar `streamlit run app.py`, el usuario accede a una aplicación con cinco páginas: Inicio, Predicción de volatilidad, Diagnóstico de datos, Comparación de modelos y Ayuda y limitaciones.

La página de inicio sirve para comprobar que la aplicación está lista. Muestra el símbolo BTCUSDT, la frecuencia 5m, el horizonte de doce velas y la disponibilidad de los artefactos locales. Si los artefactos son correctos, el usuario puede pasar a la página de predicción.

En la página *Predicción de volatilidad* se elige la fuente de datos. Hay tres opciones:

- *Binance API*: descarga las últimas velas de BTCUSDT con intervalo de cinco minutos.
- *Dataset de ejemplo*: carga el fichero local incluido con el MVP.
- *Subir CSV*: permite cargar un archivo del usuario con columnas temporales y precio de cierre.

El CSV mínimo debe contener una columna temporal y una columna `close`. La columna temporal puede llamarse `timestamp`, `open_time`, `date`, `datetime` o `time`. Un ejemplo válido es:

```
timestamp,close
2026-04-30 19:45:00+00:00,76442.71
2026-04-30 19:50:00+00:00,76415.00
```

Tras cargar datos, la aplicación muestra un resumen de filas, rango temporal, primeras y últimas observaciones. Después valida la ventana y, si no hay errores bloqueantes, construye las features de volatilidad realizada. En ese momento se muestra la disponibilidad de modelos. Esta tabla es importante: un modelo puede no estar disponible si la ventana no tiene suficiente historia.

Al pulsar *Generar predicciones*, el MVP ejecuta los modelos disponibles. La tabla de salida incluye persistencia, kNN local, AR(49), HAR-logRV global y Mini-HAR local. Para cada modelo se muestra el estado, la predicción en escala logarítmica, la volatilidad realizada transformada, la raíz de volatilidad en porcentaje y una nota de interpretación. El modelo HAR-logRV global aparece como modelo práctico recomendado.

La misma página permite calcular una evaluación *walk-forward* reciente. Esta opción genera predicciones sobre puntos pasados de la ventana cargada y las compara con la volatilidad realizada observada una hora después. El usuario puede elegir el número máximo de puntos evaluados por modelo y descargar los resultados en CSV. Esta evaluación debe leerse como diagnóstico local de la ventana cargada.

La página *Diagnóstico de datos* resume la fiabilidad de la ventana. Muestra un score entre 0 y 100, estado cualitativo, frecuencia detectada, gaps, rango temporal, precios mínimos y máximos, y disponibilidad de modelos. La página *Comparación de modelos* presenta métricas históricas formales y figuras exportadas del estudio técnico. La página *Ayuda y limitaciones* explica qué predice la aplicación, qué no predice, cómo debe formatearse el CSV y qué restricciones tiene el MVP.

9.6– Interpretación básica de las salidas

La salida principal del MVP es `log_rv_future_12`. Esta variable está en escala logarítmica y corresponde a la volatilidad realizada de las próximas doce velas. Su lectura directa no es tan intuitiva como una magnitud en porcentaje, pero es la escala usada durante el entrenamiento, la validación y la comparación histórica de los modelos.

La columna `rv_future_12` es la transformación inversa de la predicción logarítmica. Representa la volatilidad realizada esperada como suma de retornos al cuadrado durante la próxima hora. La columna `sqrt_rv_percent` toma la raíz de esa volatilidad realizada y la expresa en porcentaje. Esta última magnitud facilita una lectura aproximada de la intensidad esperada de movimiento en la ventana de una hora, pero no está anualizada.

El horizonte temporal se muestra siempre en UTC. La última vela utilizada marca el punto de partida de la predicción. El inicio del horizonte coincide con esa referencia temporal y el final se obtiene sumando sesenta minutos. Esta información es necesaria para no confundir la predicción con una estimación sin fecha ni intervalo.

Los modelos tienen papeles distintos. Persistencia es una referencia básica: asume que la volatilidad futura será igual a la volatilidad realizada actual. AR(49) es un benchmark lineal fuerte. kNN local es el modelo experimental asociado al espacio reconstruido. HAR-logRV global es el modelo práctico recomendado porque combina rendimiento histórico competitivo, baja complejidad e interpretabilidad. Mini-HAR local es una recalibración experimental con la ventana cargada.

Las métricas *walk-forward* recientes deben interpretarse con cautela. RMSE y MAE resumen el error sobre puntos pasados de la ventana cargada; el sesgo indica si el modelo tiende a sobrestimar o infraestimar en esa muestra. Sin embargo, una ventana reciente puede ser corta o pertenecer a un régimen concreto. Por ello, estas métricas no sustituyen la validación histórica formal.

La página de comparación contiene la referencia histórica del proyecto. Allí se muestran métricas y figuras exportadas desde el análisis técnico. Esa validación es la que justifica la recomendación de HAR-logRV global como modelo práctico. La evaluación reciente ayuda a inspeccionar la ventana cargada, pero no debe usarse para cambiar las conclusiones generales del estudio.

Por último, ninguna salida debe interpretarse como demostración de caos determinista en Bitcoin. La aplicación incorpora un modelo kNN basado en reconstrucción del espacio de estados y muestra figuras relacionadas con la metodología no lineal, pero el objetivo operativo del MVP es predecir volatilidad realizada. La discusión sobre estructura dinámica pertenece al análisis experimental y se interpreta con prudencia.

9.7– Generación de figuras, tablas e informes

Las figuras, tablas e informes del trabajo se generan desde el procedimiento técnico, no desde el MVP. Cada fase del análisis guarda sus salidas en directorios específicos. Las tablas se almacenan principalmente en `reports/tables/`, las figuras en `reports/figures/` y los informes o resúmenes de fase en `reports/`. Esta organización permite que los resultados incluidos en la memoria sean trazables.

Las fases iniciales generan tablas de validación de datos, resúmenes de forma del dataset, estadísticos descriptivos y eventos extremos. Las fases de caracterización producen figuras temporales, ACF, PACF, periodogramas y gráficos de recurrencia. Las fases de reconstrucción y cuantificación generan parámetros de embedding, visualizaciones 2D/3D, estimaciones dinámicas y comparaciones con controles. Las fases predictivas generan métricas de validación y test, tablas de predicciones y figuras de real frente a predicho.

La comprobación sintética con el mapa logístico genera sus propias salidas, separadas de las de Bitcoin. Entre ellas se incluyen series sintéticas, parámetros de reconstrucción, métricas dinámicas y comparación predictiva. Esta separación evita mezclar resultados sintéticos con resultados financieros.

La Fase 14 genera los artefactos finales del modelo HAR-logRV y las figuras de comparación AR/kNN/HAR. Parte de esas figuras se copian al directorio `assets/historical_validation/` del MVP para ser mostradas en la página de comparación de modelos. El MVP no recalcula esas figuras; solo las presenta como contexto histórico.

Para actualizar el MVP después de repetir el análisis técnico, debe ejecutarse el proceso de exportación de artefactos:

```
python scripts/export_model_artifacts.py
python scripts/validate_artifacts.py
```

El primer script copia modelos, métricas, figuras y dataset de ejemplo desde el repositorio técnico. El segundo verifica que los artefactos exportados son consistentes y que la aplicación puede ejecutarse sin depender de rutas absolutas ni del código de investigación en tiempo de uso.

En consecuencia, la generación de informes sigue una regla simple: el repositorio técnico produce resultados; el MVP consume una selección de esos resultados; la memoria los interpreta. Mantener esta separación evita duplicidades y facilita que cualquier cifra, figura o artefacto pueda rastrearse hasta la fase que lo generó.

Conclusiones y desarrollos futuros

10.1– Conclusiones generales

10.2– Conclusiones sobre la metodología desarrollada

10.3– Conclusiones sobre el mapa logístico

10.4– Conclusiones sobre la aplicación a Bitcoin

10.5– Conclusiones sobre la utilidad predictiva

10.6– Conclusiones sobre el MVP web

10.7– Limitaciones finales del trabajo

10.8– Desarrollos futuros

Glosario de conceptos técnicos

Este anexo recoge los términos financieros, metodológicos y de implementación que aparecen de manera recurrente en la memoria. Su finalidad es servir como referencia rápida para el lector; las explicaciones completas se desarrollan en los capítulos principales.

Activo financiero: Instrumento negociable que representa valor económico. En este trabajo el activo estudiado es el par BTCUSDT, utilizado como aproximación al precio de Bitcoin frente a una moneda estable referenciada al dólar.

Klines o velas japonesas: Estructura de datos financiera que resume el movimiento de precio de un activo durante un intervalo temporal fijo. Cada vela incluye precio de apertura, máximo, mínimo, cierre, volumen y número de operaciones. En este trabajo se emplean velas de cinco minutos.

Mercado *spot*: Mercado en el que la compraventa de un activo se liquida de forma inmediata al precio de cotización disponible, a diferencia de los mercados de futuros o derivados.

Precio de cierre: Precio final registrado dentro de una vela. Es la magnitud empleada para construir el log-precio, los retornos y las variables de volatilidad realizada.

Retorno logarítmico: Variación del logaritmo del precio entre dos instantes consecutivos. Si P_t es el precio de cierre, se calcula como

$$r_t = \log(P_t) - \log(P_{t-1}). \quad (\text{A.1})$$

Su uso es habitual en series financieras porque permite agregar retornos en el tiempo de forma sencilla.

Volatilidad realizada: Medida empírica de la variabilidad de un activo calculada a partir de retornos de alta frecuencia. En este trabajo se define mediante la suma de retornos logarítmicos al cuadrado dentro de una ventana temporal.

Volatilidad realizada pasada: Variable construida con información disponible hasta el instante t . Por ejemplo, `rv_past_12` resume la variabilidad observada durante las últimas 12 velas, equivalentes a una hora.

Volatilidad realizada futura: Variable objetivo construida con observaciones posteriores al instante t . Por ejemplo, `rv_future_12` resume la volatilidad de las 12 velas siguientes y se utiliza para entrenar y evaluar los modelos predictivos.

Log-volatilidad realizada: Transformación logarítmica de la volatilidad realizada. Reduce la asimetría y facilita el tratamiento estadístico de una magnitud positiva y muy variable.

Serie principal: Serie temporal sobre la que se centra el análisis dinámico. En este trabajo es `log_rv_past_12`, por representar la volatilidad realizada intradía de una hora.

Variable explicativa: Variable disponible en el instante de predicción. Debe construirse únicamente con información pasada o presente para evitar sesgo de anticipación.

Variable objetivo: Magnitud futura que se desea estimar. En este trabajo la variable objetivo principal es `log_rv_future_12`.

Horizonte de predicción: Número de pasos temporales hacia el futuro que se desea estimar. El horizonte principal del trabajo es de 12 velas de cinco minutos, es decir, una hora.

Retardo: Desplazamiento temporal entre una observación y otra anterior. El valor x_{t-k} representa la observación situada k pasos antes de x_t .

Frecuencia modal: Intervalo temporal que aparece con mayor frecuencia entre observaciones consecutivas. En los datos utilizados, la frecuencia modal estricta es de cinco minutos.

Sesgo de anticipación o *look-ahead bias*: Error metodológico que aparece cuando se usa información futura para construir variables, ajustar modelos o evaluar resultados. En una predicción real, esa información no estaría disponible.

Conjunto de entrenamiento: Parte inicial de la serie utilizada para ajustar modelos y estimar parámetros.

Conjunto de validación: Parte posterior al entrenamiento utilizada para seleccionar hiperparámetros o configuraciones de modelo sin tocar todavía el conjunto final de prueba.

Conjunto de prueba: Parte final de la serie reservada para evaluar el rendimiento definitivo de los modelos.

Persistencia: Modelo de referencia que utiliza el último valor disponible como predicción del valor futuro. Es una referencia natural en series con alta dependencia temporal.

Modelo autorregresivo AR(p): Modelo lineal que aproxima el valor de una serie mediante una combinación de sus p valores anteriores. En este trabajo el modelo AR(49) actúa como referencia lineal fuerte.

HAR-logRV: Modelo de regresión para volatilidad realizada que combina información de varias escalas temporales. En este trabajo usa `log_rv_past_12`, `log_rv_past_48` y `log_rv_past_288` para representar memoria de una hora, cuatro horas y un día.

Reconstrucción por retardos: Técnica que transforma una serie escalar en vectores formados por observaciones retardadas. Permite representar la evolución de la serie en un espacio de estados reconstruido.

Embedding: Representación vectorial obtenida mediante reconstrucción por retardos. En este trabajo se usa principalmente $\tau = 137$ y $m = 5$, de modo que cada vector recoge valores separados por 137 velas.

Información mutua media: Medida de dependencia no necesariamente lineal entre una serie y una versión retardada de sí misma. Se utiliza para seleccionar el retardo τ en la reconstrucción.

Falsos vecinos cercanos: Método para seleccionar la dimensión de embedding. Evalúa cuántos vecinos próximos en baja dimensión dejan de serlo al aumentar la dimensión.

Método de Cao: Procedimiento alternativo para estudiar la dimensión de embedding mediante la evolución de cocientes de distancias al aumentar la dimensión.

Ventana de Theiler: Separación temporal mínima impuesta al buscar vecinos en el espacio reconstruido. Evita considerar como vecinos útiles observaciones artificialmente próximas por continuidad temporal.

Dimensión de correlación: Indicador de complejidad geométrica estimado a partir de la relación entre el radio de vecindad y el número de pares de puntos cercanos en el espacio reconstruido.

Exponente de Lyapunov: Medida de divergencia local entre trayectorias inicialmente próximas. En este trabajo se estima de forma aproximada mediante el método de Rosenstein.

Entropía de permutación: Medida de irregularidad ordinal de una serie temporal. Valores cercanos a uno indican comportamiento altamente irregular en los patrones ordinales considerados.

Serie barajada: Versión permutada aleatoriamente de una serie temporal. Conserva la distribución marginal de los valores, pero destruye el orden temporal.

Datos subrogados: Series artificiales generadas para conservar ciertas propiedades de la serie original y destruir otras. Se utilizan como controles para interpretar si un estadístico depende de la distribución, de la estructura lineal o de una posible estructura no lineal adicional.

Phase randomization: Técnica de generación de datos subrogados que conserva aproximadamente el espectro de la serie, pero altera las relaciones de fase.

AAFT: Técnica de generación de subrogados que intenta conservar tanto la distribución marginal como la estructura lineal aproximada de la serie.

Mapa logístico: Sistema dinámico discreto definido por $x_{t+1} = rx_t(1 - x_t)$. En este trabajo se utiliza con $r = 4$ como sistema sintético de control con dinámica caótica conocida.

Walk-forward: Evaluación secuencial en la que las predicciones se realizan respetando el orden temporal. En cada instante solo se utiliza información disponible hasta ese momento.

MVP: Producto mínimo viable. En este trabajo designa la aplicación web Streamlit que permite cargar datos, calcular variables, ejecutar modelos y visualizar predicciones de volatilidad.

Artefacto: Producto tangible generado por el pipeline de análisis o por el MVP, como ficheros CSV, JSON, NPZ, SVG o datasets procesados.

Uso de la inteligencia artificial

Para la implementación del MVP se ha usado un agente de código, Codex, para generar fragmentos de código a partir de descripciones en lenguaje natural. Este enfoque ha permitido acelerar la implementación de funcionalidades específicas, que no formaban parte del núcleo metodológico del trabajo, como la construcción de la interfaz gráfica, la gestión de archivos o la integración de modelos predictivos en la aplicación web.

En este trabajo se ha utilizado inteligencia artificial de manera puntual para mejorar la redacción, la organización de ideas y la corrección de errores. Se han empleado herramientas de lenguaje natural para revisar y optimizar la redacción de algunos párrafos, así como para generar resúmenes o explicaciones adicionales cuando se ha considerado útil.

Referencias

- Abarbanel, H. D. I. (1996). *Analysis of observed chaotic data*. New York: Springer. doi: 10.1007/978-1-4612-0763-4
- Andersen, T. G., Bollerslev, T., Diebold, F. X., y Labys, P. (2003). Modeling and forecasting realized volatility. *Econometrica*, 71(2), 579–625. doi: 10.1111/1468-0262.00418
- Bandt, C., y Pompe, B. (2002). Permutation entropy: A natural complexity measure for time series. *Physical Review Letters*, 88(17), 174102. doi: 10.1103/PhysRevLett.88.174102
- Barndorff-Nielsen, O. E., y Shephard, N. (2002). Econometric analysis of realized volatility and its use in estimating stochastic volatility models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 64(2), 253–280. doi: 10.1111/1467-9868.00336
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., y Ljung, G. M. (2015). *Time series analysis: Forecasting and control* (5.^a ed.). Hoboken, NJ: Wiley.
- Brock, W. A., Dechert, W. D., Scheinkman, J. A., y LeBaron, B. (1996). A test for independence based on the correlation dimension. *Econometric Reviews*, 15(3), 197–235. doi: 10.1080/07474939608800353
- Brockwell, P. J., y Davis, R. A. (2002). *Introduction to time series and forecasting* (2.^a ed.). New York: Springer. doi: 10.1007/b97391
- Cao, L. (1997). Practical method for determining the minimum embedding dimension of a scalar time series. *Physica D: Nonlinear Phenomena*, 110(1–2), 43–50. doi: 10.1016/S0167-2789(97)00118-8
- Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2), 174–196. doi: 10.1093/jjfinec/nbp001
- Dickey, D. A., y Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366), 427–431. doi: 10.1080/01621459.1979.10482531
- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, 50(4), 987–1007. doi: 10.2307/1912773
- Farmer, J. D., y Sidorowich, J. J. (1987). Predicting chaotic time series. *Physical Review Letters*, 59(8), 845–848. doi: 10.1103/PhysRevLett.59.845
- Fraser, A. M., y Swinney, H. L. (1986). Independent coordinates for strange attractors from mutual information. *Physical Review A*, 33(2), 1134–1140. doi: 10.1103/PhysRevA.33.1134
- Grassberger, P., y Procaccia, I. (1983). Measuring the strangeness of strange attractors. *Physica D: Nonlinear Phenomena*, 9(1–2), 189–208. doi: 10.1016/0167-2789(83)90298-1
- Kantz, H., y Schreiber, T. (2003). *Nonlinear time series analysis* (2.^a ed.). Cambridge University Press. doi: 10.1017/CBO9780511755798
- Kennel, M. B., Brown, R., y Abarbanel, H. D. I. (1992). Determining embedding dimension for phase-space reconstruction using a geometrical construction. *Physical Review A*, 45(6), 3403–3411. doi: 10.1103/PhysRevA.45.3403
- Kwiatkowski, D., Phillips, P. C. B., Schmidt, P., y Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54(1–3), 159–

178. doi: 10.1016/0304-4076(92)90104-Y
- Ljung, G. M., y Box, G. E. P. (1978). On a measure of lack of fit in time series models. *Biometrika*, 65(2), 297–303. doi: 10.1093/biomet/65.2.297
- May, R. M. (1976). Simple mathematical models with very complicated dynamics. *Nature*, 261(5560), 459–467. doi: 10.1038/261459a0
- Müller, U. A., Dacorogna, M. M., Davé, R. D., Olsen, R. B., Pictet, O. V., y von Weizsäcker, J. E. (1997). Volatilities of different time resolutions: Analyzing the dynamics of market components. *Journal of Empirical Finance*, 4(2–3), 213–239. doi: 10.1016/S0927-5398(97)00007-8
- Olmedo Fernández, E. (2001). *Análisis de series temporales en el contexto de la economía dinámica no lineal y caótica. aplicación a datos de la economía española* (Tesis Doctoral no publicada). Universidad de Sevilla, Sevilla.
- Priestley, M. B. (1981). *Spectral analysis and time series*. London: Academic Press.
- Rosenstein, M. T., Collins, J. J., y De Luca, C. J. (1993). A practical method for calculating largest lyapunov exponents from small data sets. *Physica D: Nonlinear Phenomena*, 65(1–2), 117–134. doi: 10.1016/0167-2789(93)90009-P
- Schreiber, T., y Schmitz, A. (2000). Surrogate time series. *Physica D: Nonlinear Phenomena*, 142(3–4), 346–382. doi: 10.1016/S0167-2789(00)00043-9
- Takens, F. (1981). Detecting strange attractors in turbulence. En D. Rand y L.-S. Young (Eds.), *Dynamical systems and turbulence, warwick 1980* (Vol. 898, pp. 366–381). Berlin, Heidelberg: Springer. doi: 10.1007/BFb0091924
- Theiler, J. (1986). Spurious dimension from correlation algorithms applied to limited time-series data. *Physical Review A*, 34(3), 2427–2432. doi: 10.1103/PhysRevA.34.2427
- Theiler, J., Eubank, S., Longtin, A., Galdrikian, B., y Farmer, J. D. (1992). Testing for nonlinearity in time series: The method of surrogate data. *Physica D: Nonlinear Phenomena*, 58(1–4), 77–94. doi: 10.1016/0167-2789(92)90102-S